



Requirement & Design Specification

Football Field Booking System

Version: 1.0

– Can Tho, July 2026–

Record of Changes

Version	Date	A* M, D	In charge	Change Description
V1.0	24/07/26	A	BonVT	Added Account & Customer Status requirements and design specifications.
V1.0	24/07/26	A	NgocPA	Added Booking Lifecycle requirements and design specifications.
V1.0	24/07/26	A	BaoNG	Added Field, Slot, Service & Issue Management requirements and design specifications.
V1.0	24/07/26	A	AnNP	Added Payment Gateway, Refund & Invoice requirements and design specifications.
V1.0	24/07/26	A	AnPTT	Added Promotion, Membership, Notification & Report requirements and design specifications.

**A - Added M - Modified D - Deleted*

Contents

<i>Record of Changes</i>	2
<i>I. Overview</i>	5
1. <i>User Requirements</i>	5
1.2 <i>Use Cases</i>	6
2. <i>Overall Functionalities</i>	16
2.1 <i>Screens Flow</i>	16
<i>Guest's Screen Flow</i>	16
2.2 <i>Screen Descriptions</i>	20
2.3 <i>Screen Authorization</i>	23
2.4 <i>Non-UI Functions</i>	24
3. <i>System High Level Design</i>	25
3.1 <i>Database Design</i>	25
3.2 <i>Code Packages</i>	29
<i>II. Requirement Specifications</i>	30
<i>Account & Customer Status</i>	30
<i>Field, Slot, Service & Issue Management</i>	38
<i>Booking Lifecycle</i>	51
<i>Payment Gateway, Refund & Invoice</i>	65
<i>Promotion, Membership, Notification & Report</i>	78
<i>III. Design Specifications</i>	90
1. <i>Account & Customer Status</i>	90
1.1 <i>Screen Designs</i>	90
2. <i>Field, Slot, Service & Issue Management</i>	97
2.1 <i>Screen Designs</i>	97
3. <i>Booking Lifecycle</i>	106
3.1 <i>Screen Designs</i>	106
4. <i>Payment Gateway, Refund & Invoice</i>	113
4.1 <i>Screen Designs</i>	113
5. <i>Promotion, Membership, Notification & Report</i>	119
5.1 <i>Screen Designs</i>	119
<i>IV. Appendix</i>	129
1. <i>Assumptions & Dependencies</i>	129
2. <i>Limitations & Exclusions</i>	130
3. <i>Business Rules</i>	130
4. <i>Booking Status Values</i>	132

5. Code-First Alignment Notes..... 132

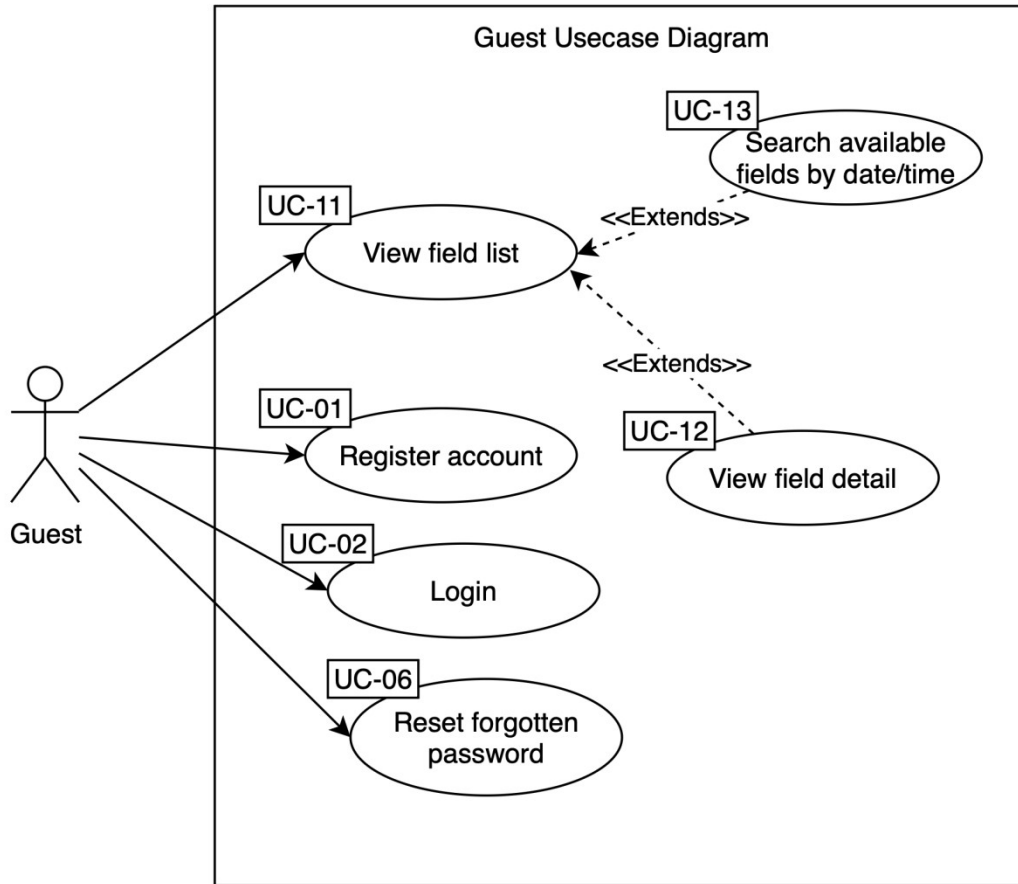
I. Overview

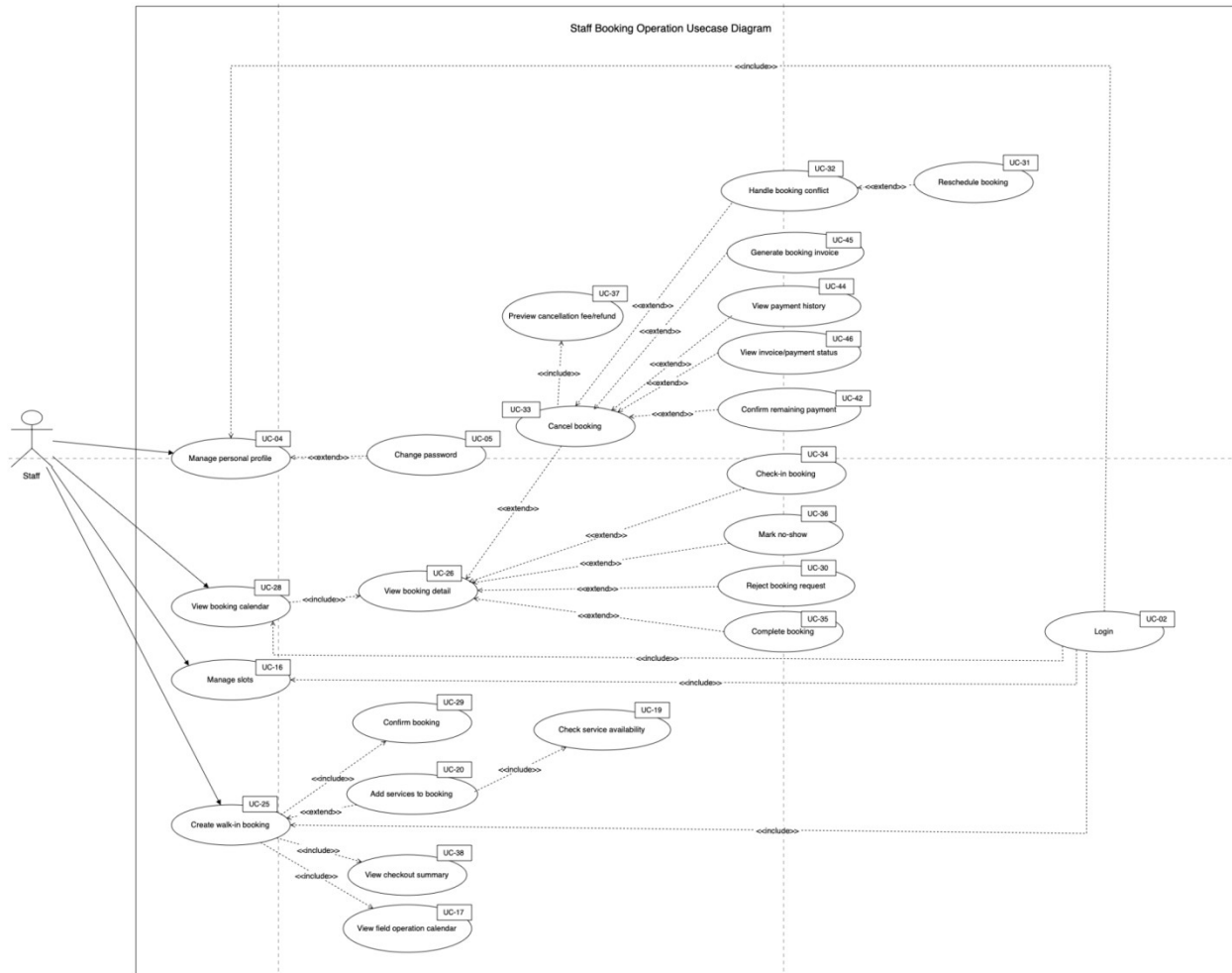
1. User Requirements

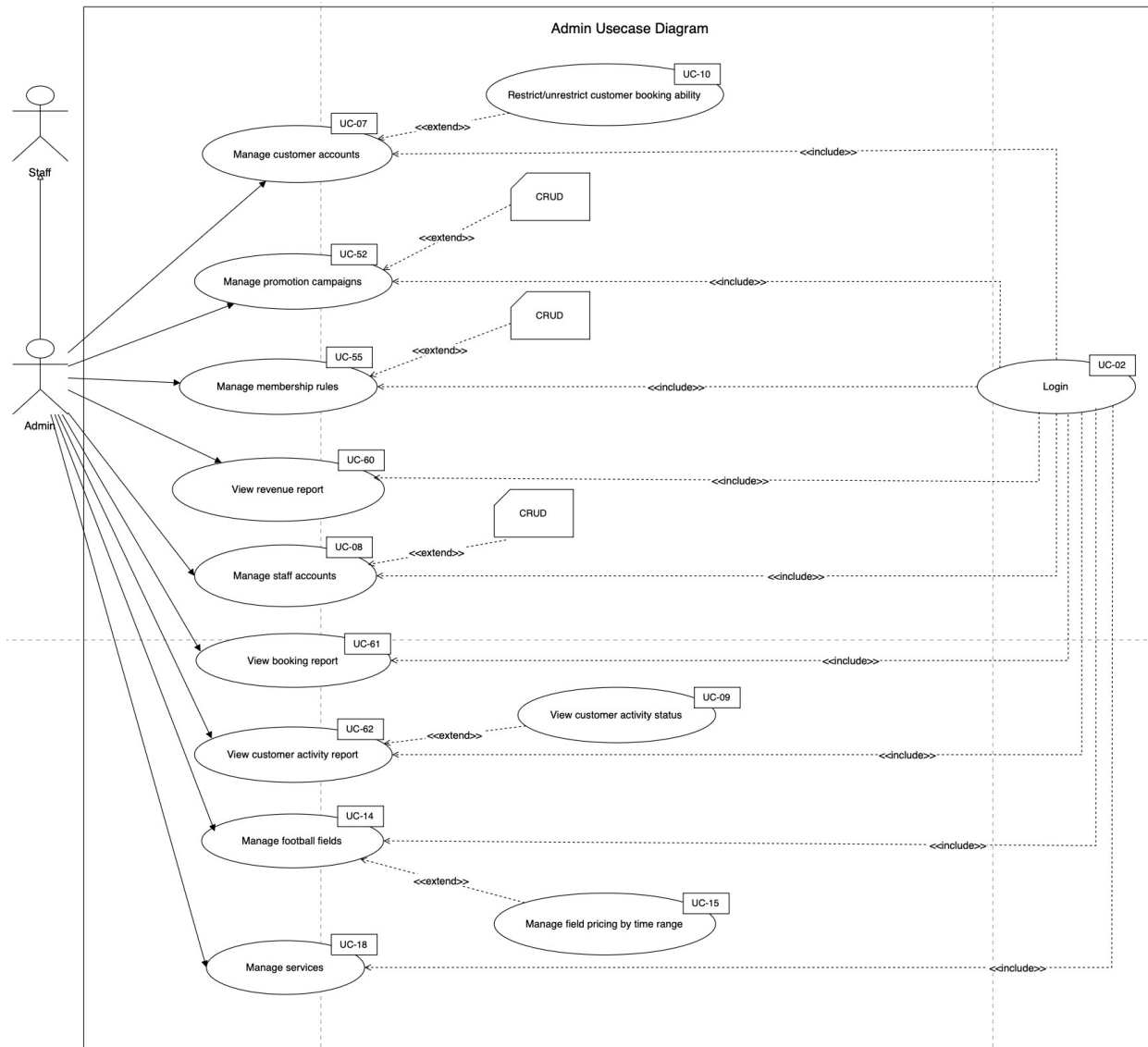
#	Actor	Description
1	Guest	Unauthenticated visitor who can view public information and register/login.
2	Customer	Registered user who searches fields, creates bookings, pays, manages own bookings, views membership and reports issues.
3	Staff	Venue operation employee who handles calendar, walk-in bookings, check-in, completion, no-show, issues and refunds.
4	Admin	System administrator/owner who manages accounts, fields, prices, services, promotions, membership rules and reports.
5	System	Internal automated actor for payment capture, invoice generation, notification and scheduled reminders.

1.2 Use Cases

a. Usecase Diagrams







b. Descriptions

Account & Customer Status

ID	Feature	Use Case	Use Case Description
UC-01	Register account	Register account	A guest registers a Customer account, receives an authenticated session, and is asked to verify the registered email address.
UC-02	Login	Login	A registered user signs in with email or phone and receives a role-aware authenticated session.
UC-03	Logout	Logout	An authenticated user signs out; the server revokes the current token version and the client clears local session data.
UC-04	Manage personal profile	Manage personal profile	An authenticated user updates only their own allowed profile fields.
UC-05	Change password	Change password	An authenticated user changes the password after confirming the current password; existing

			<i>tokens are revoked.</i>
UC-06	<i>Forgot/reset password</i>	<i>Forgot/reset password</i>	<i>A user requests a time-limited reset link and sets a new password without exposing whether an email is registered.</i>
UC-07	<i>Manage customer accounts</i>	<i>View and lock/unlock customer accounts</i>	<i>An Admin views the Customer directory and read-only activity details, then locks or unlocks sign-in access when required. Profile editing is not exposed.</i>
UC-08	<i>Manage staff accounts</i>	<i>Manage staff accounts</i>	<i>An Admin creates and maintains Staff identity/status. The Staff member receives a one-hour email invitation and privately chooses the password; Admin cannot set or replace it.</i>
UC-09	<i>View customer activity status</i>	<i>View customer activity status</i>	<i>Admin reviews a Customer's recent bookings, completed-booking count, and reported issues.</i>

Field, Slot, Service & Issue Management

ID	Feature	Use Case	Use Case Description
UC-10	<i>View field list</i>	<i>View field list</i>	<i>Display the public catalogue of active football fields. The implemented list includes field name, type, capacity, description, image, location, surface and active status; pricing is available only after opening field detail.</i>
UC-11	<i>View field detail</i>	<i>View field detail</i>	<i>Show the selected field's catalogue attributes, active price bands and price range, globally active add-ons, and up to five available slots from today through the next seven days. Reviews are explicitly not modelled.</i>
UC-12	<i>Search available fields by date/time</i>	<i>Search available fields by date/time</i>	<i>Generate missing bookable slots from active Admin rules, then search the requested date and optional field type, calculate current field price, and label each row as available, booked, or blocked.</i>
UC-13	<i>Manage football fields</i>	<i>Manage football fields</i>	<i>Create or update football fields and activate/deactivate them. Deactivation removes a field from public list and search results; deletion is not implemented.</i>
UC-14	<i>Manage field pricing by time range</i>	<i>Manage field pricing by time range</i>	<i>Create and update price rows by field, day type and time range, with optional effective dates and active/inactive status.</i>
UC-15	<i>Manage unavailable slots</i>	<i>Manage unavailable slots</i>	<i>Admin configures opening time, closing time, slot duration, and rolling horizon. Staff preserves maintenance, private-event, VIP/internal-use, and other exceptions through Block/Unblock.</i>
UC-16	<i>View field operation calendar</i>	<i>View field operation calendar</i>	<i>Ensure the selected date is materialized from Admin slot rules, then display every field slot with available, blocked, or booking-derived operational status.</i>

UC-17	Manage extra services	Manage extra services	Create, update, activate and deactivate rental, sale or staff-supported add-ons with pricing and optional stock/per-booking limits.
UC-18	Check extra service availability	Check extra service availability	Validate that each selected service is active, within its per-booking maximum, and within shared stock after counting quantities reserved by overlapping active bookings.
UC-19	Add extra services to booking	Add extra services to booking	Include selected active add-ons in online customer checkout or Staff walk-in checkout, snapshot unit price/quantity/line total, and include the service total in booking and invoice calculations.
UC-20	Update extra services before check-in	Update extra services before check-in	Replace all booking-service selections while the booking is pending or confirmed, then recalculate services, promotion, membership discount, total, deposit, remaining/refundable delta and invoice.
UC-21	Report field/service issue	Report field/service issue	Create an issue under the authenticated reporter, optionally linked to a booking, field and/or extra service. Operators may assign the issue to a Staff account at creation.
UC-22	Resolve field/service issue	Resolve field/service issue	Update issue state, optionally assign it to a Staff account, require a resolution note when closing, record/clear resolved time, and notify the reporter on every update.

Booking Lifecycle

ID	Feature	Use Case	Use Case Description
UC-23	Create online booking	Create online booking	Create a Customer-owned online booking as a pending slot hold, snapshot pricing/services/discounts/deposit, then continue to customer-only PayPal deposit/full capture, which confirms the booking.
UC-24	Create walk-in booking	Create walk-in booking	Create a walk-in booking for an existing customer as immediately confirmed, without membership discount, and normally record a cash deposit or full payment immediately afterward.
UC-25	View booking detail	View booking detail	Return role/ownership-protected booking summary plus add-ons, payments, promotions, invoice, refunds and derived payment status.
UC-26	View my bookings	View my bookings	List the authenticated Customer's bookings in descending booking ID order and allow selection for detail, billing, cancellation, reschedule, add-on and issue actions.
UC-27	View booking calendar	View booking calendar	Support daily booking monitoring through the Staff booking list and the date-based operation calendar, which presents booking status alongside each slot.

UC-28	Confirm booking	Confirm booking	Move a pending booking to confirmed after qualifying payment or operator approval and record confirmedAt. Walk-ins are already confirmed at creation.
UC-29	Reject booking request	Reject booking request	Change a pending booking to rejected, optionally store an operator/note, notify the customer and release the slot from active reservation.
UC-30	Reschedule booking	Reschedule booking	Move a pending or confirmed booking to a different slot while preserving its booking/payment history, repricing the field and discounts, calculating remaining/refundable delta, updating the invoice and notifying the customer.
UC-31	Preview cancellation fee/refund	Preview cancellation fee/refund	Calculate, without mutating the booking, hours before start, policy band, refund percentage, fee and refundable amount based on the amount already paid.
UC-32	Cancel booking	Cancel booking	Cancel a pending or confirmed booking, snapshot fee/refundable amount using the current policy, notify the customer and release the slot. Refund processing remains a separate request/review flow.
UC-33	Check-in booking	Check-in booking	Move a confirmed booking to checked_in and record checkedInAt and the operating staff.
UC-34	Complete booking	Complete booking	Move a checked-in booking to completed, record completedAt, re-evaluate membership under configured qualification periods, and generate/update the invoice.
UC-35	Mark no-show	Mark no-show	Move a confirmed booking to terminal no_show state and record the operating staff/note. Existing paid money remains on the booking, and no-show bookings are excluded from membership completion counts.
UC-36	Handle booking conflict	Handle booking conflict	Prevent new double-booking and let Staff compose existing operation-calendar, issue, reschedule, cancel, refund and notification capabilities to resolve an operational conflict. No dedicated conflict record/workflow exists.

Payment Gateway, Refund & Invoice

ID	Feature	Use Case	Use Case Description
UC-37	View checkout summary	View checkout summary	Displays a server-calculated USD checkout breakdown before a booking or payment is committed.
UC-38	Choose payment option	Choose payment option	Lets a Customer choose deposit/full for an online PayPal booking and Staff choose deposit/full cash collection for a walk-in booking.

UC-39	<i>Pay deposit/full amount via online payment sandbox</i>	<i>Pay deposit/full amount via online payment sandbox</i>	<i>Creates and captures a PayPal Orders v2 sandbox transaction for a Customer-owned online booking.</i>
UC-40	<i>Capture/confirm online payment</i>	<i>Capture/confirm online payment</i>	<i>Validates a PayPal capture and reconciles provider identifiers and payment totals into booking and invoice state.</i>
UC-41	<i>Confirm remaining payment</i>	<i>Confirm remaining payment</i>	<i>Records the exact outstanding booking balance as a venue cash transaction.</i>
UC-42	<i>Handle failed/expired payment</i>	<i>Handle failed/expired payment</i>	<i>Keeps failed attempts visible where persisted and expires unpaid booking holds so their slots become available.</i>
UC-43	<i>View payment history</i>	<i>View payment history</i>	<i>Returns persisted payment transactions, filtered to the signed-in Customer or expanded to all records for operators.</i>
UC-44	<i>Generate booking invoice</i>	<i>Generate booking invoice</i>	<i>Creates or updates one invoice per Booking with current pricing, payment, balance, discount, and refund totals.</i>
UC-45	<i>View invoice/payment status</i>	<i>View invoice/payment status</i>	<i>Shows invoice amounts, derived payment status, transaction history, and refund information for an authorized Booking.</i>
UC-46	<i>Process online refund</i>	<i>Process online refund</i>	<i>Submits or reconciles an approved refund against a PayPal capture and records the provider's actual state.</i>
UC-47	<i>Manage refund requests</i>	<i>Manage refund requests</i>	<i>Lets the booking owner create an eligible refund request and lets Staff review, approve/reject, submit/retry PayPal refunds, or record cash completion.</i>
UC-48	<i>Configure deposit rules</i>	<i>Configure deposit rules</i>	<i>Maintains the single default percentage used to calculate every booking deposit.</i>
UC-49	<i>Configure cancellation/refund policy</i>	<i>Configure cancellation/refund policy</i>	<i>Maintains percentage values used by the two implemented pre-start cancellation windows.</i>

Promotion, Membership, Notification & Report

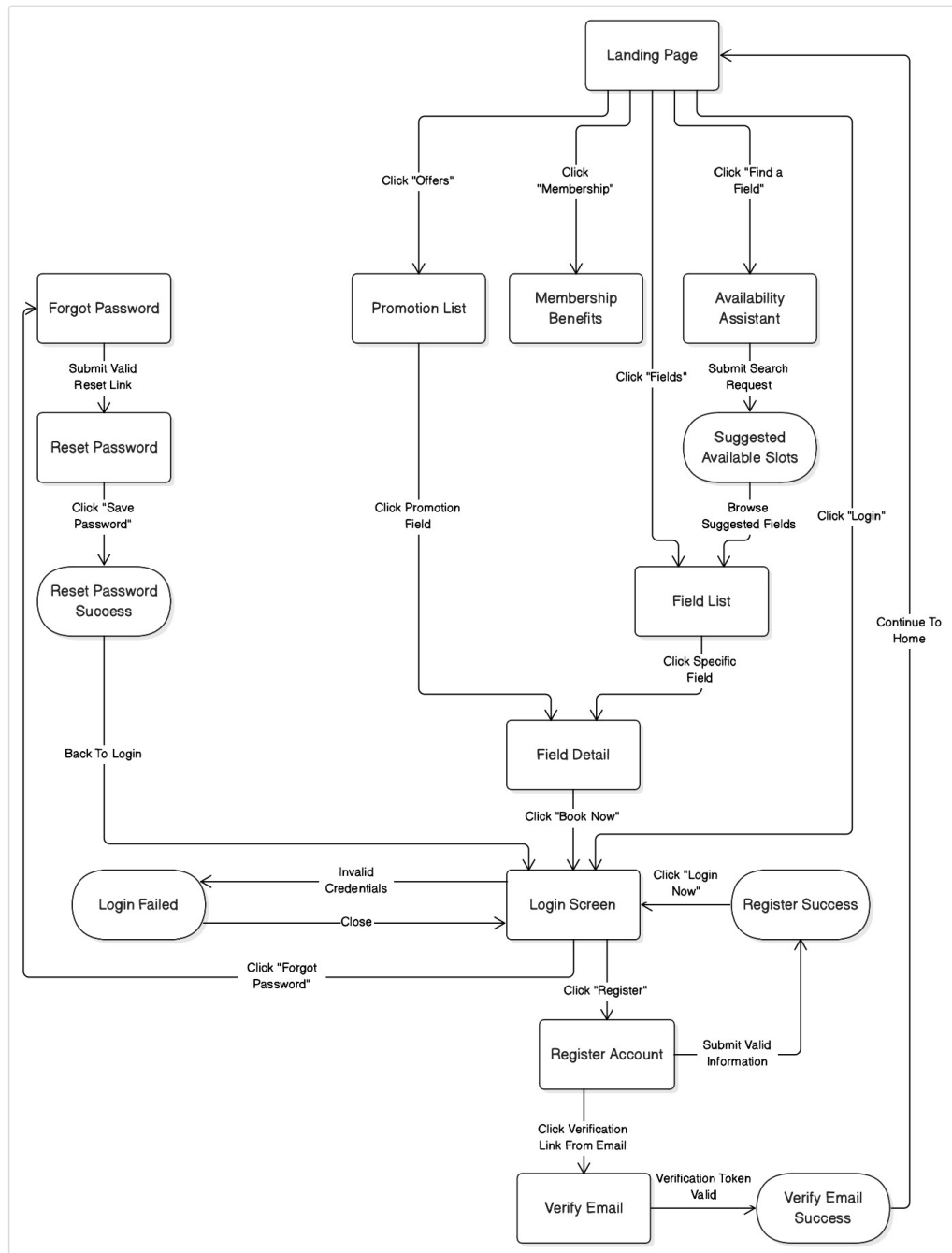
ID	Feature	Use Case	Use Case Description
UC-50	<i>View active promotions</i>	<i>View active promotions</i>	<i>Displays promotion campaign cards and banners from the public promotion endpoint.</i>
UC-51	<i>Manage promotion campaigns</i>	<i>Manage promotion campaigns</i>	<i>Creates and updates campaign identity, banner, discount, limits, dates, status, and eligibility conditions.</i>
UC-52	<i>Apply promotion to booking</i>	<i>Apply promotion to booking</i>	<i>Validates one code against campaign and booking context, calculates discount, and snapshots it when the Booking is created.</i>

UC-53	<i>View membership progress</i>	<i>View membership progress</i>	<i>Shows the current tier, lifetime completed count, configured period-aware progress, discount, next tier, and remaining target.</i>
UC-54	<i>Manage membership rules</i>	<i>Manage membership rules</i>	<i>Creates and updates membership thresholds, discounts, benefits, order, active status, and lifetime/monthly/weekly qualification rules.</i>
UC-55	<i>View membership benefits</i>	<i>View membership benefits</i>	<i>Shows active tier names, qualification rules, discount percentages, benefits, and signed-in Customer progress.</i>
UC-56	<i>Send booking confirmation notification</i>	<i>Send booking confirmation notification</i>	<i>Creates persistent in-app booking confirmation/category notifications for the Customer.</i>
UC-57	<i>Send booking reminder notification</i>	<i>Send booking reminder notification</i>	<i>Creates exactly one in-app reminder before the Booking start time.</i>
UC-58	<i>Send cancellation/refund notification</i>	<i>Send cancellation/refund notification</i>	<i>Creates persistent in-app cancellation and refund lifecycle notifications for the Customer.</i>
UC-59	<i>View revenue report</i>	<i>View revenue report</i>	<i>Displays current gross collection, tracked PayPal processor fee, refund, net revenue, booked-value, discount, and Booking totals in USD.</i>
UC-60	<i>View booking report</i>	<i>View booking report</i>	<i>Displays Booking counts by lifecycle status, counts by field, peak start times, and headline completion/cancellation/no-show counts.</i>
UC-61	<i>View customer activity report</i>	<i>View customer activity report</i>	<i>Displays tracked Customers, returning Customer count, top Customers by Booking count, and membership distribution.</i>
UC-62	<i>Ask availability assistant</i>	<i>Ask availability assistant</i>	<i>A signed-in Customer asks Gemini to interpret a natural-language field request. All fields, times, prices, promotions, and ranked results remain grounded in live GoalZone data.</i>
UC-63	<i>Get suggested available slots</i>	<i>Get suggested available slots</i>	<i>Deterministically ranks live available slots with explanations and transfers the selected slot/date into normal checkout, where availability is revalidated.</i>

2. Overall Functionalities

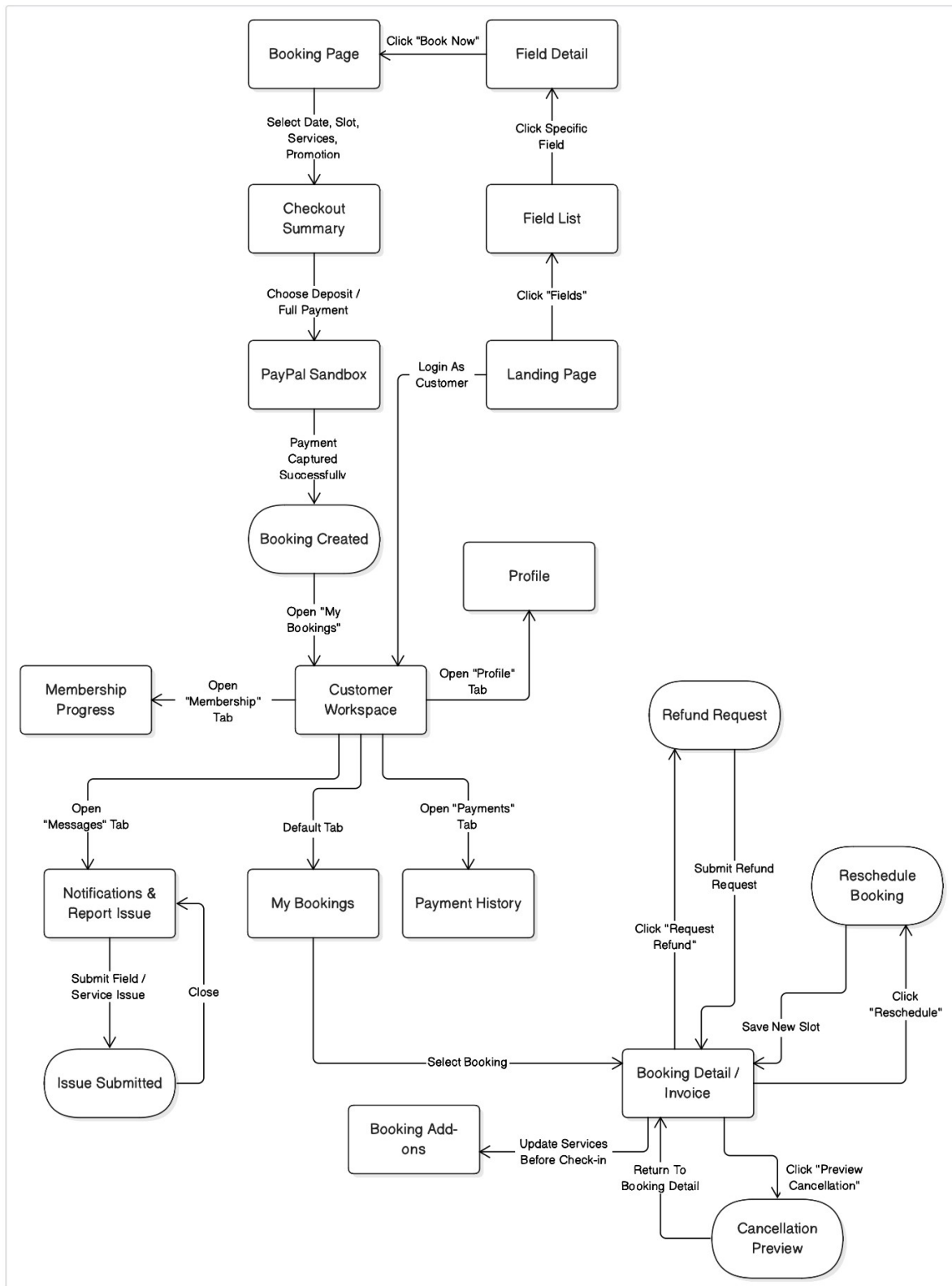
2.1 Screens Flow

Guest's Screen Flow



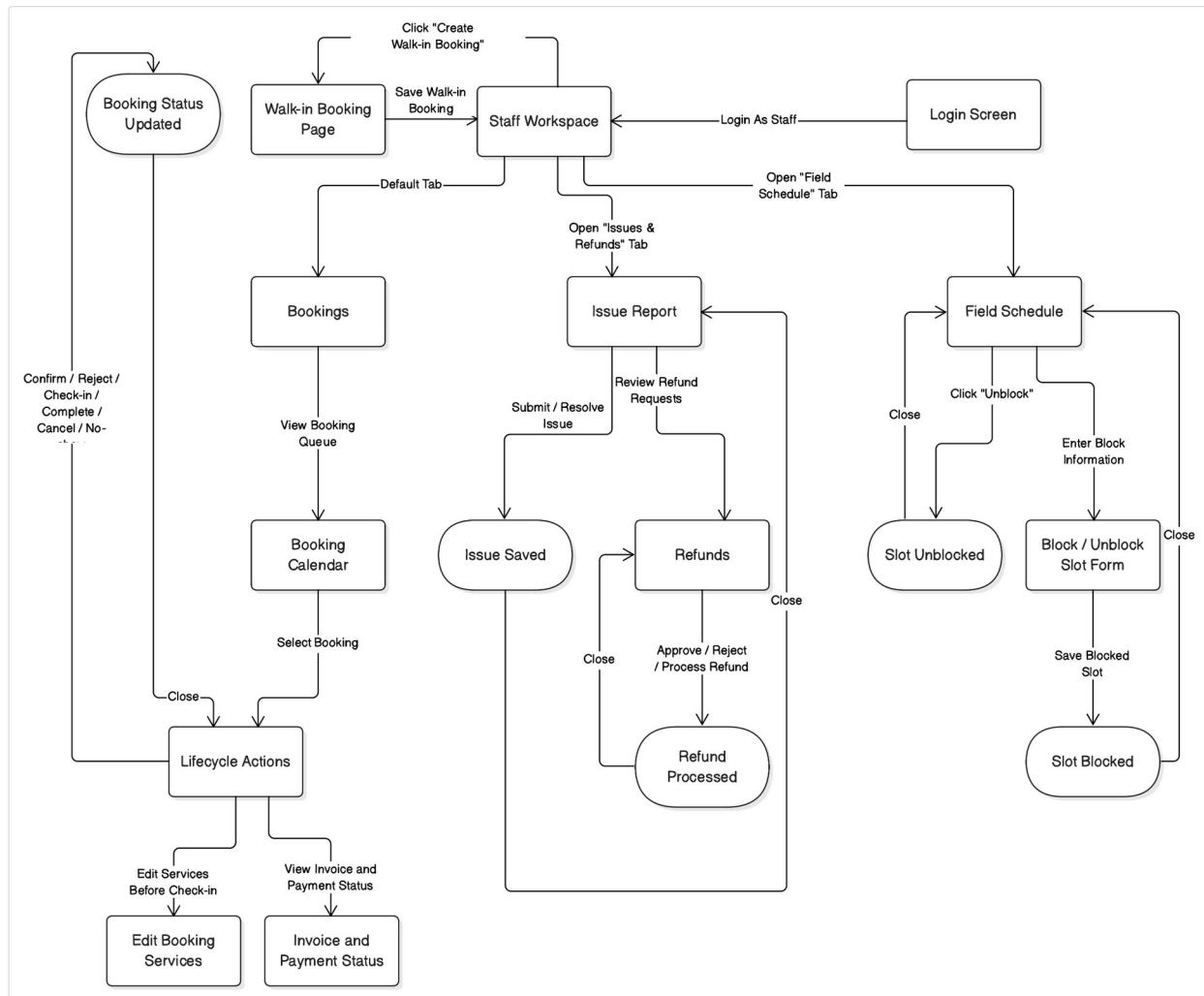
Guest Screen Flow

Customer's Screen Flow



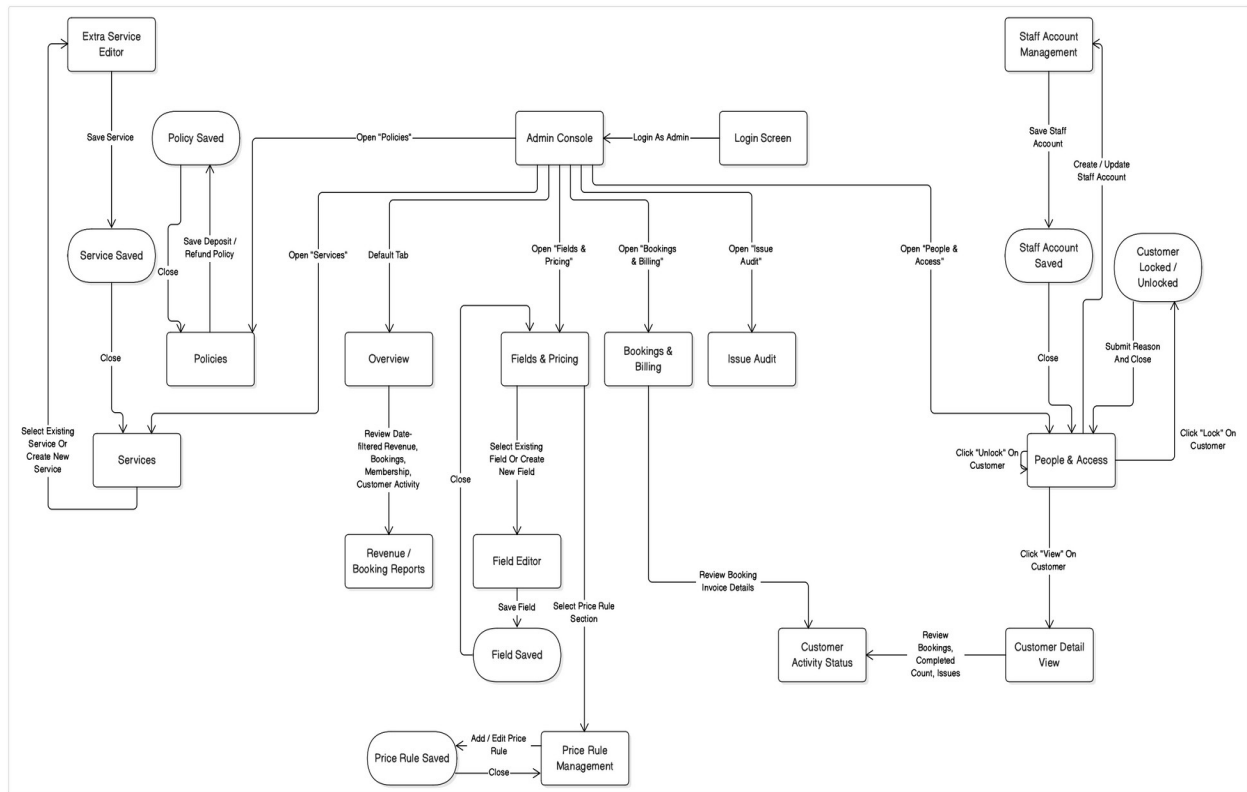
Customer Screen Flow

Staff's Screen Flow



Staff Screen Flow

Admin's Screen Flow



Admin Screen Flow

2.2 Screen Descriptions

#	Feature	Screen	Description
1	Account & Customer Status	Landing Page	Public entry point for browsing fields, active promotions, membership benefits, and account actions.
2	Account & Customer Status	Login Screen	Authenticates a registered user and routes the user to role-appropriate navigation.
3	Account & Customer Status	Register Account	Creates a unique Customer account and starts email verification.
4	Account & Customer Status	Forgot/Reset Password	Issues a time-limited reset link and accepts a replacement password.
5	Account & Customer Status	Profile / Edit Profile	Displays account status and permits safe profile/password changes.
6	Account & Customer Status	Customer Management	Admin reviews Customer accounts, manages account lock/unlock, and alone may inspect selected Customer activity.
7	Account & Customer Status	Staff Management	Admin creates and maintains Staff identity/status; Staff chooses the password from an emailed invitation.
8	Field, Slot, Service & Issue Management	Field List	Shows the public catalogue of active football fields.
9	Field, Slot, Service & Issue Management	Field Detail	Displays one active field with image, facilities, pricing, and booking entry points.
10	Field, Slot, Service & Issue Management	Search Available Fields	Searches generated slots by playing date, field type, and optional field.
11	Field, Slot, Service & Issue Management	Field Management	Admin creates, updates, activates, and deactivates football fields, including images.
12	Field, Slot, Service & Issue	Field Pricing Management	Admin maintains effective weekday/weekend price bands for each field.

	<i>Management</i>		
13	<i>Field, Slot, Service & Issue Management</i>	<i>Extra Service Selection</i>	<i>Selects and updates booking add-ons while validating stock/capacity and pre-check-in timing.</i>
14	<i>Field, Slot, Service & Issue Management</i>	<i>Extra Service Management</i>	<i>Admin maintains rental, sale, and staff-provided add-ons.</i>
15	<i>Field, Slot, Service & Issue Management</i>	<i>Unavailable Slot Management</i>	<i>Staff blocks/unblocks field time with a reason and reviews daily field operations.</i>
16	<i>Field, Slot, Service & Issue Management</i>	<i>Issue Report / Issue Management</i>	<i>Customer/Staff reports a field, booking, or service issue; Staff resolves it and notifies the reporter.</i>
17	<i>Booking Lifecycle</i>	<i>Online Booking</i>	<i>Customer creates a pending online booking and continues to Customer-only PayPal checkout.</i>
18	<i>Booking Lifecycle</i>	<i>Walk-in Booking</i>	<i>Staff creates a confirmed booking for an existing on-site Customer and records cash payment.</i>
19	<i>Booking Lifecycle</i>	<i>Booking Detail</i>	<i>Role/ownership-protected view of booking, billing, and allowed lifecycle actions.</i>
20	<i>Booking Lifecycle</i>	<i>My Bookings</i>	<i>Customer workspace for current and past owned bookings.</i>
21	<i>Booking Lifecycle</i>	<i>Booking Calendar</i>	<i>Staff daily operations queue/calendar with selected-booking controls.</i>
22	<i>Booking Lifecycle</i>	<i>Reschedule Booking</i>	<i>Moves an eligible owned/operated booking to a free slot and reconciles the price delta.</i>
23	<i>Booking Lifecycle</i>	<i>Cancellation Preview</i>	<i>Shows policy-derived cancellation fee/refundable amount before the user confirms cancellation.</i>
24	<i>Payment Gateway, Refund & Invoice</i>	<i>Checkout Summary</i>	<i>Displays a single USD pricing source of truth before Customer online or Staff walk-in payment choice.</i>

25	Payment Gateway, Refund & Invoice	Payment Sandbox	Customer-only PayPal Sandbox Orders v2 create/capture/cancel flow.
26	Payment Gateway, Refund & Invoice	Payment History	Shows real persisted transaction history subject to booking ownership/operator access.
27	Payment Gateway, Refund & Invoice	Invoice Detail	Displays the generated booking invoice and current settlement state.
28	Payment Gateway, Refund & Invoice	Refund Management	Customer submits an eligible request; Staff reviews it and completes PayPal or cash refund workflows.
29	Payment Gateway, Refund & Invoice	Booking and Refund Policies	Admin policy workspace for deposit, cancellation/refund, payment timeout, reminder, and automatic slot opening/closing/duration/horizon rules.
30	Promotion, Membership, Notification & Report	Promotion List	Public list of active, date-valid promotion campaigns and event sales.
31	Promotion, Membership, Notification & Report	Promotion Management	Admin creates/updates campaigns; checkout validates and snapshots one eligible code.
32	Promotion, Membership, Notification & Report	Membership Benefits	Public view of active membership levels, thresholds, and benefit descriptions.
33	Promotion, Membership, Notification & Report	Membership Progress	Customer view of current membership and lifetime/monthly/consecutive-week progress toward the next active level.
34	Promotion, Membership, Notification & Report	Membership Rule Management	Admin maintains ordered membership benefits and lifetime, monthly, or consecutive-week qualification rules.
35	Promotion, Membership, Notification & Report	Notification List	Displays persisted in-app booking, payment, issue, cancellation, refund, and reminder notifications.
36	Promotion, Membership, Notification & Report	Revenue Report	Admin-only aggregate of gross collections, tracked PayPal fees, refunds, net revenue, discounts, and portfolio value in USD.
37	Promotion,	Booking Report	Admin-only booking lifecycle, peak-demand, and

	Membership, Notification & Report		field-utilization statistics.
38	Promotion, Membership, Notification & Report	Customer Account state Report	Admin-only aggregate of tracked/returning Customers, top activity, and membership distribution.
39	Promotion, Membership, Notification & Report	Availability Assistant	Guest/Customer discovery page with deterministic live slot filters. A signed-in Customer may additionally ask grounded Gemini; selected results continue into ordinary checkout.

2.3 Screen Authorization

Screen	Guest	Customer	Staff	Admin
Landing Page	X	X		
Login Screen	X	X	X	X
Register Account	X			
Forgot/Reset Password	X			
Profile / Edit Profile		X	X	X
Customer Management				X
Staff Management				X
Field List	X	X	X	X
Field Detail	X	X	X	X
Search Available Fields	X	X	X	
Field Management				X
Field Pricing Management				X
Extra Service Selection	X	X	X	
Extra Service Management				X
Unavailable Slot Management			X	
Issue Report / Issue Management		X	X	X
Online Booking		X		
Walk-in Booking			X	
Booking Detail		X	X	X
My Bookings		X		
Booking Calendar			X	
Reschedule Booking		X	X	
Cancellation Preview		X	X	
Checkout Summary	X	X	X	
Payment Sandbox		X		
Payment History		X	X	X
Invoice Detail		X	X	X

<i>Refund Management</i>		<i>X</i>	<i>X</i>	<i>X</i>
<i>Booking and Refund Policies</i>				<i>X</i>
<i>Promotion List</i>	<i>X</i>	<i>X</i>	<i>X</i>	<i>X</i>
<i>Promotion Management</i>				<i>X</i>
<i>Membership Benefits</i>	<i>X</i>	<i>X</i>	<i>X</i>	<i>X</i>
<i>Membership Progress</i>		<i>X</i>		
<i>Membership Rule Management</i>				<i>X</i>
<i>Notification List</i>		<i>X</i>	<i>X</i>	<i>X</i>
<i>Revenue Report</i>				<i>X</i>
<i>Booking Report</i>				<i>X</i>
<i>Customer Account state Report</i>				<i>X</i>
<i>Availability Assistant</i>	<i>X</i>	<i>X</i>		

2.4 Non-UI Functions

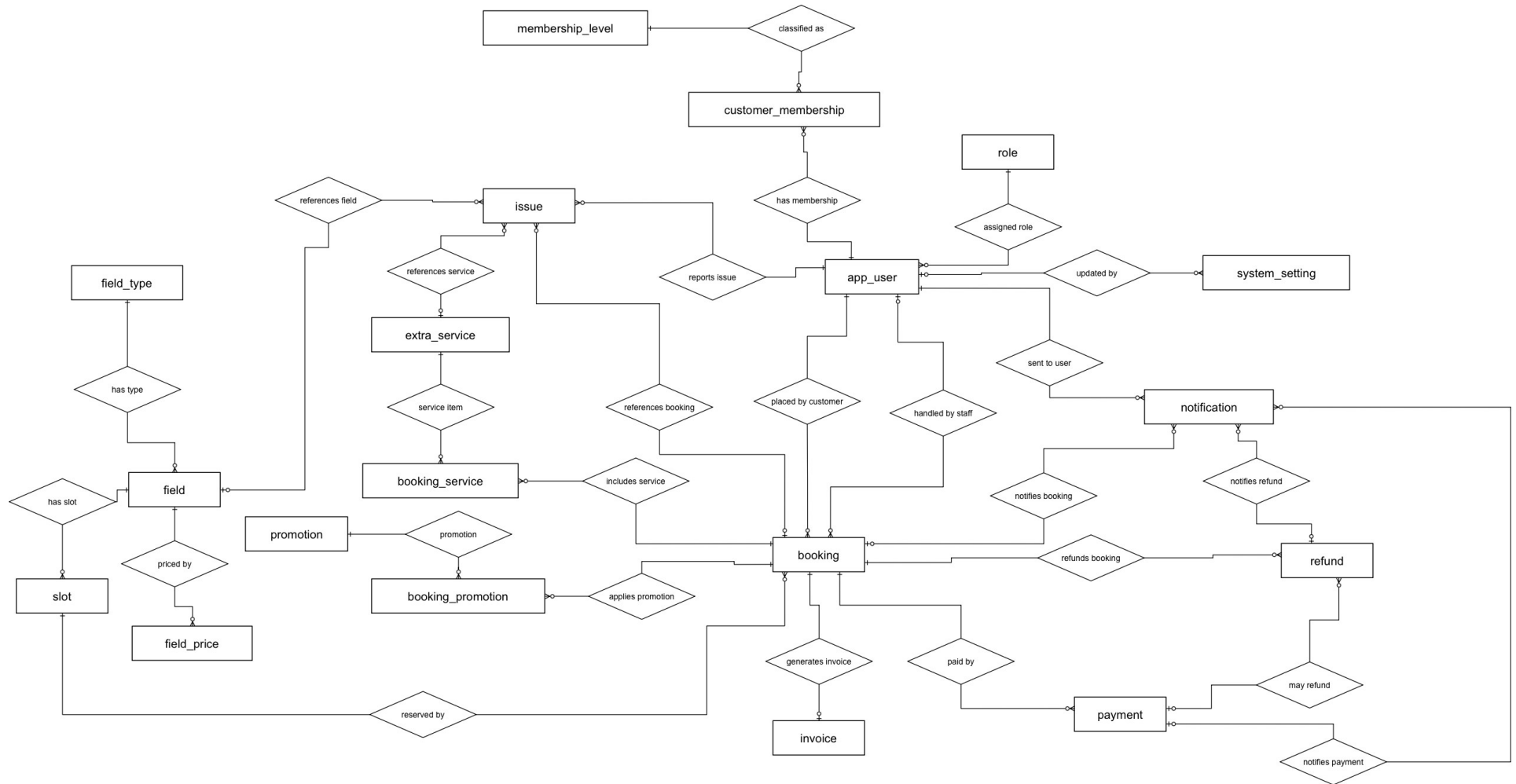
#	Feature	System Function	Description
1	Booking Lifecycle	Availability validation	Checks the selected field/slot against existing bookings, blocked slots, and past-time validation.
2	Payment Gateway, Refund & Invoice	Price calculation	Calculates field price, services, promotion discount, membership benefit, deposit and payable amount.
3	Field, Slot, Service & Issue Management	Extra service availability check	Checks service active status, stock/capacity and quantity limit.
4	Promotion, Membership, Notification & Report	Promotion validation	Validates date range, usage limit, field/time condition, membership condition and stacking rule.
5	Payment Gateway, Refund & Invoice	Payment capture callback	Receives sandbox payment result and stores transaction status.
6	Booking Lifecycle	Booking status transition	Controls valid transitions such as Pending -> Confirmed -> Checked-in -> Completed.
7	Payment Gateway, Refund & Invoice	Cancellation/refund calculation	Calculates cancellation fee and refundable amount before cancellation.
8	Payment Gateway, Refund & Invoice	Invoice generation	Generates invoice from booking price, services, discounts, paid amount and refund amount.
9	Promotion, Membership,	Membership progress update	Updates progress after booking becomes Completed.

	Notification & Report		
10	Promotion, Membership, Notification & Report	Notification dispatch	Creates in-app notifications for confirmation, reminder, cancellation, payment and refund.
11	Payment Gateway, Refund & Invoice	Payment expiration cleanup	Marks pending payment/booking as expired after timeout.
12	Promotion, Membership, Notification & Report	Report aggregation	Aggregates revenue, booking, field utilization and customer activity data.

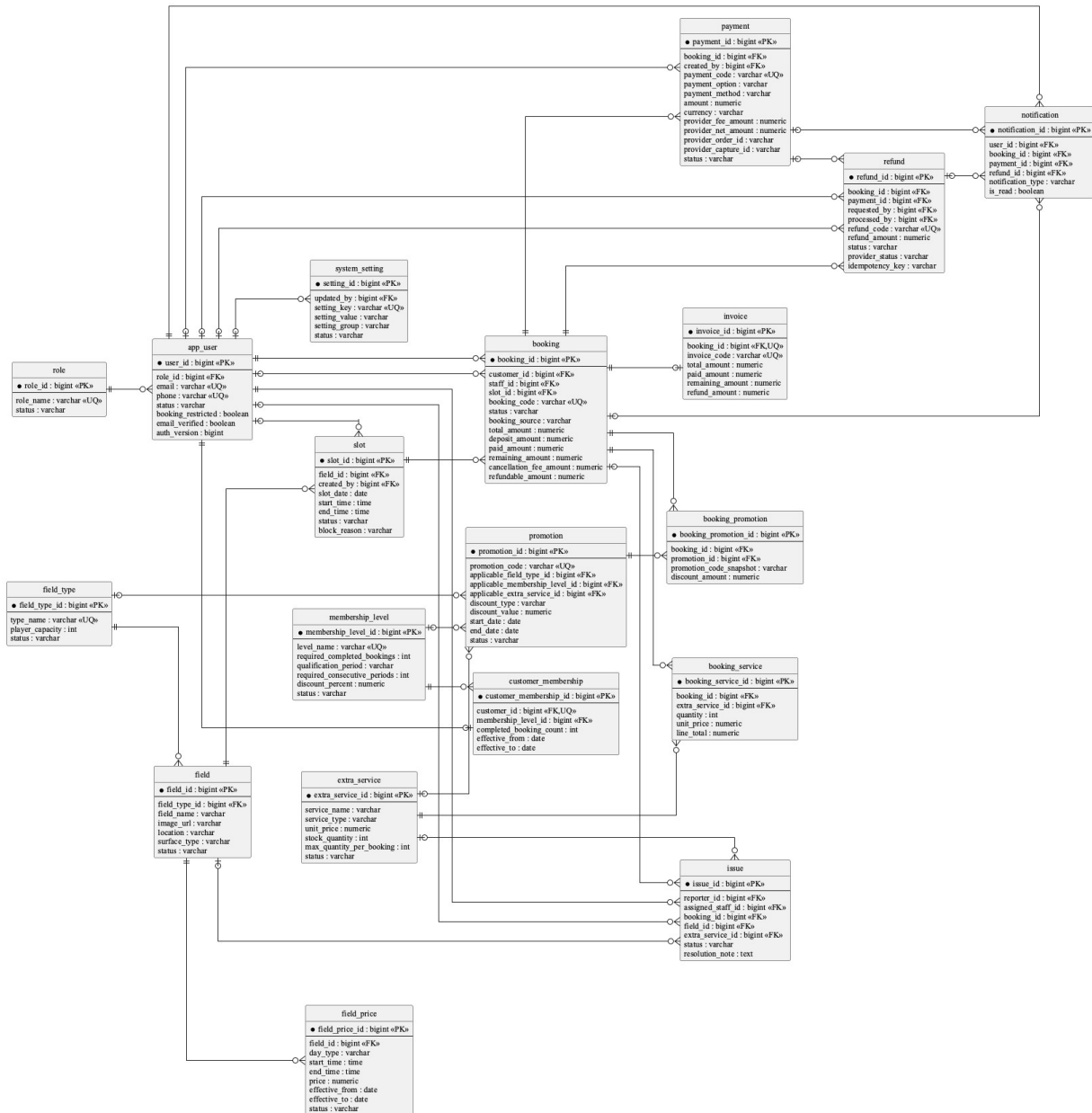
3. System High Level Design

3.1 Database Design

GAMS-SRS_v1.0



b. Database Schema



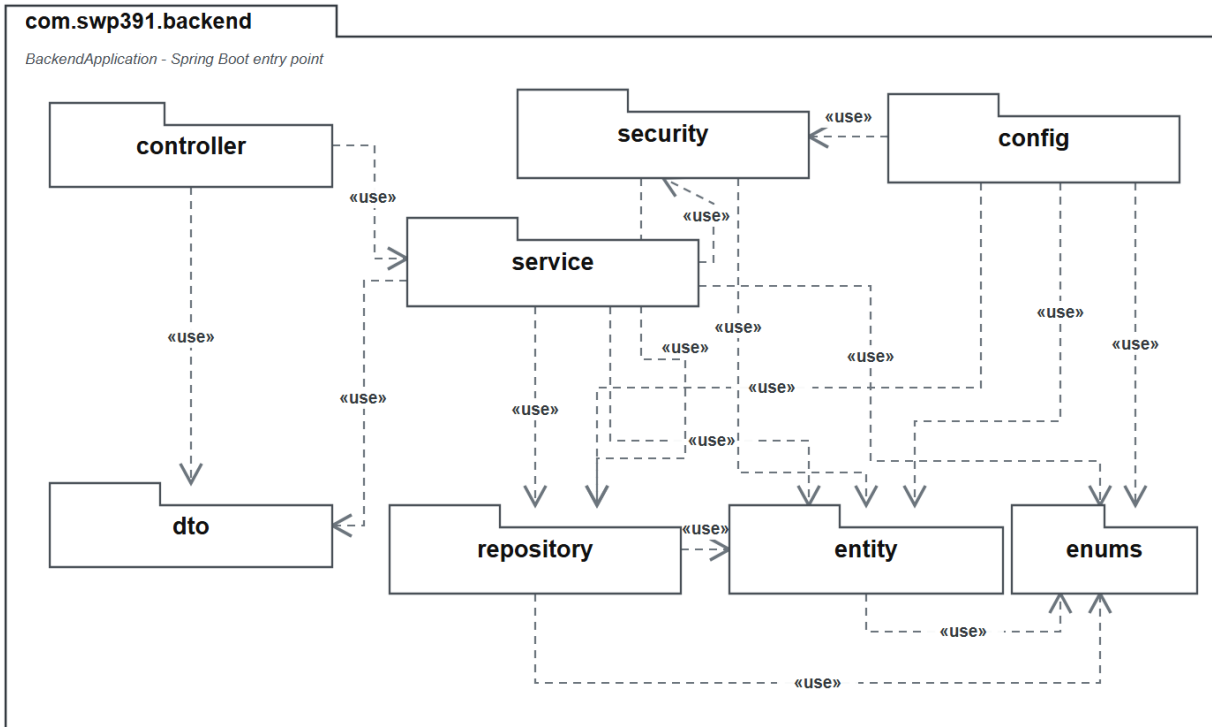
c. Table Descriptions

No	Table	Description
01	role	Primary key: role_id. Use role, not roles or user_role.
02	app_user	Primary key: user_id. Stores Customer, Staff, and Admin identities. status is the single persisted source of truth for active/inactive/locked access; lock_reason explains an administrative Customer lock and auth_version revokes JWTs.

03	<i>membership_level</i>	<i>Primary key: membership_level_id. Stores lifetime/monthly/weekly completed-booking rules with consecutive-week requirements added by V10.</i>
04	<i>customer_membership</i>	<i>Primary key: customer_membership_id. One row per customer is enforced by later integrity migration.</i>
05	<i>field_type</i>	<i>Primary key: field_type_id. Normalizes reusable field-type name and player capacity so multiple fields do not duplicate the same catalogue metadata.</i>
06	<i>field</i>	<i>Primary key: field_id. Physical table is field although the entity is FootballField.</i>
07	<i>field_price</i>	<i>Primary key: field_price_id. Keeps effective weekday/weekend and time-band prices separate from field identity so pricing can change over time without rewriting a field.</i>
08	<i>slot</i>	<i>Primary key: slot_id. Unavailable periods are represented by slot.status='blocked'; no unavailable_slot table exists.</i>
09	<i>booking</i>	<i>Primary key: booking_id. Lifecycle, financial snapshots, and timestamps live together; there is no booking_status_history table.</i>
10	<i>extra_service</i>	<i>Primary key: extra_service_id. Use extra_service, not service.</i>
11	<i>booking_service</i>	<i>Primary key: booking_service_id. Physical table is booking_service although the entity/repository use BookingServiceItem naming.</i>
12	<i>issue</i>	<i>Primary key: issue_id. Resolution fields are on issue; no issue_resolution table exists.</i>
13	<i>promotion</i>	<i>Primary key: promotion_id. Applicability dimensions are nullable columns on promotion, not join tables.</i>
14	<i>booking_promotion</i>	<i>Primary key: booking_promotion_id. Stores applied code/discount snapshots; ordinary product scope is one code per booking.</i>
15	<i>payment</i>	<i>Primary key: payment_id. Provider order/capture/status/currency/idempotency fields are on payment; V11 also records the exact provider fee and provider net amount returned by PayPal.</i>
16	<i>invoice</i>	<i>Primary key: invoice_id. booking_id is unique, so the relationship is one invoice per booking.</i>
17	<i>refund</i>	<i>Primary key: refund_id. Provider status, idempotency key, and gateway message are added by V9; V11 reconciles completed legacy refunds with booking, invoice, and payment status.</i>
18	<i>notification</i>	<i>Primary key: notification_id. Supports optional booking_id/payment_id/refund_id, but current shared</i>

		<i>notifyUser only supplies booking_id.</i>
19	system_setting	<i>Primary key: setting_id. Stores administrator-editable deposit, refund, payment-timeout, reminder, and automatic slot-generation policies as key/value configuration; it is not redundant field data.</i>

3.2 Code Packages



Package descriptions

No	Package	Description
01	<i>com.swp391.back end</i>	<i>1 Java file(s): BackendApplication. Count the Spring Boot entry point separately from feature packages.</i>
02	<i>com.swp391.back end.config</i>	<i>4 Java file(s): CorsConfig, DataSeeder, PostgresDdlSafetyEnvironmentPostProcessor, SecurityConfig. Authorization rules for many Admin/Staff endpoints live in SecurityConfig, not inside every service method.</i>
03	<i>com.swp391.back end.controller</i>	<i>9 Java file(s): AccountController, ApiExceptionHandler, AvailabilityAssistantController, BookingController, FieldOperationController, ImageUploadController, PaymentController, PromotionReportController, and SystemController.</i>

04	<i>com.swp391.back end.dto</i>	<i>1 Java file(s): ApiRequests. Request records are nested in one ApiRequests class rather than separate DTO files.</i>
05	<i>com.swp391.back end.entity</i>	<i>20 Java file(s): AppUser, AuditEntity, Booking, BookingPromotion, BookingServiceItem, CustomerMembership, ExtraService, FieldPrice, FieldType, FootballField, Invoice, Issue, MembershipLevel, Notification, Payment, Promotion, Refund, Role, Slot, SystemSetting. There are 19 table-backed entities plus the mapped superclass AuditEntity; do not count AuditEntity as a table.</i>
06	<i>com.swp391.back end.enums</i>	<i>13 Java file(s): AccountStatus, BookingSource, BookingStatus, CommonStatus, DiscountType, IssueStatus, NotificationType, PaymentMethod, PaymentOption, PaymentStatus, RefundStatus, ServiceType, SlotStatus. Status/value sets are Java enums persisted as strings; they are not lookup tables.</i>
07	<i>com.swp391.back end.repository</i>	<i>19 Java file(s): AppUserRepository, BookingPromotionRepository, BookingRepository, BookingServiceItemRepository, CustomerMembershipRepository, ExtraServiceRepository, FieldPriceRepository, FieldTypeRepository, FootballFieldRepository, InvoiceRepository, IssueRepository, MembershipLevelRepository, NotificationRepository, PaymentRepository, PromotionRepository, RefundRepository, RoleRepository, SlotRepository, SystemSettingRepository. Only BookingServiceItemRepository contains explicit JPQL; most queries are Spring Data derived or inherited CRUD.</i>
08	<i>com.swp391.back end.security</i>	<i>4 Java file(s): CustomUserDetailsService, JwtAuthenticationFilter, JwtService, SecurityUser. JWT identity and role authorization are cross-cutting implementation components, not persistence entities.</i>
09	<i>com.swp391.back end.service</i>	<i>16 Java file(s): AccountService, ApiException, AvailabilityAssistantService, BookingMaintenanceScheduler, BookingWorkflowService, DomainSupportService, FieldOperationService, ImageStorageService, PayPalCheckoutService, PaymentWorkflowService, PromotionReportService, SlotGenerationScheduler, SlotGenerationService, SystemQueryService, VerificationEmailDelivery, and VerificationEmailService.</i>

II. Requirement Specifications

Account & Customer Status

UC-01. Register account

UC ID and Name:	UC-01 Register account		
Created By:	BonVT	Date Created:	30/05/2026

Primary Actor:	Guest	Secondary Actors:	Email service
Trigger:	The guest selects Register and submits the registration form.		
Description:	A guest registers a Customer account, receives an authenticated session, and is asked to verify the registered email address.		
Preconditions:	PRE-1: The guest is not authenticated. PRE-2: The Customer role and a default membership level are configured.		
Postconditions:	POST-1: A unique active Customer account is stored with a BCrypt password hash and email_verified = false. POST-2: A default customer membership is attached. POST-3: A JWT session and user summary are returned, and a one-hour verification link is sent or its delivery failure is reported.		
Normal Flow:	A. Guest Registers an Account 1. The guest enters full name, email, phone, password, and password confirmation. 2. The system validates required fields, email format, Vietnamese phone format, and password policy. 3. The system verifies that email and phone are not already registered. 4. The system creates an active Customer account and stores the BCrypt password hash. 5. The system creates a 16-character verification token valid for one hour and sends the verification email. 6. The system attaches the default membership level. 7. The system returns the JWT, user summary, verification status, and email delivery status.		
Alternative Flow:	A1: Registration data is invalid 1. The system identifies the invalid or missing field and rejects the request. 2. The guest corrects the form and submits again. A2: Email or phone already exists 1. The system rejects the request without creating a second account. A3: Verification email cannot be delivered 1. The account remains created and the response exposes the delivery status so the user can resend verification.		
Exceptions:	EXC-01: Account or membership persistence fails; the registration transaction is rolled back.		
Priority:	Medium		
Frequency of Use:	Medium		
Business Rules:	BR-01		
Other Information:	Password: 8-72 characters, no whitespace, with uppercase, lowercase, number, and special character. Phone: 10 digits starting with 0.		
Assumptions:	SMTP and the frontend verification URL are configured; failure to send email does not delete the created account.		

UC-02. Login

UC ID and	UC-02 Login
------------------	-------------

Name:			
Created By:	BonVT	Date Created:	30/05/2026
Primary Actor:	Guest / Registered User	Secondary Actors:	None
Trigger:	The user submits an email-or-phone identity and password on the Login screen.		
Description:	A registered user signs in with email or phone and receives a role-aware authenticated session.		
Preconditions:	PRE-1: A registered account exists.		
Postconditions:	POST-1: The active user receives a JWT containing role and auth-version claims. POST-2: last_login_at is updated and the role-aware application area is opened.		
Normal Flow:	A. User Logs In 1. The user enters email or phone and password. 2. The system locates the account by exact email, then by phone when needed. 3. The system compares the password with the stored BCrypt hash. 4. The system verifies that account status is active. 5. The system records the login time and returns a JWT plus user summary. 6. The client stores the session and opens the permitted role-based area.		
Alternative Flow:	A1: Identity or password is missing 1. The system rejects the request and identifies the required field. A2: Account is not found or password is incorrect 1. The system denies authentication. A3: Account is locked or inactive 1. The system denies login even when the password is correct.		
Exceptions:	EXC-01: Account data cannot be loaded; no session is issued.		
Priority:	Medium		
Frequency of Use:	High		
Business Rules:	BR-02		
Other Information:	A locked account cannot log in. The login response asks the Customer to check email for the lock reason.		
Assumptions:	The JWT signing secret and role data are configured.		

UC-03. Logout

UC ID and Name:	UC-03 Logout		
Created By:	BonVT	Date Created:	30/05/2026
Primary Actor:	Authenticated User	Secondary Actors:	None
Trigger:	The user selects Sign out.		
Description:	An authenticated user signs out; the server revokes the current token version and the client clears local session data.		
Preconditions:	PRE-1: The client has a local authenticated session; the token may still be valid or may already be expired.		

Postconditions:	POST-1: When the token is valid, the server increments auth_version so the token cannot be reused. POST-2: The client always clears token and user data and returns to a public screen.
Normal Flow:	A. User Logs Out 1. The client calls the authenticated logout endpoint. 2. The server resolves the current account and increments auth_version. 3. The server confirms that the token has been revoked. 4. The client removes all local authentication state. 5. A later request using the old token fails authentication.
Alternative Flow:	A1: Token is missing, invalid, or expired 1. The server cannot revoke that request. 2. The client still clears local authentication state in the finalization step.
Exceptions:	EXC-01: The logout request fails because the server is unavailable; the client still signs out locally.
Priority:	Low
Frequency of Use:	High
Business Rules:	BR-02
Other Information:	Logout is server-enforced through auth_version, not only a client-side token deletion.
Assumptions:	JWT validation compares the token auth-version claim with the current account value.

UC-04. Manage personal profile

UC ID and Name:	UC-04 Manage personal profile		
Created By:	BonVT	Date Created:	30/05/2026
Primary Actor:	Customer, Staff, or Admin	Secondary Actors:	None
Trigger:	An authenticated user opens Profile and submits allowed profile changes.		
Description:	An authenticated user updates only their own allowed profile fields.		
Preconditions:	PRE-1: The requester is authenticated. PRE-2: The requester updates their own account, or is an Admin updating another account.		
Postconditions:	POST-1: Allowed profile fields are updated and the latest user summary is displayed.		
Normal Flow:	A. User Updates Profile 1. The system displays the current profile. 2. The user edits full name, phone, and/or address. 3. The system verifies that the requester owns the account. 4. The system validates phone format and uniqueness when phone is supplied. 5. The system stores cleaned values and returns the updated profile.		
Alternative Flow:	A1: Requester tries to update another account 1. The system denies the request.		

	A2: Phone is invalid or belongs to another account 1. The system rejects the update and preserves the current profile. A3: Optional full name or phone is blank 1. The system keeps the current value; address may be cleared.
Exceptions:	EXC-01: Profile persistence fails; the previous stored values remain effective.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-01, BR-02, BR-03
Other Information:	Email, role, status, email-verification, and account-lock fields are not editable through this use case.
Assumptions:	Profile images are outside the approved project scope.

UC-05. Change password

UC ID and Name:	UC-05 Change password		
Created By:	BonVT	Date Created:	30/05/2026
Primary Actor:	Authenticated User	Secondary Actors:	None
Trigger:	The user submits current password, new password, and confirmation.		
Description:	An authenticated user changes the password after confirming the current password; existing tokens are revoked.		
Preconditions:	PRE-1: The user is authenticated and is changing their own account password.		
Postconditions:	POST-1: The new BCrypt password hash is stored. POST-2: auth_version is incremented and all previously issued JWTs are invalid.		
Normal Flow:	A. User Changes Password - The user enters the current password and a confirmed new password. - The system verifies the current password against the stored hash. - The system validates the new password policy and confirmation. - The system verifies that the new password differs from the current password. - The system stores a new BCrypt hash and revokes all active tokens. - The client requires the user to sign in again.		
Alternative Flow:	A1: Current password is incorrect 1. The system denies the change. A2: New password is weak, mismatched, or unchanged 1. The system rejects the request and shows the validation reason.		
Exceptions:	EXC-01: Password persistence fails; the current password and token version remain in effect.		
Priority:	Medium		
Frequency of Use:	Low		
Business Rules:	BR-02		
Other	The same password policy as registration applies.		

Information:	
Assumptions:	<i>The client discards its JWT after a successful password change.</i>

UC-06. Forgot/reset password

UC ID and Name:	UC-06 Forgot/reset password		
Created By:	BonVT	Date Created:	30/05/2026
Primary Actor:	Guest / Registered User	Secondary Actors:	Email service
Trigger:	The user requests a password reset and later opens the emailed reset link.		
Description:	A user requests a time-limited reset link and sets a new password without exposing whether an email is registered.		
Preconditions:	PRE-1: The reset-request and reset-password screens are available.		
Postconditions:	POST-1: For an eligible account, a one-hour reset token is issued and emailed. POST-2: After a valid reset, the new BCrypt hash is stored, reset data is cleared, and all JWTs are revoked.		
Normal Flow:	A. User Resets a Forgotten Password 1. The user submits an email address. 2. The system validates the email format and returns the same neutral response for registered and unregistered addresses. 3. For an active matching account, the system creates a 16-character reset token and sends a one-hour link. 4. The user opens the link; the system validates user ID, token, and issue time. 5. The user submits and confirms a policy-compliant new password. 6. The system stores the new hash, clears reset fields, revokes prior tokens, and confirms success.		
Alternative Flow:	A1: Email is unregistered or account is inactive 1. The system returns the neutral success message and sends no reset link. A2: Link is invalid or at least one hour old 1. The system rejects validation and asks the user to request a new link. A3: New password is invalid or mismatched 1. The system preserves the existing password and reset token.		
Exceptions:	EXC-01: Email delivery fails; the user may submit another reset request without exposing account existence.		
Priority:	Medium		
Frequency of Use:	Low		
Business Rules:	BR-01		
Other Information:	The neutral response prevents account enumeration.		
Assumptions:	The password-reset frontend URL and SMTP service are configured.		

UC-07. Manage customer accounts

UC ID and Name:	UC-07 Manage customer accounts		
Created By:	BonVT	Date Created:	30/05/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	The Admin opens People & Access and chooses View, Lock, or Unlock for a Customer.		
Description:	An Admin views Customer account and activity information in read-only mode and manages sign-in access through Lock/Unlock. Customer profile editing is not exposed.		
Preconditions:	PRE-1: The requester is authenticated with the Admin role.		
Postconditions:	POST-1: The Customer directory or selected read-only detail is displayed. POST-2: A lock stores status 'locked' as the single persisted access state together with the required reason. POST-3: An unlock restores status 'active' and clears the lock reason. POST-4: Lock/Unlock revokes active tokens and creates email/in-app notification evidence.		
Normal Flow:	A. Admin Views a Customer 1. The Admin opens People & Access. 2. The system lists Customer name, email, phone, status, and current lock reason. 3. The Admin chooses View. 4. The system displays read-only profile details, recent bookings, completed-booking count, and reported issues. B. Admin Locks a Customer 5. The Admin chooses Lock and enters a required reason. 6. The system stores the locked state, revokes active tokens, sends email/in-app notification, and blocks login. C. Admin Unlocks a Customer 7. The Admin chooses Unlock. 8. The system restores access, clears the reason, revokes old tokens, and sends restoration notifications.		
Alternative Flow:	A1: The Admin closes View without changing data; no profile update is available. A2: Blank lock reason is rejected and current access is preserved. A3: Email delivery failure does not undo a successfully stored lock/unlock state.		
Exceptions:	EXC-01: Customer detail cannot be loaded; the system shows an error and applies no change. EXC-02: The selected account is not a Customer; Lock/Unlock is rejected. EXC-03: Persistence fails; the previous access state remains authoritative.		
Priority:	Medium		
Frequency of Use:	Medium		
Business Rules:	BR-01, BR-02, BR-03, BR-04		
Other Information:	UC-07 owns the Customer directory and Lock/Unlock access management. The View action uses the UC-09 read-only activity data. Neither action edits the Customer profile or deletes transaction history.		
Assumptions:	SMTP is required for external delivery. The stored account state remains authoritative when email delivery is unavailable.		

UC-08. Manage staff accounts

UC ID and Name:	UC-08 Manage staff accounts		
Created By:	BonVT	Date Created:	30/05/2026
Primary Actor:	Admin	Secondary Actors:	Email service
Trigger:	The Admin opens People & Access and selects Staff accounts.		
Description:	An Admin creates and maintains Staff identity and account status. Each new Staff member receives an invitation link and sets their own password.		
Preconditions:	PRE-1: The requester is authenticated with the Admin role. PRE-2: The Staff role is configured.		
Postconditions:	POST-1: The Staff account is created or its permitted profile/status fields are updated. POST-2: A new account has a one-time password-reset token and an invitation email delivery result; the Admin never receives or sets the Staff password.		
Normal Flow:	A. Admin Creates or Updates Staff 1. The Admin opens People & Access and selects Staff accounts. 2. For creation, the Admin enters full name, unique email, unique phone, and status. 3. The system creates the verified Staff identity with an unguessable internal credential and issues a one-hour setup token. 4. The system emails a Set my password link and reports delivery status to the Admin. 5. The Staff member follows the link and sets a password that satisfies policy. 6. For later maintenance, the Admin may edit Staff profile/status but cannot change the Staff password. 7. Non-active status revokes existing Staff sessions.		
Alternative Flow:	A1: Email/phone is invalid or already used 1. The system rejects the request. A2: Invitation email is unavailable 1. The account remains created and the UI reports delivery status; the Staff member can use Forgot password after SMTP is restored. A3: Admin submits a password in an update 1. The system rejects the password change and instructs the Staff member to change/reset it personally.		
Exceptions:	EXC-01: Staff persistence fails; the previous account state remains effective.		
Priority:	Medium		
Frequency of Use:	Low		
Business Rules:	BR-01, BR-02, BR-03		
Other Information:	Administrators manage Staff identity and status only. Password ownership stays with the Staff member through invitation, Change password, and Forgot password flows.		
Assumptions:	The Staff member controls the destination email inbox and opens the invitation within its one-hour validity window.		

UC-09. View customer activity status

UC ID and Name:	UC-09 View customer activity status		
Created By:	BonVT	Date Created:	30/05/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	Admin selects a Customer from People & access to review operational history.		
Description:	Admin reviews a Customer's recent bookings, completed-booking count, and reported issues.		
Preconditions:	PRE-1: The requester has Admin role. PRE-2: The selected Customer account exists.		
Postconditions:	POST-1: The system returns the customer summary, up to 20 recent bookings, completed-booking count, and up to 20 reported issues.		
Normal Flow:	A. Admin Views Customer Activity 1. Admin selects View for a Customer in People & access. 2. The system verifies Admin access. 3. The system loads the Customer summary. 4. The system loads recent bookings in descending booking-ID order and counts all completed bookings. 5. The system loads recent issues reported by the Customer. 6. The system displays the consolidated read-only activity view.		
Alternative Flow:	A1: Customer has no bookings or issues 1. The system shows zero counts and empty lists. A2: A Customer requests another customer's activity 1. The system denies access.		
Exceptions:	EXC-01: Account state data cannot be loaded; the system shows an error instead of partial invented data.		
Priority:	Low		
Frequency of Use:	Medium		
Business Rules:	BR-02, BR-03		
Other Information:	This is an operational customer-history view, not the aggregate Admin report in UC-61.		
Assumptions:	Recent-list limits are sufficient for the Admin workflow.		

Field, Slot, Service & Issue Management

UC-10. View field list

UC ID and Name:	UC-10 View field list		
Created By:	BaoNG	Date Created:	30/05/2026
Primary Actor:	Guest or Customer	Secondary Actors:	None
Trigger:	The visitor opens the Home or Fields page.		
Description:	Display the public catalogue of active football fields. The implemented list includes field name, type, capacity, description, image, location, surface and active status; pricing is available only after opening field detail.		
Preconditions:	PRE-1: The application and public GET /api/fields endpoint are available.		

	<i>PRE-2: At least one FootballField may exist; no authentication is required.</i>
Postconditions:	<i>POST-1: Active fields are returned and rendered as selectable field cards. POST-2: No field, slot or booking data is changed.</i>
Normal Flow:	<i>A. View field list 1. The visitor navigates to Home or Fields. 2. The frontend requests GET /api/fields. 3. The service queries fields whose status is active. 4. The API maps the catalogue attributes for each active field. 5. The frontend renders each active field and lets the visitor open its details.</i>
Alternative Flow:	<i>A1: No active fields 1. The API returns an empty list. 2. The page displays that no active fields are currently available. A2: Authenticated customer browsing 1. A signed-in Customer uses the same public endpoint and receives the same active catalogue as a Guest.</i>
Exceptions:	<i>EXC-1: A catalogue request failure prevents the current list from loading. EXC-2: Inactive fields are deliberately omitted from this endpoint.</i>
Priority:	<i>Low</i>
Frequency of Use:	<i>Frequent; normally every public browsing session</i>
Business Rules:	<i>BR-05</i>
Other Information:	<i>Implementation rule: Only fields with CommonStatus.active are listed publicly. Basic price is not included in the implemented list response. Backend: FieldOperationController.fields and FieldOperationService.fields. Frontend: FieldsPage.</i>
Assumptions:	<i>Guests and authenticated customers use the same public catalogue response.</i>

UC-11. View field detail

UC ID and Name:	<i>UC-11 View field detail</i>		
Created By:	<i>BaoNG</i>	Date Created:	<i>30/05/2026</i>
Primary Actor:	<i>Guest or Customer</i>	Secondary Actors:	<i>None</i>
Trigger:	<i>The visitor selects a field card.</i>		
Description:	<i>Show the selected field's catalogue attributes, active price bands and price range, globally active add-ons, and up to five available slots from today through the next seven days. Reviews are explicitly not modelled.</i>		
Preconditions:	<i>PRE-1: The selected field ID exists. PRE-2: No authentication is required.</i>		
Postconditions:	<i>POST-1: The field-detail modal displays the returned information. POST-2: The visitor may continue to availability search with the field/type filters preselected.</i>		
Normal Flow:	<i>A. View field detail 1. The visitor selects an active field card.</i>		

	2. The frontend requests <i>GET /api/fields/{fieldId}</i>. 3. The service loads active price rules and calculates minimum/maximum prices. 4. The service gathers globally active extra services and the next five unblocked, unreserved slots in an eight-day window. 5. The modal displays field facts, price bands, add-ons and a Check availability action.
Alternative Flow:	A1: No configured active prices 1. The price list is empty and the returned range is USD 0 to USD 0. 2. The UI states that pricing is not configured. A2: No available slots or services 1. The corresponding lists are empty. 2. The UI displays zero services or no configured services; availability count is zero.
Exceptions:	EXC-1: A nonexistent field ID returns not found. EXC-2: An inactive or unknown field ID returns not found. EXC-3: Reviews always return an empty list because there is no review model.
Priority:	Low
Frequency of Use:	Frequent; commonly before availability search
Business Rules:	BR-05, BR-06
Other Information:	Implementation rule: Only active field-price rows contribute to detail pricing. Availability excludes blocked slots and slots reserved by pending, confirmed or checked-in bookings. Services are not field-specific in the schema. The frontend does not render a review area or the returned availability summary; it renders field facts, price bands and services.
Assumptions:	Catalogue navigation supplies an active field ID, even though the endpoint itself does not enforce active status.

UC-12. Search available fields by date/time

UC ID and Name:	UC-12 Search available fields by date/time		
Created By:	BaoNG	Date Created:	30/05/2026
Primary Actor:	Guest or Customer	Guest or Customer	None
Trigger:	The visitor opens booking search or changes the playing date or field type.		
Description:	Generate missing bookable slots from active Admin rules, then search the requested date and optional field type, calculate current field price, and label each row as available, booked, or blocked.		
Preconditions:	PRE-1: At least one active field exists and the four slot-generation rules are active. PRE-2: No authentication is required for ordinary slot search.		
Postconditions:	POST-1: Missing non-overlapping slots for the requested date are materialized idempotently. POST-2: Matching rows are displayed with price and availability; an available row may continue to checkout.		
Normal Flow:	A. Search available fields by date/time		

	1. The visitor chooses a date and optional field type/field. 2. The frontend requests GET /api/slots/search. 3. The backend reads opening time, closing time, duration, and generation horizon. 4. Missing slots are generated for every active field without overlapping blocked/booked rows. 5. The service derives booking state and price, filters by field type, and returns the daily list. 6. The UI enables only rows with available=true.
Alternative Flow:	A1: No date supplied 1. The service defaults to tomorrow and generates it on demand. A2: Existing exception 1. A booked or blocked row is preserved and returned as unavailable. A3: Daily scheduler 1. The midnight job extends the rolling calendar before any customer search.
Exceptions:	EXC-1: Past dates are rejected. EXC-2: Dates beyond the configured generation horizon are rejected. EXC-3: Invalid or non-divisible schedule rules are rejected when Admin saves them. EXC-4: A database or network error prevents results from refreshing.
Priority:	High
Frequency of Use:	Very frequent; expected before each new booking
Business Rules:	BR-05, BR-06, BR-07, BR-08, BR-09
Other Information:	Implementation rule: generation is idempotent and synchronized in the application instance; the database unique/exclusion constraints remain the final overlap guard.
Assumptions:	The venue uses one global operating schedule for all active fields; field-specific hours may be introduced later without changing the booking lifecycle.

UC-13. Manage football fields

UC ID and Name:	UC-13 Manage football fields		
Created By:	BaoNG	Date Created:	30/05/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	An administrator opens the Fields section of the Admin console or selects a field to edit.		
Description:	Create or update football fields and activate/deactivate them. Deactivation removes a field from public list and search results; deletion is not implemented.		
Preconditions:	PRE-1: The user is authenticated with the Admin role. PRE-2: A field type exists for the field being saved.		
Postconditions:	POST-1: A new or updated field is persisted. POST-2: Public catalogue/search visibility reflects active or inactive status.		
Normal Flow:	A. Manage football fields 1. Admin opens managed fields.		

	2. The system loads all fields, including inactive rows. 3. Admin enters field type, name, location, surface, optional description/image, and status. 4. The backend validates required values and saves the field. 5. The Admin console refreshes the managed list.
Alternative Flow:	A1: Activate or deactivate 1. Admin edits the status to active or inactive. 2. Inactive fields disappear from public catalogue and slot search. A2: Upload image 1. Admin uploads/selects an image and saves its URL with the field.
Exceptions:	EXC-1: Missing name, location, surface or field type is rejected. EXC-2: An unknown field or field type returns not found. EXC-3: Direct booking and reschedule reject slots owned by an inactive field.
Priority:	Medium
Frequency of Use:	Occasional; during setup and facility changes
Business Rules:	BR-02, BR-03, BR-30
Other Information:	Implementation rule: Status values are active/inactive. There is no delete endpoint. Public endpoints normally expose only active fields. Admin endpoints are under /api/admin/fields and are protected by SecurityConfig.
Assumptions:	Existing bookings remain associated with a field after it is deactivated.

UC-14. Manage field pricing by time range

UC ID and Name:	UC-14 Manage field pricing by time range		
Created By:	BaoNG	Date Created:	30/05/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	Admin selects a field's pricing configuration.		
Description:	Create and update price rows by field, day type and time range, with optional effective dates and active/inactive status.		
Preconditions:	PRE-1: The user is authenticated as Admin. PRE-2: The target football field exists.		
Postconditions:	POST-1: The pricing rule is persisted and visible in field detail/admin pricing. POST-2: Active matching rules may affect checkout pricing.		
Normal Flow:	A. Manage field pricing by time range 1. Admin selects a field and opens its price rows. 2. The frontend loads all active and inactive price rows for that field. 3. Admin enters day type, start/end time, positive USD price, optional effective dates and status. 4. The backend validates the values and saves the row. 5. The UI refreshes pricing and public field detail shows active bands.		
Alternative Flow:	A1: Edit or deactivate rule 1. Admin selects an existing rule, changes values or status, and saves it. 2. Inactive rules no longer participate in calculation or public detail.		

	A2: No matching rule 1. Checkout calculation uses the hard-coded USD 12 fallback.
Exceptions:	EXC-1: Price must be greater than zero. EXC-2: Start time must precede end time. EXC-3: Day type must be all, weekday or weekend. EXC-4: Effective-from cannot be after effective-to. EXC-5: An active rule overlapping the same applicable day, time, and effective-date range is rejected.
Priority:	High
Frequency of Use:	Occasional; more often around pricing changes
Business Rules:	BR-02, BR-03, BR-06, BR-30
Other Information:	Implementation rule: Calculation matches active rows by weekday/weekend or all and containment of the slot time. It uses findFirst without explicit ordering and does not apply effective dates. Currency is USD throughout the code-first project.
Assumptions:	If no active price band matches, the code-first fallback is USD 12.

UC-15. Manage unavailable slots

UC ID and Name:	UC-15 Manage unavailable slots		
Created By:	BaoNG	Date Created:	30/05/2026
Primary Actor:	Admin or Staff	Admin and Staff	None
Trigger:	Admin changes automatic availability rules, or Staff blocks/unblocks an exceptional field period.		
Description:	Admin configures opening time, closing time, slot duration, and rolling horizon. Staff preserves maintenance, private-event, VIP/internal-use, and other exceptions through Block/Unblock.		
Preconditions:	PRE-1: Admin is authenticated to edit slot-generation rules, or Staff is authenticated to manage exceptions. PRE-2: The target field exists; Block requires date, start, end, and reason.		
Postconditions:	POST-1: A valid rule change rebuilds unbooked auto-generated availability while preserving bookings and blocks. POST-2: Block affects every overlapping slot without an active booking; Unblock restores one blocked row. POST-3: Customer search immediately reflects the resulting calendar.		
Normal Flow:	A. Configure automatic availability 1. Admin opens Policies and edits opening time, closing time, duration, or horizon. 2. The backend validates the combined rule set. 3. Unbooked auto-generated rows in the rolling window are rebuilt; booked and blocked rows are preserved. B. Block an exception 1. Staff opens Field schedule and chooses a field, date, time range, and reason. 2. The backend rejects the request if any overlapping slot has an active booking. 3. Every overlapping slot is marked blocked; if none exists, a future exception row is created.		

	<i>4. Calendar and public availability refresh.</i>
Alternative Flow:	<i>A1: Unblock exception</i> <i>1. Staff selects Unblock and the backend clears reason/note and restores availability.</i> <i>A2: Future private event</i> <i>1. Staff may create a blocked exception before that date enters the rolling horizon; later generation skips its range.</i>
Exceptions:	<i>EXC-1: Rule time must use HH:mm; opening must precede closing.</i> <i>EXC-2: Duration must be 30-360 minutes in 30-minute steps and divide the operating window exactly.</i> <i>EXC-3: Horizon must be 1-90 days.</i> <i>EXC-4: Past blocks, missing reasons, invalid ranges, and ranges containing active bookings are rejected.</i>
Priority:	<i>High</i>
Frequency of Use:	<i>Occasional; whenever maintenance, weather or internal use affects a field</i>
Business Rules:	<i>BR-02, BR-03, BR-09</i>
Other Information:	<i>Block/Unblock is intentionally retained for maintenance, weather, special events, VIP/internal use, and other operational exceptions. Manual Add Slot is not part of the normal workflow.</i>
Assumptions:	<i>Only unbooked rows originally generated by the system are replaceable when rules change.</i>

UC-16. View field operation calendar

UC ID and Name:	UC-16 View field operation calendar		
Created By:	BaoNG	Date Created:	30/05/2026
Primary Actor:	Staff or Admin	Secondary Actors:	None
Trigger:	Staff selects the Field schedule tab or changes its date.		
Description:	<i>Ensure the selected date is materialized from Admin slot rules, then display every field slot with available, blocked, or booking-derived operational status.</i>		
Preconditions:	<i>PRE-1: The caller has Staff or Admin backend authority; Staff owns the current React calendar.</i> <i>PRE-2: Slot-generation rules are valid for the requested date.</i>		
Postconditions:	<i>POST-1: The requested date exists in the rolling calendar and the operator sees available, blocked, and booked states.</i> <i>POST-2: Viewing does not change bookings or block exceptions.</i>		
Normal Flow:	<i>A. View field operation calendar</i> <i>1. Staff opens Field schedule or changes its date.</i> <i>2. The frontend requests GET /api/operations/calendar?date=... .</i> <i>3. The backend materializes any missing slots from Admin rules.</i> <i>4. It loads slots and bookings, sorts by field/time, and derives operationalStatus.</i> <i>5. The UI displays field, time, booking/customer or block reason, and status.</i>		
Alternative Flow:	<i>A1: Date omitted</i> <i>1. The backend defaults to today and generates remaining non-elapsd slots.</i> <i>A2: No active fields</i> <i>1. The calendar returns an empty list.</i>		

Exceptions:	<i>EXC-1: Unauthenticated and Customer callers are forbidden. EXC-2: Past or beyond-horizon operation dates are rejected by the generation rules.</i>
Priority:	<i>Medium</i>
Frequency of Use:	<i>Daily and throughout venue operation</i>
Business Rules:	<i>BR-02, BR-03</i>
Other Information:	<i>Implementation rule: booking status takes precedence over slot status in the operational display; Block/Unblock remains adjacent to this calendar.</i>
Assumptions:	<i>The calendar is a daily operations view rather than a month-grid planning engine.</i>

UC-17. Manage extra services

UC ID and Name:	<i>UC-17 Manage extra services</i>		
Created By:	<i>BaoNG</i>	Date Created:	<i>30/05/2026</i>
Primary Actor:	<i>Admin</i>	Secondary Actors:	<i>None</i>
Trigger:	<i>Admin opens the Services section or selects an existing add-on.</i>		
Description:	<i>Create, update, activate and deactivate rental, sale or staff-supported add-ons with pricing and optional stock/per-booking limits.</i>		
Preconditions:	<i>PRE-1: The user is authenticated as Admin.</i>		
Postconditions:	<i>POST-1: The service record is saved. POST-2: Only active services appear in public field detail and booking selection.</i>		
Normal Flow:	<i>A. Manage extra services</i> <i>1. Admin opens managed services.</i> <i>2. Admin enters name, type, unit, unit price, optional description, stock, per-booking maximum and status.</i> <i>3. The backend validates and saves the service.</i> <i>4. The frontend reloads the managed service catalogue.</i> <i>5. The managed list refreshes.</i>		
Alternative Flow:	<i>A1: Activate or deactivate</i> <i>1. Admin toggles the status and saves the same service.</i> <i>2. Inactive services disappear from public/booking service lists.</i> <i>A2: Edit an existing service</i> <i>1. Admin selects an existing row, changes its catalogue, price or inventory values, and updates it without deleting historical booking snapshots.</i>		
Exceptions:	<i>EXC-1: Name and unit are required.</i> <i>EXC-2: Unit price and stock cannot be negative.</i> <i>EXC-3: A per-booking maximum, when provided, must be at least one.</i> <i>EXC-4: There is no delete endpoint.</i>		
Priority:	<i>Medium</i>		
Frequency of Use:	<i>Occasional; catalogue maintenance</i>		
Business Rules:	<i>BR-02, BR-03, BR-10, BR-30</i>		
Other Information:	<i>Implementation rule: Allowed service types are rental, sale and staff_service. Null stock means no shared-inventory cap.</i>		

	<i>Services are global; there is no field-to-service mapping.</i>
Assumptions:	<i>Deactivating a service does not delete historical booking-service snapshots.</i>

UC-18. Check extra service availability

UC ID and Name:	UC-18 Check extra service availability		
Created By:	BaoNG	Date Created:	30/05/2026
Primary Actor:	Customer or Staff	Secondary Actors:	None
Trigger:	Checkout preview, booking creation or pre-check-in service editing submits add-on quantities.		
Description:	Validate that each selected service is active, within its per-booking maximum, and within shared stock after counting quantities reserved by overlapping active bookings.		
Preconditions:	PRE-1: The selected slot and service IDs exist. PRE-2: Requested quantities are submitted or default to one.		
Postconditions:	POST-1: Valid service totals are calculated from current unit prices. POST-2: Unavailable selections are rejected before booking/service changes are committed.		
Normal Flow:	A. Check extra service availability 1. The actor selects service quantities. 2. The system loads each service and validates active status and quantity. 3. For finite stock, it sums the service quantity on overlapping pending, confirmed and checked-in bookings. 4. The requested quantity is added to the reserved quantity and checked against stock. 5. The system returns the rounded service total.		
Alternative Flow:	A1: Unlimited inventory 1. When stockQuantity is null, only active status and per-booking maximum are checked. A2: Editing current booking 1. The current booking ID is excluded from the reserved-stock aggregate before validating its replacement quantities.		
Exceptions:	EXC-1: Inactive service, excessive quantity or insufficient overlapping inventory is rejected. EXC-2: An unknown service ID returns not found. EXC-3: Duplicate service IDs in one raw request are not normalized before validation and can ultimately violate the booking-service uniqueness constraint.		
Priority:	Medium		
Frequency of Use:	Every checkout preview, booking creation and service amendment		
Business Rules:	BR-10		
Other Information:	Implementation rule: Inventory overlap uses same date and intersecting start/end time. Only pending, confirmed and checked_in bookings reserve inventory. Staff-service capacity uses the same optional numeric stock model; there is no separate staff roster/schedule.		

Assumptions:	<i>Clients send each service at most once per request.</i>
---------------------	--

UC-19. Add extra services to booking

UC ID and Name:	<i>UC-19 Add extra services to booking</i>		
Created By:	<i>BaoNG</i>	Date Created:	<i>30/05/2026</i>
Primary Actor:	<i>Customer (online) or Staff (walk-in)</i>	Secondary Actors:	<i>None</i>
Trigger:	<i>The actor selects add-ons before creating a booking.</i>		
Description:	<i>Include selected active add-ons in online customer checkout or Staff walk-in checkout, snapshot unit price/quantity/line total, and include the service total in booking and invoice calculations.</i>		
Preconditions:	<i>PRE-1: A slot has been selected. PRE-2: Each selected service passes UC-18 availability checks. PRE-3: The actor is a Customer creating their own online booking or an operator creating a walk-in.</i>		
Postconditions:	<i>POST-1: BookingServiceItem rows are stored with price snapshots. POST-2: Booking total, discounts, deposit and remaining amount include service value. POST-3: The generated invoice includes service amount after payment/completion.</i>		
Normal Flow:	<i>A. Add extra services to booking 1. The actor chooses services and quantities in checkout. 2. Checkout preview validates them and recalculates service and total amounts. 3. The actor creates the online or walk-in booking. 4. The backend revalidates services and stores one snapshot item per selection. 5. Payment capture or completion creates/updates an invoice with the service amount.</i>		
Alternative Flow:	<i>A1: No add-ons 1. The booking is created with serviceTotalAmount equal to zero and no booking-service rows. A2: Promotion depends on service 1. The selected service list is also used to validate a required-service promotion.</i>		
Exceptions:	<i>EXC-1: Any UC-18 validation failure rejects the preview/create request. EXC-2: Inactive services cannot be newly added even if their ID is known.</i>		
Priority:	<i>Medium</i>		
Frequency of Use:	<i>Frequent; optional per booking</i>		
Business Rules:	<i>BR-10, BR-11</i>		
Other Information:	<i>Implementation rule: Service prices are snapshotted at booking time. Membership discount is applied only to online bookings; walk-ins receive no membership discount. The Staff UI combines walk-in booking creation with immediate cash deposit/full capture.</i>		

Assumptions:	<i>The selected service catalogue remains unchanged between preview and creation; the backend revalidates if it changes.</i>
---------------------	--

UC-20. Update extra services before check-in

UC ID and Name:	UC-20 Update extra services before check-in		
Created By:	BaoNG	Date Created:	30/05/2026
Primary Actor:	Owning Customer or Staff	Secondary Actors:	None
Trigger:	<i>The booking owner or operator saves a replacement set of add-ons.</i>		
Description:	<i>Replace all booking-service selections while the booking is pending or confirmed, then recalculate services, promotion, membership discount, total, deposit, remaining/refundable delta and invoice.</i>		
Preconditions:	<i>PRE-1: The booking exists and is pending or confirmed. PRE-2: A Customer owns the booking, or the caller is Staff/Admin. PRE-3: The replacement selections pass active, quantity and overlapping-inventory checks.</i>		
Postconditions:	<i>POST-1: Old booking-service rows are replaced by the submitted set. POST-2: All monetary snapshots and the invoice are updated in the same transaction. POST-3: The customer receives an in-app booking notification.</i>		
Normal Flow:	<i>A. Update extra services before check-in 1. The actor opens a pending/confirmed booking and adjusts quantities. 2. The frontend submits PUT /api/bookings/{id}/services with the complete selection list. 3. The backend checks visibility and lifecycle state, then validates inventory excluding the current booking. 4. The system recalculates applicable discounts and total/balance/refundable delta. 5. The system replaces service rows, updates the invoice and notifies the customer.</i>		
Alternative Flow:	<i>A1: Remove all services 1. An empty/null list deletes all booking-service rows and reprices the booking. A2: New total below paid amount 1. Remaining becomes zero and refundableAmount becomes the paid-minus-total difference. 2. A refund is not automatic; it can be requested separately.</i>		
Exceptions:	<i>EXC-1: Checked-in, completed, cancelled, rejected, expired and no-show bookings cannot be edited. EXC-2: A Customer cannot edit another customer's booking. EXC-3: Unavailable/inactive service selections are rejected.</i>		
Priority:	Medium		
Frequency of Use:	Occasional; before check-in		
Business Rules:	BR-10, BR-11		
Other Information:	<i>Implementation rule: The submitted list is a full replacement. Promotion eligibility is recalculated against the proposed services;</i>		

	membership discount remains online-only. Both Customer Account and Staff workspace expose editors; Admin has backend authority but no matching editor.
Assumptions:	Any refundable delta is handled by the separate refund workflow.

UC-21. Report field/service issue

UC ID and Name:	UC-21 Report field/service issue		
Created By:	BaoNG	Date Created:	30/05/2026
Primary Actor:	Customer or Staff	Secondary Actors:	None
Trigger:	An authenticated user submits an issue report.		
Description:	Create an issue under the authenticated reporter, optionally linked to a booking, field and/or extra service. Operators may assign the issue to a Staff account at creation.		
Preconditions:	PRE-1: The reporter is authenticated. PRE-2: The issue title is provided. PRE-3: A Customer-linked booking, if present, belongs to that customer.		
Postconditions:	POST-1: An open Issue row is persisted under the authenticated reporter. POST-2: Staff/Admin can see it in the issue list.		
Normal Flow:	A. Report field/service issue 1. The reporter opens the issue form and provides title/description. 2. The reporter optionally relates the issue to a permitted booking or field. 3. The backend derives the reporter from authentication and verifies booking ownership for Customers. 4. The backend resolves any supplied booking, field, service or Staff assignment references. 5. The issue is saved with status open.		
Alternative Flow:	A1: General issue 1. No booking, field or service association is supplied; the issue is still accepted. A2: Operator report/assignment 1. Staff/Admin may report against any booking and may assign the issue to an existing Staff user.		
Exceptions:	EXC-1: Unauthenticated requests are rejected. EXC-2: A Customer cannot report against another customer's booking. EXC-3: An assignment target whose role is not Staff is rejected. EXC-4: The backend permits blank description and no association, although the React Customer/Staff forms require a description.		
Priority:	Medium		
Frequency of Use:	Occasional; event-driven		
Business Rules:	BR-02, BR-29		
Other Information:	Implementation rule: Reporter identity is always the authenticated principal. Only operators may assign at creation. Customers cannot retrieve the global issue list.		

	Customer UI supports booking, field or general reports; direct extra-service linking exists only at API level.
Assumptions:	An issue may legitimately relate to more than one optional target in raw API usage.

UC-22. Resolve field/service issue

UC ID and Name:	UC-22 Resolve field/service issue		
Created By:	BaoNG	Date Created:	30/05/2026
Primary Actor:	Staff	Secondary Actors:	Customer
Trigger:	An operator starts, resolves or rejects an issue.		
Description:	Update issue state, optionally assign it to a Staff account, require a resolution note when closing, record/clear resolved time, and notify the reporter on every update.		
Preconditions:	PRE-1: The issue exists. PRE-2: The caller has Staff or Admin authority.		
Postconditions:	POST-1: Issue status, assignment, resolution note and resolved timestamp reflect the update. POST-2: The reporter receives an in-app issue notification.		
Normal Flow:	A. Resolve field/service issue 1. Staff opens Issues & refunds and reviews open/in-progress issues. 2. Staff optionally starts the issue, assigning it to themselves in the current UI. 3. Staff enters a resolution note and selects Resolve. 4. The backend validates the closing note and records resolvedAt. 5. The backend stores resolved status/time and sends a reporter notification.		
Alternative Flow:	A1: Reject issue 1. An API operator submits rejected with a required resolution note. 2. The issue is closed and the reporter is notified. A2: Reopen/change state 1. An operator submits open or in_progress. 2. resolvedAt is cleared; there is no transition guard.		
Exceptions:	EXC-1: Customer callers are forbidden. EXC-2: Resolved/rejected updates without a nonblank resolution note are rejected. EXC-3: Only a Staff-role user can be assigned; an Admin account cannot be the assignee.		
Priority:	Medium		
Frequency of Use:	Daily or as reports arrive		
Business Rules:	BR-02, BR-03, BR-29		
Other Information:	Implementation rule: Statuses are open, in_progress, resolved and rejected. The service permits transitions between any statuses; the React Staff UI exposes Start and Resolve only. There is no service replacement or refund automatically coupled to issue resolution; operators use separate booking/refund actions.		
Assumptions:	Resolution notes contain enough operational detail for later review.		

Booking Lifecycle

UC-23. Create online booking

UC ID and Name:	UC-23 Create online booking		
Created By:	NgocPA	Date Created:	30/05/2026
Primary Actor:	Customer	Secondary Actors:	PayPal Sandbox
Trigger:	A verified Customer starts PayPal checkout after selecting a slot and payment option.		
Description:	Create a Customer-owned online booking as a pending slot hold, snapshot pricing/services/discounts/deposit, then continue to customer-only PayPal deposit/full capture, which confirms the booking.		
Preconditions:	PRE-1: The actor is authenticated as the same Customer identified by customerId. PRE-2: The Customer email is verified and `status` is `active`. PRE-3: The slot exists, is not blocked, is not before today's date, and has no active booking. PRE-4: Selected services and promotion are valid.		
Postconditions:	POST-1: A pending online booking and any service/promotion snapshots are stored. POST-2: The slot is reserved against another active booking. POST-3: The Customer receives a Booking created in-app notification. POST-4: After valid PayPal capture of at least the deposit, the booking is confirmed and invoiced.		
Normal Flow:	A. Create online booking 1. Customer searches and selects an available slot, add-ons, promotion and deposit/full option. 2. The public preview calculates the checkout amounts. 3. The frontend creates a pending online booking before opening PayPal. 4. The backend revalidates identity, account lock, verification, slot, services and pricing and stores snapshots. 5. Customer authorizes sandbox payment; server-side capture validates provider amount/currency. 6. Payment updates paid/remaining amounts, confirms the pending booking and generates the invoice.		
Alternative Flow:	A1: PayPal cancelled 1. The cancellation endpoint expires the pending booking/payment hold. 2. The slot is released for another booking. A2: Pending hold awaiting payment 1. Booking creation succeeds but payment has not yet captured. 2. The booking remains pending until capture, explicit cancellation or maintenance expiry.		
Exceptions:	EXC-1: Staff/Admin cannot create online bookings; Customers cannot create walk-ins. EXC-2: Unverified, locked, or wrong-owner Customer requests are forbidden.		

	EXC-3: Blocked, past-date or actively reserved slots are rejected. EXC-4: A same-day slot whose start time has elapsed is rejected. EXC-5: Database active-slot uniqueness is the concurrency backstop for simultaneous requests.
Priority:	High
Frequency of Use:	Frequent; core customer transaction
Business Rules:	BR-04, BR-05, BR-06, BR-07, BR-08, BR-09, BR-10, BR-21, BR-22
Other Information:	Implementation rule: Online source starts pending. Membership discount applies only online. Active statuses reserve a slot. Deposit percent comes from system settings. Payment is a subsequent operation, not atomic with booking creation. Online payment is Customer-only and PayPal Sandbox-only in current scope.
Assumptions:	Normal UI navigation supplies an active-field, current/future available slot.

UC-24. Create walk-in booking

UC ID and Name:	UC-24 Create walk-in booking		
Created By:	NgocPA	Date Created:	30/05/2026
Primary Actor:	Staff	Secondary Actors:	Customer
Trigger:	Staff uses the venue booking desk for an existing customer.		
Description:	Create a walk-in booking for an existing customer as immediately confirmed, without membership discount, and normally record a cash deposit or full payment immediately afterward.		
Preconditions:	PRE-1: The caller is Staff in the UI, or Staff/Admin at API level. PRE-2: An existing Customer and a bookable slot are selected. PRE-3: The Customer account is not locked. PRE-4: Selected services and promotion are valid.		
Postconditions:	POST-1: A confirmed walk_in booking is stored and assigned to the operating staff/operator. POST-2: The Customer receives a Booking created notification. POST-3: In the Staff UI, cash deposit/full payment is captured and an invoice is generated.		
Normal Flow:	A. Create walk-in booking 1. Staff opens Walk-in booking and selects customer, slot, services, promotion and deposit/full cash option. 2. The backend validates operator/source/customer/slot and creates a confirmed walk-in. 3. The backend stores service/promotion snapshots and assigns the operating staff. 4. The Staff frontend immediately calls counter-payment capture using cash. 5. The system records paid/remaining values, generates the invoice and navigates to Staff operations.		
Alternative Flow:	A1: API creation without payment 1. An operator calls booking creation but does not call cash capture. 2. The booking remains confirmed with zero paid and the full amount		

	remaining. A2: Admin API operation 1. An Admin may create the walk-in and record cash through protected APIs, although Admin checkout is intentionally hidden in React.
Exceptions:	EXC-1: Customers cannot create walk-in bookings. EXC-2: A locked customer, blocked/past-date/actively reserved slot, invalid service or invalid promotion is rejected. EXC-3: Walk-in does not require the customer's email to be verified. EXC-4: Cash capture failure can leave the already confirmed booking unpaid.
Priority:	Medium
Frequency of Use:	Frequent at the venue
Business Rules:	BR-05, BR-06, BR-07, BR-08, BR-09, BR-10, BR-12
Other Information:	Implementation rule: Walk-in bookings are confirmed at creation. Only cash is accepted by the counter-payment endpoint. Membership discount is zero for walk-ins. The current product decision says Staff handles the counter flow; Admin backend permission is broader than the React experience.
Assumptions:	Staff selects the correct existing Customer; walk-in customer creation is not part of this use case.

UC-25. View booking detail

UC ID and Name:	UC-25 View booking detail		
Created By:	NgocPA	Date Created:	30/05/2026
Primary Actor:	Owning Customer, Staff, or Admin	Secondary Actors:	None
Trigger:	The actor selects a booking in Customer Account or Staff operations.		
Description:	Return role/ownership-protected booking summary plus add-ons, payments, promotions, invoice, refunds and derived payment status.		
Preconditions:	PRE-1: The actor is authenticated. PRE-2: The booking exists. PRE-3: A Customer owns the booking, or the actor is Staff/Admin.		
Postconditions:	POST-1: The permitted actor sees current booking and billing detail. POST-2: No data is changed.		
Normal Flow:	A. View booking detail 1. The actor selects a booking. 2. The frontend requests GET /api/bookings/{bookingId}. 3. The service checks Customer ownership or operator role. 4. The service aggregates booking, service items, transactions, promotions, invoice, refunds and payment state. 5. The appropriate account/staff panels render the result.		
Alternative Flow:	A1: No invoice or related records 1. The response contains null invoice and/or empty lists while still returning the booking summary. A2: Operator view 1. Staff/Admin may open any booking; the ownership comparison is		

	<i>skipped for operator roles.</i>
Exceptions:	<i>EXC-1: A Customer requesting another customer's booking receives forbidden. EXC-2: A nonexistent booking returns not found.</i>
Priority:	<i>Low</i>
Frequency of Use:	<i>Frequent; throughout booking and venue workflows</i>
Business Rules:	<i>BR-02, BR-03</i>
Other Information:	<i>Implementation rule: Staff/Admin can view any booking. Payment status is derived from completed refunds, booking expiry, failed payments, paid and remaining amounts. The endpoint is authenticated by SecurityConfig; seeding code may call the same service internally without HTTP authentication.</i>
Assumptions:	<i>Historical monetary fields are trustworthy booking snapshots.</i>

UC-26. View my bookings

UC ID and Name:	<i>UC-26 View my bookings</i>		
Created By:	<i>NgocPA</i>	Date Created:	<i>30/05/2026</i>
Primary Actor:	<i>Customer</i>	Secondary Actors:	<i>None</i>
Trigger:	<i>Customer opens My bookings/Account.</i>		
Description:	<i>List the authenticated Customer's bookings in descending booking ID order and allow selection for detail, billing, cancellation, reschedule, add-on and issue actions.</i>		
Preconditions:	<i>PRE-1: The Customer is authenticated.</i>		
Postconditions:	<i>POST-1: Only that Customer's booking summaries are listed. POST-2: No data changes merely by viewing the list.</i>		
Normal Flow:	<i>A. View my bookings</i> <i>1. Customer opens Account.</i> <i>2. The frontend requests the bookings endpoint with the current customer ID.</i> <i>3. The service forces non-operator callers to their own user ID.</i> <i>4. The repository returns that Customer's bookings in descending booking ID order.</i> <i>5. The UI displays all returned current and historical bookings and loads the selected booking detail.</i>		
Alternative Flow:	<i>A1: No bookings</i> <i>1. The Account page displays a no-bookings message and a path to start booking.</i> <i>A2: Select a different booking</i> <i>1. Customer selects another row and the application loads its protected booking detail and related actions.</i>		
Exceptions:	<i>EXC-1: A Customer cannot request another customer's list. EXC-2: The implemented Customer list has no date/status filtering; supplying date does not narrow the result because customer lookup takes precedence.</i>		
Priority:	<i>Low</i>		
Frequency of Use:	<i>Frequent; before and after bookings</i>		

Business Rules:	BR-02
Other Information:	Implementation rule: Customer scoping is enforced server-side regardless of the customerId submitted by the client. The Account page organizes actions in tabs but does not expose booking list filters.
Assumptions:	Descending booking ID approximates newest-first creation order.

UC-27. View booking calendar

UC ID and Name:	UC-27 View booking calendar		
Created By:	NgocPA	Date Created:	30/05/2026
Primary Actor:	Staff	Secondary Actors:	None
Trigger:	Staff opens the booking queue or selects a date in the operational schedule.		
Description:	Support daily booking monitoring through the Staff booking list and the date-based operation calendar, which presents booking status alongside each slot.		
Preconditions:	PRE-1: Staff is authenticated in the React workspace, or Staff/Admin calls the protected APIs.		
Postconditions:	POST-1: Staff can select a booking for lifecycle actions and billing detail. POST-2: No booking changes merely by viewing.		
Normal Flow:	A. View booking calendar 1. Staff opens Staff workspace. 2. The application loads the operator-visible booking list. 3. Staff selects a booking to view detail and lifecycle controls. 4. The application displays the selected booking's invoice/payment information and available state transitions. 5. For a field/date schedule, Staff opens Field schedule and the app loads /api/operations/calendar for that date.		
Alternative Flow:	A1: Date-specific API list 1. An operator may request GET /api/bookings?date=YYYY-MM-DD with no customerId to return bookings ordered by slot start time. A2: Empty queue/date 1. The UI shows an empty state and links to walk-in booking or field schedule.		
Exceptions:	EXC-1: Customers are limited to their own booking list and cannot use the operation calendar. EXC-2: Admin lacks the Staff calendar UI despite backend operator access.		
Priority:	Medium		
Frequency of Use:	Continuous/daily for venue staff		
Business Rules:	BR-02, BR-03		
Other Information:	Implementation rule: Operator list without customer/date returns all bookings newest first; date-specific operation calendar is slot-centric. The implementation is list/calendar-like rather than a graphical month/week calendar.		
Assumptions:	Staff uses the field schedule for the operational date view and		

	booking list for the broader queue.
--	-------------------------------------

UC-28. Confirm booking

UC ID and Name:	UC-28 Confirm booking		
Created By:	NgocPA	Date Created:	30/05/2026
Primary Actor:	Customer or Staff	Secondary Actors:	None
Trigger:	A pending booking reaches the required deposit through a valid capture, or an operator explicitly selects Confirm.		
Description:	Move a pending booking to confirmed after qualifying payment or operator approval and record confirmedAt. Walk-ins are already confirmed at creation.		
Preconditions:	PRE-1: The booking exists and is pending for an explicit status transition. PRE-2: For automatic confirmation, paidAmount after capture is at least depositAmount. PRE-3: For manual confirmation, the caller is Staff/Admin.		
Postconditions:	POST-1: Booking status is confirmed and confirmedAt is stored. POST-2: A manual confirm sends a booking-confirmation notification; payment capture sends a payment notification and updates invoice/amounts.		
Normal Flow:	A. Confirm booking 1. Customer captures PayPal deposit/full payment, or Staff records a valid cash payment on a pending booking. 2. The payment service updates paid and remaining amounts. 3. When paid amount meets the deposit, the service changes pending to confirmed and records confirmedAt. 4. The system stores the payment transaction and sends the corresponding payment notification. 5. The invoice is generated/updated.		
Alternative Flow:	A1: Staff manual approval 1. Staff selects Confirm for a pending booking. 2. The booking workflow validates pending-to-confirmed, stores the operator and sends confirmation notification even without checking payment. A2: Walk-in 1. Walk-in creation starts confirmed; no pending-to-confirmed action occurs.		
Exceptions:	EXC-1: Only pending bookings can explicitly transition to confirmed. EXC-2: Customers cannot invoke the status-confirm action. EXC-3: Invalid/failed payment does not increase paid amount or confirm the booking.		
Priority:	High		
Frequency of Use:	Per successful online booking or manual approval		
Business Rules:	BR-12, BR-23		
Other Information:	Implementation rule: The lifecycle transition is pending to confirmed. Operator manual confirmation is allowed independently of paid amount; automatic capture requires paidAmount >= depositAmount.		

	Automatic payment confirmation and manual workflow produce slightly different notification types/messages.
Assumptions:	Staff uses manual confirmation only when venue policy justifies approval without captured payment.

UC-29. Reject booking request

UC ID and Name:	UC-29 Reject booking request		
Created By:	NgocPA	Date Created:	30/05/2026
Primary Actor:	Staff	Secondary Actors:	Customer
Trigger:	An operator determines a pending booking request should not proceed.		
Description:	Change a pending booking to rejected, optionally store an operator/note, notify the customer and release the slot from active reservation.		
Preconditions:	PRE-1: The booking exists and is pending. PRE-2: The caller is Staff/Admin.		
Postconditions:	POST-1: Booking status is rejected. POST-2: The slot is no longer reserved by an active booking. POST-3: The customer receives a rejection notification.		
Normal Flow:	A. Reject booking request 1. Staff selects a pending booking in Lifecycle actions. 2. Staff selects Reject. 3. The backend validates the transition, associates the operating staff and stores the supplied note. 4. The backend changes the booking out of the active slot-reserving states. 5. The system changes status to rejected and notifies the Customer.		
Alternative Flow:	A1: Admin API rejection 1. Admin submits the same protected status update through the API. A2: Slot reused later 1. Because rejected is not active, a later valid booking can reserve the released slot.		
Exceptions:	EXC-1: Confirmed, checked-in and terminal bookings cannot transition to rejected. EXC-2: Customers cannot reject a booking. EXC-3: A rejection reason is not mandatory in the backend.		
Priority:	Medium		
Frequency of Use:	Occasional		
Business Rules:	BR-13		
Other Information:	Implementation rule: Rejected is terminal in requireValidTransition and is not an active slot-reserving status. No rejectedAt timestamp is modelled.		
Assumptions:	The Staff UI's default note is sufficient unless a more specific operational note is needed.		

UC-30. Reschedule booking

UC ID and Name:	<i>UC-30 Reschedule booking</i>		
Created By:	<i>NgocPA</i>	Date Created:	<i>30/05/2026</i>
Primary Actor:	<i>Owning Customer or Staff</i>	Secondary Actors:	<i>Customer when Staff reschedules on the Customer's behalf</i>
Trigger:	<i>The permitted actor selects a different available slot for an unstarted booking.</i>		
Description:	<i>Move a pending or confirmed booking to a different slot while preserving its booking/payment history, repricing the field and discounts, calculating remaining/refundable delta, updating the invoice and notifying the customer.</i>		
Preconditions:	<i>PRE-1: The booking is pending or confirmed.</i> <i>PRE-2: The Customer owns it, or the caller is Staff/Admin.</i> <i>PRE-3: The new slot is different, not blocked, not before today's date, and has no active booking.</i>		
Postconditions:	<i>POST-1: The booking references the new slot and the old slot is released.</i> <i>POST-2: Field price, discounts, total, deposit, remaining/refundable amounts and invoice are recalculated.</i> <i>POST-3: The customer receives a reschedule notification.</i>		
Normal Flow:	<i>A. Reschedule booking</i> <i>1. The actor selects a pending/confirmed booking and a different available slot.</i> <i>2. The frontend submits PUT /api/bookings/{id}/reschedule.</i> <i>3. The backend checks visibility/state and validates the new slot.</i> <i>4. The system recalculates field price, promotion applicability, membership discount and monetary delta while preserving paid amount/history.</i> <i>5. The system changes the slot, updates the invoice and notifies the Customer.</i>		
Alternative Flow:	<i>A1: New total is higher</i> <i>1. remainingAmount is the positive difference and the notification states the balance due.</i> <i>A2: New total is lower</i> <i>1. refundableAmount is the positive paid-minus-new-total difference.</i> <i>2. A separate refund request is required.</i> <i>A3: Promotion no longer applies</i> <i>1. Its booking-promotion snapshot discount is set to zero for recalculation.</i>		
Exceptions:	<i>EXC-1: The same slot, blocked/past-date/reserved slot, unauthorized owner or non-pending/non-confirmed state is rejected.</i> <i>EXC-2: Inactive-field and same-day elapsed-time checks are missing for a directly submitted new slot.</i> <i>EXC-3: Current service quantities must remain within overlapping inventory at the destination slot.</i>		
Priority:	<i>High</i>		
Frequency of	<i>Occasional</i>		

Use:	
Business Rules:	BR-04, BR-06, BR-07, BR-08, BR-09
Other Information:	Implementation rule: Original Booking and Payment rows are retained. Membership discounts are online-only. The reschedule itself does not capture additional money or create a refund. Customer Account and Staff workspace expose the action; Admin has backend authority only.
Assumptions:	The actor separately settles a balance or requests an available refund after reschedule.

UC-31. Preview cancellation fee/refund

UC ID and Name:	UC-31 Preview cancellation fee/refund		
Created By:	NgocPA	Date Created:	30/05/2026
Primary Actor:	Owning Customer or Staff	Secondary Actors:	None
Trigger:	A permitted actor reviews cancellation terms before cancelling.		
Description:	Calculate, without mutating the booking, hours before start, policy band, refund percentage, fee and refundable amount based on the amount already paid.		
Preconditions:	PRE-1: The booking exists and is pending or confirmed. PRE-2: A Customer owns it, or the caller is Staff/Admin.		
Postconditions:	POST-1: A cancellation preview is returned; the booking remains unchanged. POST-2: The Customer UI requires this review before showing its Cancel action.		
Normal Flow:	A. Preview cancellation fee/refund 1. Customer selects Review cancellation terms. 2. The frontend requests GET /api/bookings/{id}/cancellation-preview. 3. The service computes whole hours from now to slot start. 4. The service selects the configured >=24-hour percentage, configured 0-23-hour percentage, or zero after start. 5. The UI displays refundable amount and cancellation fee and enables confirmation.		
Alternative Flow:	A1: At least 24 hours 1. Use refund.before_24h_percent, default 100 percent. A2: Less than 24 hours but before start 1. Use refund.same_day_percent, default 80 percent. A3: Start time passed 1. Refund percentage and refundable amount are zero; fee equals paid amount.		
Exceptions:	EXC-1: Only pending/confirmed bookings can be previewed. EXC-2: A Customer cannot preview another customer's booking. EXC-3: Invalid numeric setting values can cause calculation failure.		
Priority:	Medium		
Frequency of Use:	Occasional; before eligible cancellations		
Business Rules:	BR-16, BR-17, BR-18		
Other	Implementation rule: Refund and fee are calculated from paidAmount,		

Information:	<i>not totalAmount. Preview is read-only and does not reserve/refund money. Staff UI can directly cancel without displaying this preview; Customer UI deliberately previews first.</i>
Assumptions:	<i>System-setting percentages are valid values between 0 and 100, although this method does not itself clamp them.</i>

UC-32. Cancel booking

UC ID and Name:	UC-32 Cancel booking		
Created By:	NgocPA	Date Created:	30/05/2026
Primary Actor:	Owning Customer or Staff	Secondary Actors:	Customer when Staff cancels on the Customer's behalf
Trigger:	The Customer confirms reviewed cancellation terms or an operator selects Cancel.		
Description:	Cancel a pending or confirmed booking, snapshot fee/refundable amount using the current policy, notify the customer and release the slot. Refund processing remains a separate request/review flow.		
Preconditions:	PRE-1: The booking is pending or confirmed. PRE-2: The Customer owns it, or the caller is Staff/Admin.		
Postconditions:	POST-1: Booking status is cancelled and cancelledAt is recorded. POST-2: Cancellation fee and refundable amount are stored. POST-3: The slot becomes available to new bookings and the customer receives a notification.		
Normal Flow:	A. Cancel booking 1. The permitted actor requests status cancelled. 2. The backend checks ownership/operator authority and validates the lifecycle transition. 3. The service calculates and stores cancellation fee/refundable amount. 4. The service records cancellation time/status and sends an in-app notification. 5. If refundableAmount is positive, the Customer or operator may create a separate refund request.		
Alternative Flow:	A1: No payment/refund 1. With paidAmount zero, fee and refundable amount are both zero and no refund request is available. A2: Staff cancellation 1. Staff uses Lifecycle actions; the operating staff and note are recorded.		
Exceptions:	EXC-1: Checked-in, completed, cancelled, rejected, expired and no-show bookings cannot be cancelled. EXC-2: A Customer cannot cancel another customer's booking. EXC-3: Cancellation does not automatically create or complete a Refund row.		
Priority:	High		
Frequency of	Occasional		

Use:	
Business Rules:	BR-16, BR-17, BR-18, BR-28
Other Information:	Implementation rule: Allowed transitions are pending-to-cancelled and confirmed-to-cancelled. Cancelled is terminal and no longer reserves the slot. Customer UI enforces preview-first presentation; the API does not require a separate prior preview call.
Assumptions:	Eligible refundable money is handled by UC-46/47 after cancellation.

UC-33. Check-in booking

UC ID and Name:	UC-33 Check-in booking		
Created By:	NgocPA	Date Created:	30/05/2026
Primary Actor:	Staff	Secondary Actors:	None
Trigger:	Staff confirms that the Customer has arrived.		
Description:	Move a confirmed booking to checked_in and record checkedInAt and the operating staff.		
Preconditions:	PRE-1: The caller is Staff/Admin. PRE-2: The booking exists and is confirmed.		
Postconditions:	POST-1: Booking status is checked_in. POST-2: checkedInAt and operator association are recorded.		
Normal Flow:	A. Check-in booking 1. Staff selects the confirmed booking. 2. Staff selects Check-in. 3. The backend validates confirmed-to-checked_in. 4. The backend records checkedInAt and resolves the operating Staff association. 5. The system stores the new status and returns refreshed booking detail.		
Alternative Flow:	A1: Admin API check-in 1. Admin may submit the protected status update without a corresponding Admin UI control. A2: Staff note 1. When a nonblank operation note is supplied, it replaces the booking note during check-in.		
Exceptions:	EXC-1: Only confirmed bookings can be checked in. EXC-2: Customers cannot perform check-in. EXC-3: The service does not enforce a check-in time window relative to slot start.		
Priority:	Medium		
Frequency of Use:	Per attended booking		
Business Rules:	BR-14		
Other Information:	Implementation rule: checked_in remains an active slot-reserving status and is the sole predecessor of completed. No notification is sent specifically for check-in.		
Assumptions:	Venue staff verifies identity/arrival outside the system.		

UC-34. Complete booking

UC ID and Name:	UC-34 Complete booking		
Created By:	NgocPA	Date Created:	30/05/2026
Primary Actor:	Staff	Secondary Actors:	Customer
Trigger:	Staff records that field use has finished.		
Description:	Move a checked-in booking to completed, record completedAt, re-evaluate membership under configured qualification periods, and generate/update the invoice.		
Preconditions:	PRE-1: The caller is Staff/Admin. PRE-2: The booking exists and is checked_in.		
Postconditions:	POST-1: Booking status is completed and completedAt is recorded. POST-2: Customer membership count/level is recalculated from completed bookings. POST-3: The invoice is generated or updated.		
Normal Flow:	A. Complete booking 1. Staff selects a checked-in booking and selects Complete. 2. The backend validates checked_in-to-completed. 3. The system stores completion time/status and operating staff. 4. The membership service recounts only completed bookings for the Customer. 5. The system selects the highest eligible membership level and generates the invoice.		
Alternative Flow:	A1: Default membership missing for customer 1. The system attaches the first configured membership level before updating progress. A2: Outstanding balance 1. Completion still succeeds when remainingAmount is positive; the invoice preserves the outstanding balance.		
Exceptions:	EXC-1: A booking not checked in cannot be completed. EXC-2: Customers cannot complete bookings. EXC-3: If no membership levels are configured, default membership attachment fails.		
Priority:	Medium		
Frequency of Use:	Per attended booking		
Business Rules:	BR-15, BR-20		
Other Information:	Implementation rule: Only completed bookings count. Membership eligibility uses each level's lifetime, monthly, or consecutive-week rule. CustomerMembership.completedBookingCount remains the lifetime audit count while eligibility may use a narrower configured period.		
Assumptions:	Any remaining payment is operationally handled before or independently of completion; the status transition does not require remainingAmount to be zero.		

UC-35. Mark no-show

UC ID and Name:	UC-35 Mark no-show
------------------------	--------------------

Created By:	NgocPA	Date Created:	30/05/2026
Primary Actor:	Staff	Secondary Actors:	None
Trigger:	Staff determines that the Customer did not arrive for a confirmed booking.		
Description:	Move a confirmed booking to terminal no_show state and record the operating staff/note. Existing paid money remains on the booking, and no-show bookings are excluded from membership completion counts.		
Preconditions:	PRE-1: The caller is Staff/Admin. PRE-2: The booking exists and is confirmed.		
Postconditions:	POST-1: Booking status is no_show and the slot is no longer active-reserved. POST-2: The booking will not count as completed membership progress. POST-3: Existing payment amounts remain unchanged.		
Normal Flow:	A. Mark no-show 1. Staff selects a confirmed booking after the expected arrival. 2. Staff selects No-show. 3. The backend validates confirmed-to-no_show. 4. The backend records the operating Staff and any nonblank note. 5. The booking leaves the active slot-reserving states and updated detail is returned.		
Alternative Flow:	A1: Paid walk-in no-show 1. Captured cash remains recorded; the transition does not calculate a refund or reduce paidAmount. A2: Admin API action 1. Admin may mark a confirmed booking no-show through the protected status API, although the Admin console has no lifecycle control.		
Exceptions:	EXC-1: Only confirmed bookings can be marked no-show. EXC-2: Customers cannot mark no-show. EXC-3: No arrival-time threshold is enforced. EXC-4: There is no explicit deposit-forfeiture calculation, noShowAt timestamp or notification.		
Priority:	Medium		
Frequency of Use:	Occasional		
Business Rules:	BR-19, BR-20		
Other Information:	Implementation rule: no_show is terminal, does not reserve the slot, and is never counted by membership progress because only completed status is counted. The practical loss of deposit is represented only by leaving captured money unchanged and providing no automatic refund.		
Assumptions:	Staff applies venue policy and chooses the action at the appropriate time.		

UC-36. Handle booking conflict

UC ID and Name:	UC-36 Handle booking conflict
------------------------	-------------------------------

Created By:	NgocPA	Date Created:	30/05/2026
Primary Actor:	Staff	Secondary Actors:	Customer
Trigger:	Staff observes a field/slot problem affecting an existing booking or a booking attempt encounters an occupied slot.		
Description:	Prevent new double-booking and let Staff compose existing operation-calendar, issue, reschedule, cancel, refund and notification capabilities to resolve an operational conflict. No dedicated conflict record/workflow exists.		
Preconditions:	PRE-1: For prevention, a booking/reschedule targets a slot. PRE-2: For manual handling, Staff has identified an affected booking through calendar/list/issue information. PRE-3: Any target replacement slot is bookable.		
Postconditions:	POST-1: A conflicting new active booking is rejected, or an existing booking is rescheduled/cancelled through the corresponding use case. POST-2: Price/refund delta and notifications reflect the chosen action. POST-3: After moving/cancelling the affected booking, Staff may block the faulty slot.		
Normal Flow:	A. Handle booking conflict 1. Staff reviews the operation calendar or reported issue and identifies the affected booking/slot. 2. Staff chooses a valid available replacement and reschedules the booking. 3. The system releases the old slot, reprices the booking and notifies the Customer. 4. Staff blocks the old slot if it must remain unavailable. 5. Staff collects a balance or initiates a refund request if the price delta requires it.		
Alternative Flow:	A1: Cancel instead of reschedule 1. Staff cancels a pending/confirmed booking. 2. The system stores refundable amount and notifies the Customer. 3. Staff/Customer uses the separate refund workflow when value is refundable. A2: Conflict prevented at creation 1. The service detects pending/confirmed/checked-in use of the same slot and returns conflict. 2. PostgreSQL's partial unique active-slot index protects concurrent creation. A3: Service/field issue 1. Staff tracks the problem in the issue workflow and resolves it separately from booking status.		
Exceptions:	EXC-1: A blocked or already active-booked replacement slot is rejected. EXC-2: Staff cannot block an exact slot while an active booking remains on it, so reschedule/cancel must occur first. EXC-3: There is no atomic rollback across the separate reschedule/cancel, block, refund and notification actions. EXC-4: Inactive-field and same-day elapsed-time gaps described in UC-23/30 remain applicable.		

Priority:	<i>High</i>
Frequency of Use:	<i>Rare but operationally important</i>
Business Rules:	<i>BR-08, BR-09, BR-28</i>
Other Information:	<i>Implementation rule: Active slot reservation comprises pending, confirmed and checked_in. PostgreSQL enforces one active booking per slot. Manual conflict handling reuses UC-15, UC-21/22, UC-30, UC-32 and refund/payment workflows. There is no conflict entity, conflict status, dedicated controller endpoint or UI button named Handle conflict.</i>
Assumptions:	<i>Staff coordinates the multi-step resolution and verifies that each step succeeds before proceeding.</i>

Payment Gateway, Refund & Invoice

UC-37. View checkout summary

UC ID and Name:	<i>UC-37 View checkout summary</i>		
Created By:	<i>AnNP</i>	Date Created:	<i>30/05/2026</i>
Primary Actor:	<i>Guest, Customer, or Staff</i>	Secondary Actors:	<i>None</i>
Trigger:	<i>The actor selects a slot or changes services, promotion code, customer context, or booking source on the Booking page.</i>		
Description:	<i>Displays a server-calculated USD checkout breakdown before a booking or payment is committed.</i>		
Preconditions:	<i>PRE-1: A valid slot exists. PRE-2: Selected extra services, if any, exist and satisfy active-status, quantity, and overlapping-inventory rules. PRE-3: Customer login is not required for an anonymous preview; it is required to save a booking.</i>		
Postconditions:	<i>POST-1: No booking or payment record is created by preview. POST-2: The response contains field price, service total, promotion discount, membership discount, total, deposit, and remaining-after-deposit values.</i>		
Normal Flow:	<i>A. View checkout summary 1. Actor selects a date, field/slot, and optional services and promotion code. 2. Client posts the basket to /api/bookings/checkout-preview. 3. System resolves field pricing and service availability. 4. System validates the promotion and calculates the online-customer membership discount when applicable. 5. System applies the configured default deposit percentage and returns the breakdown. 6. Client renders the checkout summary and deposit/full choices.</i>		
Alternative Flow:	<i>A1: Alternative scenario 1. Guest previews without customer membership context, so membership discount is zero. A2: Alternative scenario 1. Staff previews a walk-in source; membership discount is zero by rule.</i>		

	A3: Alternative scenario 1. Actor changes an input and the debounced client recalculates the summary.
Exceptions:	EXC-1: Unknown slot, promotion, service, or customer data produces an API error. EXC-2: Inactive/expired/ineligible promotion or unavailable service inventory rejects the preview. EXC-3: A client-supplied customer ID is accepted by the public preview to calculate context, although customer profile data is not returned.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-06, BR-10, BR-21, BR-25
Other Information:	Implementation rule: USD only. Total = field price + services - promotion discount - online-customer membership discount. Deposit uses system_setting deposit.default_percent. Membership discount does not apply to guest preview or walk-in source. Evidence: BookingController.checkoutPreview, BookingWorkflowService.previewCheckout, DomainSupportService pricing methods, CheckoutSummary and BookingPage.
Assumptions:	The returned slot data is sufficient context for the UI; preview values are recalculated again during booking creation rather than trusted from the client.

UC-38. Choose payment option

UC ID and Name:	UC-38 Choose payment option		
Created By:	AnNP	Date Created:	30/05/2026
Primary Actor:	Customer or Staff	Secondary Actors:	None
Trigger:	The actor chooses Deposit or Full in the checkout payment selector.		
Description:	Lets a Customer choose deposit/full for an online PayPal booking and Staff choose deposit/full cash collection for a walk-in booking.		
Preconditions:	PRE-1: A checkout summary is available. PRE-2: Customer is signed in for online checkout, or Staff is signed in for walk-in checkout. PRE-3: The selected booking amount is greater than zero.		
Postconditions:	POST-1: The selected option is passed to booking/payment processing. POST-2: Deposit charges the configured deposit balance; full charges the remaining total.		
Normal Flow:	A. Choose payment option 1. System shows Deposit and Full with their payable amounts. 2. Actor selects one option. 3. Customer continues to PayPal, or Staff creates the walk-in and records matching cash. 4. Server recalculates and validates the exact payable amount.		
Alternative Flow:	A1: Alternative scenario 1. Customer pays only the deposit and leaves a venue balance.		

	A2: Alternative scenario 1. Staff records full cash immediately for a walk-in booking.
Exceptions:	EXC-1: A supplied amount that differs from the server-derived payable amount is rejected. EXC-2: Customer cannot create walk-in bookings; Staff/Admin cannot create online bookings. EXC-3: Admin has no checkout UI, although backend operator guards currently allow Admin walk-in/cash calls.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-21, BR-22
Other Information:	Implementation rule: Online PayPal checkout is Customer-only and online-booking-only. Staff UI records cash for walk-ins. Deposit means unpaid configured deposit; full means remaining booking balance. Evidence: PaymentOptionModal text area, BookingPage, BookingWorkflowService.requireBookingCreationRole, PaymentWorkflowService.payableAmount, PayPalCheckoutService.payableAmount.
Assumptions:	The intended product role is Staff for walk-in checkout and Admin for administration, despite broader backend operator authorization.

UC-39. Pay deposit/full amount via online payment sandbox

UC ID and Name:	UC-39 Pay deposit/full amount via online payment sandbox		
Created By:	AnNP	Date Created:	30/05/2026
Primary Actor:	Customer	Secondary Actors:	PayPal Sandbox
Trigger:	Customer clicks the PayPal button after preparing an online booking and choosing deposit or full payment.		
Description:	Creates and captures a PayPal Orders v2 sandbox transaction for a Customer-owned online booking.		
Preconditions:	PRE-1: Authenticated Customer owns an online booking. PRE-2: Booking is not cancelled, rejected, expired, or no-show and has a positive payable amount. PRE-3: PayPal credentials are configured and mock mode is off for real sandbox use. PRE-4: Customer ID in the request matches the signed-in Customer.		
Postconditions:	POST-1: A pending Payment stores provider order ID, amount, USD currency, payment option, and idempotency key. POST-2: Successful capture delegates to UC-40 state reconciliation. POST-3: Customer cancellation marks the Payment expired and may expire an unpaid pending Booking.		
Normal Flow:	A. Pay deposit/full amount via online payment sandbox 1. Client prepares the online booking. 2. Server derives the payable amount and creates a PayPal order with CAPTURE intent. 3. Customer approves in the PayPal Sandbox UI. 4. Client asks the server to capture the returned order ID.		

	5. Server validates provider status, currency, and amount before applying payment state.
Alternative Flow:	A1: Alternative scenario 1. Automated/local tests use explicit mock mode and synthetic provider IDs. A2: Alternative scenario 1. Customer cancels the PayPal popup; the cancel endpoint expires the pending order and unpaid hold.
Exceptions:	EXC-1: Missing credentials return service unavailable and the client reports online payment unavailable. EXC-2: Non-Customer, non-owner, actor mismatch, or walk-in booking is rejected. EXC-3: Provider/OAuth/order errors return bad gateway. EXC-4: Booking balance changed after order creation or provider amount/currency mismatch is rejected.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-22, BR-23
Other Information:	Implementation rule: Customer-only; online-booking-only; USD settlement; exact server-derived amount; PayPal Sandbox is classroom scope, not production/live-money certification. Evidence: PayPalCheckout React component, PaymentController PayPal endpoints, PayPalCheckoutService, PayPalCheckoutServiceTest and E2E authorization tests.
Assumptions:	A final external Sandbox buyer smoke test is required because automated tests use mock mode.

UC-40. Capture/confirm online payment

UC ID and Name:	UC-40 Capture/confirm online payment		
Created By:	AnNP	Date Created:	30/05/2026
Primary Actor:	Customer	Secondary Actors:	PayPal Sandbox
Trigger:	The PayPal SDK returns an approved order and the client calls the capture endpoint.		
Description:	Validates a PayPal capture and reconciles provider identifiers and payment totals into booking and invoice state.		
Preconditions:	PRE-1: A pending PayPal Payment exists for the Customer-owned online Booking and provider order ID. PRE-2: Current payable amount still equals the stored order amount. PRE-3: The order has not already been captured under another Payment.		
Postconditions:	POST-1: Payment becomes paid with capture ID, transaction code, provider status, gross amount, exact PayPal fee/provider net when returned, gateway message, and paid timestamp. POST-2: Booking paid and remaining amounts are updated. POST-3: A pending Booking becomes confirmed once paid amount reaches its deposit amount. POST-4: Invoice is upserted and a payment notification is created.		

Normal Flow:	A. Capture/confirm online payment 1. System locates the Payment by provider order ID and verifies Booking association. 2. System captures the remote order. 3. System requires provider status COMPLETED and validates USD amount. 4. System prevents duplicate capture IDs. 5. System persists Payment including seller_receivable_breakdown, Booking, invoice, and notification changes in one transaction. 6. System returns full booking billing detail.
Alternative Flow:	A1: Alternative scenario 1. If the Payment is already paid, system returns current booking detail idempotently. A2: Alternative scenario 1. The isolated automated-test profile replaces the provider call; the normal runtime does not expose this path.
Exceptions:	EXC-1: Order missing or linked to another Booking is rejected. EXC-2: Changed balance, duplicate capture ID, missing capture ID, or amount/currency mismatch is rejected. EXC-3: A non-COMPLETED provider capture throws an error while the local failed status/provider message is preserved for payment history.
Priority:	High
Frequency of Use:	High
Business Rules:	BR-12, BR-23
Other Information:	Implementation rule: Only verified provider COMPLETED capture is credited. Booking confirmation requires cumulative paid amount at least equal to deposit amount. Provider order ID, capture ID, status, currency, idempotency key, transaction code, gross amount, processor fee, provider net, and messages are persisted on Payment.
Assumptions:	No webhook reconciliation is implemented; browser/server capture is the authoritative path.

UC-41. Confirm remaining payment

UC ID and Name:	UC-41 Confirm remaining payment		
Created By:	AnNP	Date Created:	30/05/2026
Primary Actor:	Staff	Secondary Actors:	None
Trigger:	Staff selects Remaining payment for a booking with an outstanding balance.		
Description:	Records the exact outstanding booking balance as a venue cash transaction.		
Preconditions:	PRE-1: Staff is authenticated and the Booking has a positive remaining amount. PRE-2: Booking is not cancelled, rejected, expired, or no-show. PRE-3: Payment method is cash.		
Postconditions:	POST-1: A paid or failed cash Payment row is stored.		

	POST-2: On success, paid amount increases, remaining amount decreases, invoice is updated, and Customer is notified. POST-3: A pending Booking is confirmed if the cumulative paid amount reaches its deposit.
Normal Flow:	A. Confirm remaining payment 1. Staff opens the selected Booking in the Staff workspace. 2. Staff selects Remaining payment. 3. Server derives the current remaining amount and requires exact match. 4. Server stores a paid cash transaction with a generated cash transaction code. 5. System updates Booking and invoice and returns refreshed billing detail.
Alternative Flow:	A1: Alternative scenario 1. The same counter endpoint can record deposit or full cash payment for walk-in handling. A2: Alternative scenario 1. Admin is accepted by backend authorization although no Admin checkout action is exposed in the React UI. A3: Alternative scenario 1. A failed collection can be recorded with success=false.
Exceptions:	EXC-1: Customer call is forbidden. EXC-2: Non-cash method, zero payable amount, ineligible Booking state, or amount mismatch is rejected.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-23, BR-24
Other Information:	Implementation rule: Remaining payment is the current Booking.remainingAmount and venue counter payments must be cash. Evidence: StaffPage remaining-payment action, /api/payments/capture, PaymentWorkflowService.capturePayment.
Assumptions:	Physical cash handling and reconciliation occur outside the application.

UC-42. Handle failed/expired payment

UC ID and Name:	UC-42 Handle failed/expired payment		
Created By:	AnNP	Date Created:	30/05/2026
Primary Actor:	Customer or Time	Secondary Actors:	PayPal Sandbox
Trigger:	A payment attempt fails, Customer cancels PayPal checkout, or the pending unpaid hold exceeds the configured timeout.		
Description:	Keeps failed attempts visible where persisted and expires unpaid booking holds so their slots become available.		
Preconditions:	PRE-1: A pending payment/booking attempt exists, or a pending Booking has no paid amount. PRE-2: Scheduler is enabled for automatic timeout processing.		

Postconditions:	POST-1: Cancelled PayPal Payment becomes expired and an unpaid pending Booking becomes expired. POST-2: Scheduler marks aged unpaid pending Bookings expired and creates a Customer notification. POST-3: Expired Bookings no longer count as active slot conflicts.
Normal Flow:	A. Handle failed/expired payment 1. Scheduler reads payment.pending_timeout_minutes. 2. System finds pending Bookings older than the cutoff with zero paid amount. 3. System marks them expired, timestamps them, and notifies Customers. 4. Slot search can show the released slots as available.
Alternative Flow:	A1: Alternative scenario 1. Customer explicitly cancels PayPal checkout and the cancel endpoint expires the local Payment/Booking immediately. A2: Alternative scenario 1. A failed cash collection is persisted as Payment.status=failed and shown in billing history.
Exceptions:	EXC-1: Paid pending Bookings and newly created holds are skipped by the scheduler. EXC-2: Captured PayPal orders cannot be cancelled. EXC-3: Remote non-COMPLETED capture failure persistence is uncertain because an exception rolls back the transactional method.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-23, BR-28
Other Information:	Implementation rule: Default timeout is 15 minutes when no setting exists; configured value is clamped to at least one minute. Only zero-paid pending Bookings auto-expire. Evidence: BookingMaintenanceScheduler.expireUnpaidPendingBookings, PayPalCheckoutService.cancelOrder, payment status derivation, scheduler E2E test.
Assumptions:	No provider webhook or separate asynchronous reconciliation job exists.

UC-43. View payment history

UC ID and Name:	UC-43 View payment history		
Created By:	AnNP	Date Created:	30/05/2026
Primary Actor:	Customer, Staff, or Admin	Secondary Actors:	None
Trigger:	Actor opens payment history or booking billing detail.		
Description:	Returns persisted payment transactions, filtered to the signed-in Customer or expanded to all records for operators.		
Preconditions:	PRE-1: Actor is authenticated. PRE-2: Customer accesses only their own account/booking context.		
Postconditions:	POST-1: Payment records are displayed newest first with code,		

	<i>booking, option, method, amount, status, transaction code, gateway message, and paid time where available.</i>
Normal Flow:	<i>A. View payment history</i> <i>1. Client calls /api/payments.</i> <i>2. System identifies actor role.</i> <i>3. Customer receives payments for their own Bookings; Staff/Admin receives all payments.</i> <i>4. Client renders Account payment history or operator data.</i>
Alternative Flow:	<i>A1: Alternative scenario</i> <i>1. Booking detail returns only that Booking's transaction timeline alongside its invoice.</i>
Exceptions:	<i>EXC-1: Unauthenticated request is rejected.</i> <i>EXC-2: Customer cannot obtain another Customer's booking detail through the protected booking endpoint.</i>
Priority:	<i>Low</i>
Frequency of Use:	<i>High</i>
Business Rules:	<i>BR-02, BR-03, BR-23</i>
Other Information:	<i>Implementation rule: Role/ownership limits history: Customer sees own; Staff/Admin are operators and see all.</i> <i>Evidence: PaymentWorkflowService.payments, BookingWorkflowService.bookingDetail, AccountPage and BillingDetails.</i>
Assumptions:	<i>Payment history is an on-screen list; export and provider-led reconciliation are out of scope.</i>

UC-44. Generate booking invoice

UC ID and Name:	<i>UC-44 Generate booking invoice</i>		
Created By:	<i>AnNP</i>	Date Created:	<i>30/05/2026</i>
Primary Actor:	<i>Customer or Staff</i>	Secondary Actors:	<i>None</i>
Trigger:	<i>Payment capture, booking completion, reschedule, service amendment, or other invoice-upsert call completes.</i>		
Description:	<i>Creates or updates one invoice per Booking with current pricing, payment, balance, discount, and refund totals.</i>		
Preconditions:	<i>PRE-1: Booking exists and contains calculated financial totals.</i> <i>PRE-2: A workflow invokes invoice generation.</i>		
Postconditions:	<i>POST-1: Exactly one invoice is associated with the Booking.</i> <i>POST-2: Invoice contains field, service, combined discount, total, paid, remaining, refund, code, and issue time values.</i>		
Normal Flow:	<i>A. Generate booking invoice</i> <i>1. System looks up invoice by Booking ID.</i> <i>2. System creates an invoice/code if absent or reuses the existing invoice.</i> <i>3. System copies the latest Booking financial amounts.</i> <i>4. System timestamps and saves the invoice.</i>		
Alternative Flow:	<i>A1: Alternative scenario</i> <i>1. Completed refunds separately recalculate invoice.refundAmount from all completed refunds.</i>		

	A2: Alternative scenario 1. Failed counter payment still causes an invoice upsert with unchanged paid totals.
Exceptions:	EXC-1: Before the first invoice-generating event, booking detail may return invoice=null. EXC-2: Invoice does not contain separate promotion and membership discount columns; they are combined.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-24
Other Information:	Implementation rule: One-to-one Booking-to-Invoice. Invoice mirrors current Booking totals and completed refund aggregate. Evidence: Invoice entity unique booking join and DomainSupportService.generateInvoice/syncInvoiceRefundAmount.
Assumptions:	Invoice is an in-app billing record; PDF generation, tax numbering, and email delivery are not implemented.

UC-45. View invoice/payment status

UC ID and Name:	UC-45 View invoice/payment status		
Created By:	AnNP	Date Created:	30/05/2026
Primary Actor:	Customer, Staff, or Admin	Secondary Actors:	None
Trigger:	Actor selects a Booking in Account, Staff, or Admin-loaded booking data.		
Description:	Shows invoice amounts, derived payment status, transaction history, and refund information for an authorized Booking.		
Preconditions:	PRE-1: Actor is authenticated and either owns the Booking or is Staff/Admin. PRE-2: Booking exists.		
Postconditions:	POST-1: The actor sees current billing detail without changing financial state.		
Normal Flow:	A. View invoice/payment status 1. Client calls /api/bookings/{bookingId}. 2. System checks ownership/operator visibility. 3. System loads services, payments, promotions, invoice, refunds, and derives paymentStatus. 4. BillingDetails displays invoice totals and transaction timeline.		
Alternative Flow:	A1: Alternative scenario 1. If no invoice exists yet, UI shows 'Invoice not issued' and falls back to Booking amounts. A2: Alternative scenario 1. Completed refunds produce partially_refunded or refunded status.		
Exceptions:	EXC-1: Unauthorized Customer access to another Booking is forbidden. EXC-2: Missing Booking returns not found.		
Priority:	Low		
Frequency of	High		

Use:	
Business Rules:	BR-02, BR-03, BR-24
Other Information:	Implementation rule: Derived statuses are unpaid, failed, expired, partially_paid, paid, partially_refunded, or refunded based on paid amount, remaining amount, Booking expiry, failed Payment presence, and completed Refund totals. Evidence: bookingDetail, paymentStatus, BillingDetails, AccountPage and StaffPage.
Assumptions:	Admin can load billing data, although the main dedicated billing interaction is presented in Customer/Staff workspaces.

UC-46. Process online refund

UC ID and Name:	UC-46 Process online refund		
Created By:	AnNP	Date Created:	30/05/2026
Primary Actor:	Staff	Secondary Actors:	PayPal Sandbox
Trigger:	Staff approves an eligible refund tied to a captured PayPal payment and selects Send/Retry to PayPal.		
Description:	Submits or reconciles an approved refund against a PayPal capture and records the provider's actual state.		
Preconditions:	PRE-1: Refund is approved, processing, or failed for retry. PRE-2: Selected Payment is paid, uses paypal_sandbox, has a provider capture ID, and has enough uncommitted value. PRE-3: Refund amount is positive and does not exceed Booking and Payment refund capacity. PRE-4: Staff/Admin is authenticated.		
Postconditions:	POST-1: Refund stores provider status, gateway message, provider refund ID when returned, and processed time for terminal states. POST-2: Only PayPal COMPLETED produces local completed state and updates invoice refunded amount. POST-3: Customer receives a refund-status notification.		
Normal Flow:	A. Process online refund 1. Staff approves the requested Refund. 2. Staff requests processing. 3. System calls PayPal Payments v2 captures/{captureId}/refund using the stored idempotency key. 4. System maps COMPLETED to completed, PENDING to processing, and other provider status to failed. 5. System stores provider evidence, updates invoice when completed, and notifies Customer.		
Alternative Flow:	A1: Alternative scenario 1. If a provider refund ID already exists, retry performs GET /v2/payments/refunds/{id} to reconcile state. A2: Alternative scenario 1. On network uncertainty without a returned ID, system leaves PENDING/processing and retries with the same PayPal-Request-Id. A3: Alternative scenario 1. Automated provider-adapter tests use an isolated test profile; the normal runtime always requires real Sandbox credentials and		

	<i>provider state.</i>
Exceptions:	<i>EXC-1: A PayPal Refund cannot be directly changed from approved to completed; this transition is rejected.</i> <i>EXC-2: Missing capture, wrong method/status, insufficient per-payment capacity, or absent credentials results in failed/rejected processing.</i> <i>EXC-3: Provider sandbox confirmation remains an external smoke test.</i>
Priority:	<i>High</i>
Frequency of Use:	<i>Medium</i>
Business Rules:	<i>BR-18, BR-23, BR-28</i>
Other Information:	<i>Implementation rule: Provider state is authoritative. Stable idempotency prevents duplicate refunds. Each Refund is tied to a single paid Payment with sufficient remaining capacity. Cash refund completion is not this online flow.</i> <i>Evidence:</i> <i>PaymentWorkflowService.processProviderRefund/resolveRefundPayment, PayPalCheckoutService.refundCapture, V9 provider-refund migration, StaffPage controls, E2E transition test.</i>
Assumptions:	<i>PayPal merchant credentials and a genuinely captured sandbox transaction are required for final external verification.</i>

UC-47. Manage refund requests

UC ID and Name:	UC-47 Manage refund requests		
Created By:	AnNP	Date Created:	30/05/2026
Primary Actor:	Staff	Secondary Actors:	Customer; PayPal Sandbox
Trigger:	The booking-owning Customer creates a refund request, or Staff opens the refund queue.		
Description:	Lets the booking owner create an eligible refund request and lets Staff review, approve/reject, submit/retry PayPal refunds, or record cash completion.		
Preconditions:	<i>PRE-1: Booking has a positive refundable amount.</i> <i>PRE-2: The authenticated Customer owns the Booking.</i> <i>PRE-3: A single paid Payment has sufficient remaining refundable capacity.</i>		
Postconditions:	<i>POST-1: Refund request is persisted with code, requester, amount, selected Payment, reason, idempotency key, and requested time.</i> <i>POST-2: Approved/rejected/processing/completed/failed transitions are recorded with operator/provider evidence.</i> <i>POST-3: Customer is notified on request and each handled status.</i>		
Normal Flow:	<i>A. Manage refund requests</i> <i>1. The booking-owning Customer submits refund amount/reason for the Booking.</i> <i>2. System subtracts non-rejected/non-failed committed refunds from available refundable amount.</i> <i>3. System selects a qualifying paid Payment, preferring PayPal when</i>		

	no payment ID is supplied. 4. Staff reviews the refund queue. 5. Staff approves or rejects. 6. For approved PayPal, Staff submits/retries provider processing; for approved cash, Staff records manual completion.
Alternative Flow:	A1: Alternative scenario 1. Staff/Admin can view all requests while a Customer sees only requests for their own bookings. A2: Alternative scenario 1. Rejected or failed requests no longer consume refund capacity. A3: Alternative scenario 1. Cash completion generates a manual cash-refund transaction code and MANUAL_CASH_REFUND provider marker.
Exceptions:	EXC-1: Unauthorized ownership, no refundable amount, invalid amount, wrong Payment, or insufficient single-payment capacity is rejected. EXC-2: Only requested may transition to approved/rejected; approved PayPal must go through processing; approved cash alone may transition directly to completed. EXC-3: Staff/Admin cannot submit a Customer request, and Customers cannot approve or process a request.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-18, BR-23
Other Information:	Implementation rule: Committed capacity counts requested, approved, processing, and completed refunds; rejected/failed are excluded. PayPal completion is provider-only; cash completion is manual venue action. The Customer account page owns the request action. The Staff workspace owns handling actions; Admin receives read/audit access and the same operator authorization at API level.
Assumptions:	Refund eligibility amount was previously established by cancellation policy or a lower-price reschedule/service amendment.

UC-48. Configure deposit rules

UC ID and Name:	UC-48 Configure deposit rules		
Created By:	AnNP	Date Created:	30/05/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	Admin edits deposit.default_percent in Policies or chooses the 30%/50% shortcut.		
Description:	Maintains the single default percentage used to calculate every booking deposit.		
Preconditions:	PRE-1: Admin is authenticated. PRE-2: deposit.default_percent setting exists.		
Postconditions:	POST-1: Setting value is saved and used by subsequent preview, booking, reschedule, and service-amendment deposit calculations.		
Normal Flow:	A. Configure deposit rules		

	1. Admin opens Policies. 2. System lists settings. 3. Admin enters a percentage or selects 30%/50%. 4. Server validates 0 through 100 and saves the setting.
Alternative Flow:	A1: Alternative scenario 1. If no setting exists at calculation time, system falls back to 30%.
Exceptions:	EXC-1: Non-Admin update is forbidden. EXC-2: Blank, nonnumeric, below-zero, or above-100 value is rejected. EXC-3: Unknown setting key returns not found.
Priority:	Medium
Frequency of Use:	Low
Business Rules:	BR-22
Other Information:	Implementation rule: Deposit = booking total multiplied by one global default percentage and rounded to two decimals. Historical examples such as higher deposits for locked customers or forced full payment are not implemented.
Assumptions:	Existing Bookings retain stored deposit amounts until another repricing workflow recalculates them.

UC-49. Configure cancellation/refund policy

UC ID and Name:	UC-49 Configure cancellation/refund policy		
Created By:	AnNP	Date Created:	30/05/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	Admin edits refund percentage settings in Policies.		
Description:	Maintains percentage values used by the two implemented pre-start cancellation windows.		
Preconditions:	PRE-1: Admin is authenticated. PRE-2: refund.before_24h_percent and refund.same_day_percent settings exist.		
Postconditions:	POST-1: Updated percentage is used in subsequent cancellation previews and cancellations.		
Normal Flow:	A. Configure cancellation/refund policy 1. Admin opens Policies and reviews settings. 2. Admin edits a refund percentage. 3. Server validates 0 through 100 and saves the setting. 4. Later cancellation preview selects the applicable window and calculates refund/fee from paid amount.		
Alternative Flow:	A1: Alternative scenario 1. At least 24 hours before start uses before_24h value, default 100%. A2: Alternative scenario 1. From under 24 hours until start uses same_day value, default 80%. A3: Alternative scenario 1. After start always uses 0%.		
Exceptions:	EXC-1: Non-Admin update is forbidden. EXC-2: Invalid numeric percentage is rejected.		

	<i>EXC-3: Only pending or confirmed Bookings can be previewed/cancelled.</i>
Priority:	<i>Medium</i>
Frequency of Use:	<i>Low</i>
Business Rules:	<i>BR-16, BR-17, BR-18</i>
Other Information:	<i>Implementation rule: Refund and cancellation fee are percentages of paid amount, not booking total. No-show and post-start refunds are not configurable in this implementation. This is a compact settings editor, not a general multi-window rule engine.</i>
Assumptions:	<i>Policy changes affect future preview calculations immediately and are not versioned per Booking.</i>

Promotion, Membership, Notification & Report

UC-50. View active promotions

UC ID and Name:	<i>UC-50 View active promotions</i>		
Created By:	<i>AnPTT</i>	Date Created:	<i>30/05/2026</i>
Primary Actor:	<i>Guest or Customer</i>	Secondary Actors:	<i>None</i>
Trigger:	<i>Visitor opens /promotions or the application loads public catalogue data.</i>		
Description:	<i>Displays promotion campaign cards and banners from the public promotion endpoint.</i>		
Preconditions:	<i>PRE-1: Promotion records exist. PRE-2: No authentication is required for the public list.</i>		
Postconditions:	<i>POST-1: Visitor sees promotion code/name, banner, description, value, period, usage, status, and eligibility metadata returned by the API.</i>		
Normal Flow:	<i>A. View active promotions 1. Client calls GET /api/promotions without includeInactive. 2. Backend returns promotions whose status is active. 3. Non-Admin client filters status active and paginates cards. 4. Visitor reviews available offers.</i>		
Alternative Flow:	<i>A1: Alternative scenario 1. Admin UI requests includeInactive=true and sees the full campaign catalogue.</i>		
Exceptions:	<i>EXC-1: Backend list does not filter by current date; an active-status campaign outside its date window can still appear. EXC-2: Because includeInactive is a public query flag, any caller can technically retrieve inactive campaigns.</i>		
Priority:	<i>Low</i>		
Frequency of Use:	<i>Medium</i>		
Business Rules:	<i>BR-25</i>		
Other Information:	<i>Implementation rule: Displayed public catalogue is status-filtered; actual usability is revalidated against status, dates, usage, and other conditions during application.</i>		

	<i>Evidence: SecurityConfig public GET /api/promotions, PromotionReportService.promotions, PromotionsPage.</i>
Assumptions:	<i>Campaign visibility and campaign eligibility are separate concepts in the current code.</i>

UC-51. Manage promotion campaigns

UC ID and Name:	UC-51 Manage promotion campaigns		
Created By:	AnPTT	Date Created:	30/05/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	Admin adds a promotion or edits an existing promotion in the Promotions page.		
Description:	Creates and updates campaign identity, banner, discount, limits, dates, status, and eligibility conditions.		
Preconditions:	PRE-1: Admin is authenticated. PRE-2: Referenced field type, service, or membership level exists when supplied.		
Postconditions:	POST-1: Promotion is created or updated and returned to the Admin catalogue. POST-2: New campaigns start with usedCount zero.		
Normal Flow:	A. Manage promotion campaigns 1. Admin opens promotion editor. 2. Admin enters unique code/name, discount, date range, status, usage/min/max values, banner, and optional conditions. 3. Server validates and normalizes the fields. 4. System saves and refreshes campaign list.		
Alternative Flow:	A1: Alternative scenario 1. Admin deactivates a campaign by setting status inactive. A2: Alternative scenario 1. Discount may be percent or fixed_amount; percent cannot exceed 100. A3: Alternative scenario 1. Eligibility may be global or limited by field type, service, membership, weekday/weekend, and time range.		
Exceptions:	EXC-1: Non-Admin create/update is forbidden. EXC-2: Duplicate/blank code, blank name, nonpositive discount, invalid date/time range, percent above 100, or invalid referenced IDs is rejected. EXC-3: Usage limit/min/max non-negativity is mostly dependent on UI constraints; backend validation is incomplete for some numeric fields.		
Priority:	High		
Frequency of Use:	Low		
Business Rules:	BR-02, BR-03, BR-25, BR-30		
Other Information:	Implementation rule: Promotion code is unique and stored uppercase. Only one code can enter the booking flow even though a stackable flag is stored. Banner uses the shared image upload field. There is no delete endpoint; activation status is the lifecycle control.		

Assumptions:	<i>Admin follows UI numeric constraints where backend validation is not exhaustive.</i>
---------------------	---

UC-52. Apply promotion to booking

UC ID and Name:	<i>UC-52 Apply promotion to booking</i>		
Created By:	<i>AnPTT</i>	Date Created:	<i>30/05/2026</i>
Primary Actor:	<i>Customer or Staff</i>	Secondary Actors:	<i>None</i>
Trigger:	<i>Actor enters a promotion code during checkout.</i>		
Description:	<i>Validates one code against campaign and booking context, calculates discount, and snapshots it when the Booking is created.</i>		
Preconditions:	<i>PRE-1: Promotion code exists. PRE-2: Slot, selected services, base amount, booking source, and optional Customer membership context are available.</i>		
Postconditions:	<i>POST-1: Valid preview includes promotion discount. POST-2: Booking creation stores BookingPromotion code snapshot and amount and increments usedCount. POST-3: Invoice total includes the discount.</i>		
Normal Flow:	<i>A. Apply promotion to booking 1. Actor enters one code. 2. System validates active status, current date, usage limit, minimum amount, field type, required service, membership level, day type, and time range. 3. System calculates percent or fixed discount and applies maximum cap. 4. System returns revised checkout total. 5. On booking creation, system snapshots the application and increments usage.</i>		
Alternative Flow:	<i>A1: Alternative scenario 1. No code yields zero promotion discount. A2: Alternative scenario 1. On reschedule/service edit, the stored promotion is recalculated for the new context without consuming usage again; an ineligible stored promotion becomes zero.</i>		
Exceptions:	<i>EXC-1: Unknown, inactive, out-of-date, exhausted, below-minimum, or context-ineligible promotion is rejected during initial preview/creation. EXC-2: Only one code is accepted; multiple-code stacking is not available regardless of stackable flag.</i>		
Priority:	<i>High</i>		
Frequency of Use:	<i>Medium</i>		
Business Rules:	<i>BR-25, BR-26</i>		
Other Information:	<i>Implementation rule: Discount cannot exceed base amount; percent rounds to two decimals; optional max cap applies; one promotion code per Booking. Membership-level condition requires an actual Customer membership; anonymous preview cannot qualify for such a code.</i>		
Assumptions:	<i>Concurrent usage-limit enforcement relies on normal transactional</i>		

	use; no explicit database lock is visible in calculation.
--	---

UC-53. View membership progress

UC ID and Name:	UC-53 View membership progress		
Created By:	AnPTT	Date Created:	30/05/2026
Primary Actor:	Customer	Secondary Actors:	None
Trigger:	Customer opens Account or Membership Benefits, or completion updates membership.		
Description:	Shows the current tier, lifetime completed count, configured period-aware progress, discount, next tier, and remaining target.		
Preconditions:	PRE-1: Customer is authenticated and requests their own progress; Admin may request any Customer. PRE-2: CustomerMembership exists.		
Postconditions:	POST-1: Progress data is displayed without mutation. POST-2: On Booking completion, membership count/tier is recalculated from completed history.		
Normal Flow:	A. View membership progress 1. Client calls /api/membership/{customerId}/progress. 2. System enforces self-or-Admin access. 3. System loads active ordered levels and resolves the highest level whose lifetime, monthly, or weekly rule is met. 4. System calculates next-level progress using completed bookings in the configured qualification period. 5. Client renders the current tier and next-level target.		
Alternative Flow:	A1: Alternative scenario 1. At the highest tier, nextLevel is null and target values are zero. A2: Alternative scenario 1. Weekly progress is the current consecutive qualifying-week streak; monthly progress is the current month's completed bookings. A3: Alternative scenario 1. Completion creates default membership if absent before recalculation.		
Exceptions:	EXC-1: Another Customer or Staff requesting this Customer's progress is forbidden. EXC-2: Missing membership returns not found on the read endpoint.		
Priority:	Medium		
Frequency of Use:	Medium		
Business Rules:	BR-02, BR-20		
Other Information:	Implementation rule: Only completed bookings count. A level may qualify by lifetime total, current-month total, or a consecutive-week streak where each week meets requiredCompletedBookings. CustomerMembership.completedBookingCount remains the lifetime audit count; nextLevelProgress/Target carry the configured period-aware progress.		
Assumptions:	Display order defines progression order; active levels are evaluated against LocalDate.now().		

UC-54. Manage membership rules

UC ID and Name:	UC-54 Manage membership rules		
Created By:	AnPTT	Date Created:	30/05/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	Admin adds or edits a membership tier in /membership/rules.		
Description:	Creates and updates membership thresholds, discounts, benefits, order, active status, and lifetime/monthly/weekly qualification rules.		
Preconditions:	PRE-1: Admin is authenticated. PRE-2: Tier name is unique.		
Postconditions:	POST-1: MembershipLevel is created or updated. POST-2: Subsequent membership recalculation and online checkout use the configured threshold/discount.		
Normal Flow:	A. Manage membership rules 1. Admin opens Tier rules. 2. Admin enters level identity, booking threshold, discount, benefit, display order, status, qualification period, and any consecutive-week requirement. 3. Server validates unique name, allowed period, and consecutive count, then saves the level. 4. Client refreshes the complete admin catalogue including inactive levels.		
Alternative Flow:	A1: Alternative scenario 1. Admin marks a tier inactive instead of deleting it. A2: Alternative scenario 1. Admin edits an existing tier without changing its name.		
Exceptions:	EXC-1: Non-Admin create/update or includeInactive read is forbidden. EXC-2: Blank or duplicate name is rejected. EXC-3: Qualification period outside lifetime/monthly/weekly or consecutive periods below one is rejected. EXC-4: The backend does not yet enforce nonnegative thresholds, 0-100 discount bounds, or unique display order.		
Priority:	Medium		
Frequency of Use:	Low		
Business Rules:	BR-02, BR-03, BR-27, BR-30		
Other Information:	Implementation rule: Qualification period is lifetime, monthly, or weekly. Weekly qualification requires requiredCompletedBookings in each of requiredConsecutivePeriods consecutive weeks. Public catalogue filters inactive levels; Admin requests includeInactive=true and receives status/displayOrder/period fields.		
Assumptions:	Admin maintains ascending, non-overlapping thresholds and sensible discount values.		

UC-55. View membership benefits

UC ID and Name:	UC-55 View membership benefits		
Created By:	AnPTT	Date Created:	30/05/2026

Primary Actor:	Guest or Customer	Secondary Actors:	None
Trigger:	Visitor opens /membership/benefits or the app loads public membership levels.		
Description:	Shows active tier names, qualification rules, discount percentages, benefits, and signed-in Customer progress.		
Preconditions:	PRE-1: Membership levels exist. PRE-2: No authentication is required for the tier catalogue.		
Postconditions:	POST-1: Visitor sees benefit descriptions and thresholds. POST-2: Signed-in Customer sees current tier and progress to the next tier.		
Normal Flow:	A. View membership benefits 1. Client calls public GET /api/membership/levels. 2. System returns active levels ordered by displayOrder with qualification fields. 3. Page renders lifetime, monthly, or consecutive-week rule text. 4. If Customer membership is loaded, page marks the current tier and progress.		
Alternative Flow:	A1: Alternative scenario 1. Guest sees tier benefits without personal progress. A2: Alternative scenario 1. Customer without loaded membership is shown as not enrolled.		
Exceptions:	EXC-1: No configured active levels produces an empty catalogue. EXC-2: Admin-only includeInactive=true is rejected for non-Admin callers.		
Priority:	Low		
Frequency of Use:	Medium		
Business Rules:	BR-27		
Other Information:	Implementation rule: Only active levels are public. Each summary includes display order, status, qualification period, and consecutive-period requirement. Admin management uses includeInactive=true; public benefits use the default false value.		
Assumptions:	Public ordering follows persisted displayOrder.		

UC-56. Send booking confirmation notification

UC ID and Name:	UC-56 Send booking confirmation notification		
Created By:	AnPTT	Date Created:	30/05/2026
Primary Actor:	Customer or Staff	Secondary Actors:	None
Trigger:	Booking is created or later transitions to confirmed; payment/reschedule/service changes also generate related in-app messages.		
Description:	Creates persistent in-app booking confirmation/category notifications for the Customer.		
Preconditions:	PRE-1: Booking and Customer exist. PRE-2: A notification-producing booking workflow completes.		
Postconditions:	POST-1: Notification is stored with Customer, Booking, type, title, message, sent time, and unread state.		

	POST-2: Customer can see it through the notification bell/account feed.
Normal Flow:	A. Send booking confirmation notification 1. Booking workflow completes creation or confirmation. 2. System creates an unread in-app notification. 3. Client loads/polls the Customer notification endpoint. 4. Customer opens the bell and may toggle read state or mark all read.
Alternative Flow:	A1: Alternative scenario 1. Booking creation itself generates 'Booking created' even before an online Booking is confirmed. A2: Alternative scenario 1. Staff rejection, reschedule, and service amendment use the same booking_confirmation type with different titles.
Exceptions:	EXC-1: Notifications are in-app only; external email is not part of this UC. EXC-2: Only the owner or Admin can list/toggle the Customer's notifications; Staff may access only their own notification list.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-12, BR-28
Other Information:	Implementation rule: Notification read/unread changes do not alter Booking state. Notification access is self-or-Admin. The UI polls every 15 seconds and supports individual toggle and mark-all-read.
Assumptions:	A 15-second polling delay is acceptable for the classroom deployment; real-time push is out of scope.

UC-57. Send booking reminder notification

UC ID and Name:	UC-57 Send booking reminder notification		
Created By:	AnPTT	Date Created:	30/05/2026
Primary Actor:	Time	Secondary Actors:	Customer
Trigger:	Scheduled maintenance runs and finds an eligible upcoming pending/confirmed Booking within the configured reminder window.		
Description:	Creates exactly one in-app reminder before the Booking start time.		
Preconditions:	PRE-1: Scheduler is enabled and runs. PRE-2: Booking is pending or confirmed and its start is between now and the configured reminder window. PRE-3: No booking_reminder notification already exists for the Booking.		
Postconditions:	POST-1: One unread reminder notification is stored for the Customer. POST-2: Repeated scheduler runs do not duplicate the reminder.		
Normal Flow:	A. Send booking reminder notification 1. Scheduler reads notification.booking_reminder_hours, default 24. 2. System scans pending and confirmed Bookings. 3. System calculates minutes until slot start. 4. For each eligible Booking without a prior reminder, system creates		

	<i>the notification.</i>
Alternative Flow:	A1: Alternative scenario 1. Admin can change the reminder-hours setting through the generic Policies settings editor.
Exceptions:	EXC-1: Past, outside-window, terminal-status, or already-reminded Bookings are skipped. EXC-2: Configured hours below one are rejected and runtime calculation clamps to at least one hour.
Priority:	Medium
Frequency of Use:	High
Business Rules:	BR-28
Other Information:	Implementation rule: At most one booking_reminder per Booking, enforced by repository existence check. Scheduler default delay is 60 seconds. E2E scheduler test invokes maintenance and confirms reminder type and nonduplication.
Assumptions:	Application process and database are available during the reminder window; no external queue/email fallback exists.

UC-58. Send cancellation/refund notification

UC ID and Name:	UC-58 Send cancellation/refund notification		
Created By:	AnPTT	Date Created:	30/05/2026
Primary Actor:	Customer or Staff	Secondary Actors:	None
Trigger:	Booking is cancelled, refund is requested, or Staff changes refund status.		
Description:	Creates persistent in-app cancellation and refund lifecycle notifications for the Customer.		
Preconditions:	PRE-1: Customer and relevant Booking exist. PRE-2: Cancellation/refund transaction reaches the notification call.		
Postconditions:	POST-1: Customer receives an unread cancellation/refund notification with current outcome. POST-2: Notification is available in the bell/account feed.		
Normal Flow:	A. Send cancellation/refund notification 1. Customer or operator cancels an eligible Booking, or submits/updates a Refund. 2. System persists the workflow state. 3. System creates the matching in-app notification. 4. Customer retrieves and reads it.		
Alternative Flow:	A1: Alternative scenario 1. Refund titles cover requested, approved, rejected, processing, completed, and failed. A2: Alternative scenario 1. Cancellation uses the cancellation type and Refund uses refund type.		
Exceptions:	EXC-1: A failed/rolled-back transaction does not leave a committed notification. EXC-2: No external email/SMS notification is implemented for these		

	events.
Priority:	Medium
Frequency of Use:	Medium
Business Rules:	BR-18, BR-28
Other Information:	Implementation rule: Notification reflects local persisted workflow status; PayPal completed wording is used only after provider COMPLETED is mapped locally. A Refund request creates one notification and each later status update creates another.
Assumptions:	Multiple lifecycle notifications for the same Refund are intentional audit communication.

UC-59. View revenue report

UC ID and Name:	UC-59 View revenue report		
Created By:	AnPTT	Date Created:	30/05/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	Admin opens the Admin Overview and the client loads /api/reports.		
Description:	Displays current gross collection, tracked PayPal processor fee, refund, net revenue, booked-value, discount, and Booking totals in USD.		
Preconditions:	PRE-1: Admin is authenticated. PRE-2: Report data can be read from Booking, Payment, and Refund repositories.		
Postconditions:	POST-1: Admin sees net/gross revenue, tracked processor fee, untracked-fee warning count, completed refund total, field/service booked value, promotion/membership discounts, and Booking counts.		
Normal Flow:	A. View revenue report 1. Client requests /api/reports. 2. System sums paid, partially refunded, and refunded Payment amounts as gross collected. 3. System sums completed Refund amounts and exact provider_fee_amount values returned by PayPal. 4. System calculates net revenue as gross collected minus completed refunds minus tracked provider fees. 5. System aggregates Booking financial components. 6. Admin page renders metrics and field revenue data.		
Alternative Flow:	A1: Alternative scenario 1. With no data, aggregates return zero/empty collections.		
Exceptions:	EXC-1: Non-Admin access is forbidden. EXC-2: No date range, export, currency conversion, or accounting-period filter exists. EXC-3: Booked field/service/discount values exclude rejected/expired only and may include cancelled/no-show bookings; they are portfolio values, not strictly realized revenue.		
Priority:	Medium		
Frequency of Use:	Medium		

Business Rules:	BR-31, BR-32
Other Information:	Implementation rule: Gross collected retains original captured/cash amounts. Net revenue = gross collected - completed Refunds - exact tracked provider fees. Unknown legacy fees are flagged, never estimated. All monetary display uses USD. bookedRevenueByField is also returned. PayPal fee/net values come from seller_receivable_breakdown and are persisted by Flyway V19 columns.
Assumptions:	A whole-life aggregate without temporal filters is acceptable for the classroom scope; external PayPal settlement statements remain the final accounting authority.

UC-60. View booking report

UC ID and Name:	UC-60 View booking report		
Created By:	AnPTT	Date Created:	30/05/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	Admin opens the Admin Overview.		
Description:	Displays Booking counts by lifecycle status, counts by field, peak start times, and headline completion/cancellation/no-show counts.		
Preconditions:	PRE-1: Admin is authenticated. PRE-2: Booking records exist or empty results are acceptable.		
Postconditions:	POST-1: Admin sees current all-time Booking operational statistics.		
Normal Flow:	A. View booking report 1. System loads all Bookings. 2. System groups them by field name, start time, and status. 3. System counts completed, cancelled, and no-show Bookings. 4. Admin page renders field utilization and booking trends lists.		
Alternative Flow:	A1: Alternative scenario 1. Empty repository returns zero counts and empty maps.		
Exceptions:	EXC-1: Non-Admin access is forbidden. EXC-2: Field utilization is raw Booking count, not occupancy percentage or time-normalized utilization. EXC-3: Peak slots are grouped by start-time string only, with no date/day segmentation.		
Priority:	Medium		
Frequency of Use:	Medium		
Business Rules:	BR-31, BR-32		
Other Information:	Implementation rule: Report includes all Booking statuses in grouping/count totals; no temporal filter is applied. Evidence: PromotionReportService.reports and AdminPage Booking trends/Field utilization panels.		
Assumptions:	All-time raw counts satisfy the assessed classroom reporting scope.		

UC-61. View customer activity report

UC ID and Name:	UC-61 View customer activity report
------------------------	-------------------------------------

Created By:	AnPTT	Date Created:	30/05/2026
Primary Actor:	Admin	Secondary Actors:	None
Trigger:	Admin opens the Customer activity section of the Admin Overview.		
Description:	Displays tracked Customers, returning Customer count, top Customers by Booking count, and membership distribution.		
Preconditions:	PRE-1: Admin is authenticated. PRE-2: Customer and membership data can be read.		
Postconditions:	POST-1: Admin sees current all-time Customer activity aggregates.		
Normal Flow:	A. View customer activity report 1. System loads all Customer users. 2. System counts each Customer's Bookings and sorts descending. 3. System counts Customers with more than one Booking as returning. 4. System groups CustomerMembership records by level name. 5. Admin page renders returning/tracked metrics, top five, and membership distribution.		
Alternative Flow:	A1: Alternative scenario 1. Customers with zero Bookings remain in topCustomers/tracked-customer list after sorting. A2: Alternative scenario 1. Empty memberships produce an empty distribution.		
Exceptions:	EXC-1: Non-Admin access is forbidden. EXC-2: No date range, cohort, cancellation/no-show rate, spend ranking, or export exists.		
Priority:	Medium		
Frequency of Use:	Medium		
Business Rules:	BR-31, BR-32		
Other Information:	Implementation rule: Returning means more than one Booking of any status. Top Customer ranking uses raw Booking count, not completed count or revenue. Admin also has a per-Customer activity modal elsewhere, but UC-61 is the aggregate report.		
Assumptions:	Raw all-status Booking count is the agreed customer-activity measure.		

UC-62. Ask availability assistant

UC ID and Name:	UC-62 Ask availability assistant		
Created By:	AnPTT	Date Created:	30/05/2026
Primary Actor:	Customer	Secondary Actors:	Gemini API
Trigger:	An authenticated Customer opens /assistant and submits a natural-language availability request.		
Description:	Gemini extracts date, preferred time, field type, and budget from an English question, while GoalZone's deterministic slot service remains the source of truth for availability, prices, and eligible promotions.		
Preconditions:	PRE-1: The Customer is authenticated. PRE-2: The Gemini API key and model are configured. PRE-3: Active field, pricing, and slot-generation data exist.		

Postconditions:	POST-1: A grounded answer, interpreted criteria, and up to eight live ranked slots are returned. POST-2: No Booking or Payment is created by the assistant request.
Normal Flow:	A. Ask the Gemini availability assistant 1. The Customer asks for a field in plain English. 2. The client posts the question to /api/assistant/availability. 3. The server verifies the authenticated Customer and asks Gemini to extract constrained search criteria. 4. The server validates the date, time, field type, and budget. 5. FieldOperationService materializes and ranks only live available slots. 6. Gemini phrases a short answer using only the verified criteria and slots. 7. The client displays the grounded answer and booking-ready recommendations.
Alternative Flow:	A1: The visitor uses the deterministic filters 1. Guest or Customer selects date, time, field type, and budget manually. 2. The client calls GET /api/slots/suggestions without requiring Gemini. A2: No matching slot exists 1. The assistant reports no result and suggests broadening one criterion. A3: Optional criteria are omitted 1. The date defaults to tomorrow and omitted filters remain unrestricted.
Exceptions:	EXC-01: Missing Gemini configuration returns service unavailable. EXC-02: Gemini rejection, rate limit, timeout, or invalid output returns a gateway error without fabricated slots. EXC-03: A past date or invalid request returns bad request.
Priority:	High
Frequency of Use:	Medium
Business Rules:	BR-05, BR-06, BR-07, BR-08, BR-09
Other Information:	Gemini interprets and phrases the request only. GoalZone data remains authoritative for every field, slot, price, and promotion shown. UC-63 handles selection and booking handoff.
Assumptions:	The Customer asks in English using at most 500 characters, and the configured Gemini service is reachable.

UC-63. Get suggested available slots

UC ID and Name:	UC-63 Get suggested available slots		
Created By:	AnPTT	Date Created:	30/05/2026
Primary Actor:	Guest or Customer	Secondary Actors:	None
Trigger:	A Guest or Customer selects Book this slot on a ranked recommendation.		
Description:	Transfers the suggested slot id/date into the real booking journey and revalidates live availability before selection.		

Preconditions:	<i>PRE-1: UC-62 or the deterministic filter returned at least one live available suggestion. PRE-2: The visitor selects a result.</i>
Postconditions:	<i>POST-1: Booking page is opened for the suggested date. POST-2: The suggested slot is selected only if it remains available.</i>
Normal Flow:	<i>A. Continue a suggested slot into booking 1. The client stores the suggested slot id and date in sessionStorage. 2. The client navigates to /booking. 3. The Booking page reloads live slots for the suggested date. 4. If still available, the app selects the exact suggested slot and clears the handoff value. 5. A Guest signs in or registers when required; the Customer then continues the normal checkout flow.</i>
Alternative Flow:	<i>A1: Alternative scenario 1. If the slot became unavailable, the app shows current availability and selects another available slot when present.</i>
Exceptions:	<i>EXC-1: Invalid handoff data is discarded. EXC-2: Normal booking validation prevents stale or reserved slot creation.</i>
Priority:	<i>Medium</i>
Frequency of Use:	<i>Medium</i>
Business Rules:	<i>BR-05, BR-06, BR-07, BR-08, BR-09, BR-25</i>
Other Information:	<i>Implementation rule: A suggestion is not a reservation. Booking creation remains the only operation that creates a slot hold. UC-63 adds ranked explanation and handoff value beyond UC-12's ordinary availability list.</i>
Assumptions:	<i>sessionStorage is available within the same browser tab.</i>

III. Design Specifications

1. Account & Customer Status

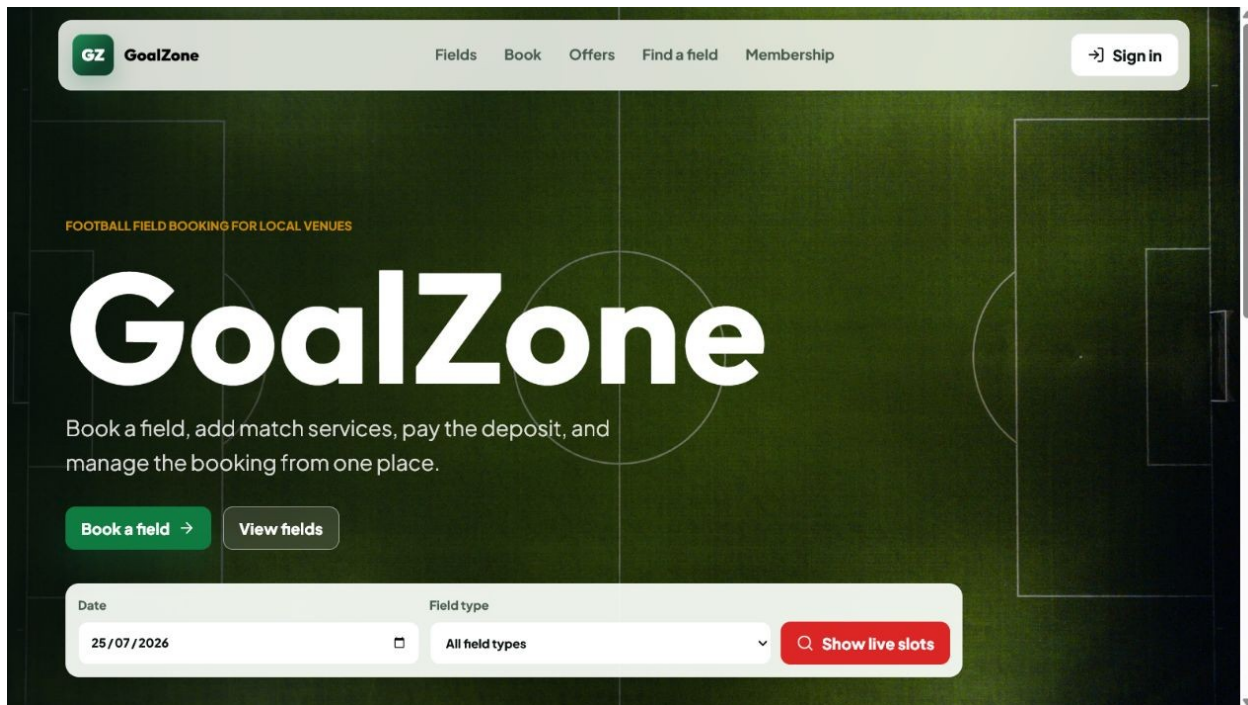
1.1 Screen Designs

a. Landing Page

Public entry point for browsing fields, active promotions, membership benefits, and account actions.

Related use cases: UC-10, UC-11, UC-50, UC-55

UI Design



Field Name	Field Type	Description
Primary navigation	Navigation	Opens Fields, Find a slot, promotion/membership information, Login, or the current role workspace.
Featured fields	Card list	Shows active field image, type, location, surface, capacity, and price context.
Active promotions	Card list	Shows active campaign name, eligibility period, banner, and discount summary.
Membership benefits	Summary list	Shows active membership levels and benefit descriptions.
Sign in / Register	Buttons	Opens the account authentication flows.

Database Access

Table	CRUD	Description
field, field_type, field_price	R	Loads public active field catalogue and price context.
promotion	R	Loads active public promotions.
membership_level	R	Loads active public membership benefits.

Detailed as-built SQL is specified under UC-10, UC-11, UC-50, UC-55 in the SDS Database Queries sections.

b. Login Screen

Authenticates a registered user and routes the user to role-appropriate navigation.

Related use cases: UC-02, UC-03

UI Design

GoalZone Fields Book Offers Find a field Membership →] Sign in

ACCOUNT

Sign in before checkout

Customers need an account to hold a field and receive payment, cancellation, and refund updates.

Login Register

Email or phone

Password

Forgot password?

→] Login

Field Name	Field Type	Description
Email or phone	Text box	Required account identity.
Password	Password box	Required credential; characters are masked.
Login	Button	Submits credentials and stores the returned JWT/user summary.
Forgot password	Link	Opens the password-reset request screen.
Register	Link	Opens Customer registration.

Database Access

Table	CRUD	Description
app_user, role	R/U	Finds the identity with role, validates account state, and records last_login_at.

Detailed as-built SQL is specified under UC-02, UC-03 in the SDS Database Queries sections.

c. Register Account

Creates a unique Customer account and starts email verification.

Related use cases: UC-01

UI Design

GoalZone Fields Book Offers Find a field Membership [→ Sign in](#)

ACCOUNT

Sign in before checkout

Customers need an account to hold a field and receive payment, cancellation, and refund updates.

Login Register

Full name

Email

Phone

Password

At least 8 characters with uppercase, lowercase, number, and special character.

Field Name	Field Type	Description
Full name	Text box	Required Customer name.
Email	Email box	Required unique email used for verification and login.
Phone	Text box	Required unique 10-digit phone beginning with 0.
Password	Password box	Required 8-72 character policy-compliant password.
Confirm password	Password box	Must match Password.
Create account	Button	Validates and creates the Customer account.

Database Access

Table	CRUD	Description
app_user, role	R/C	Checks identity uniqueness, loads Customer role, and creates the account.
membership_level, customer_membership	R/C	Finds the default level and creates Customer membership progress.

Detailed as-built SQL is specified under UC-01 in the SDS Database Queries sections.

d. [Forgot/Reset Password](#)

Issues a time-limited reset link and accepts a replacement password.

Related use cases: UC-06

UI Design

GoalZone Fields Book Offers Find a field Membership →] Sign in

PASSWORD RESET

Forgot password

Enter your email and we will send a reset link.

Email

Send reset link

← Back to login

GoalZone Fields Booking Login

Book local football fields, manage payments, and keep match-day service in one place.

Field Name	Field Type	Description
Email	Email box	Requests a reset link without revealing whether the address exists.
Reset link token	URL parameter	One-hour single-use token tied to user ID.
New password	Password box	Must satisfy the shared password policy.
Confirm password	Password box	Must match the new password.
Reset password	Button	Stores the new hash and revokes existing JWTs.

Database Access

Table	CRUD	Description
app_user	R/U	Finds active account, stores reset token/time, then stores the new hash and clears reset data.

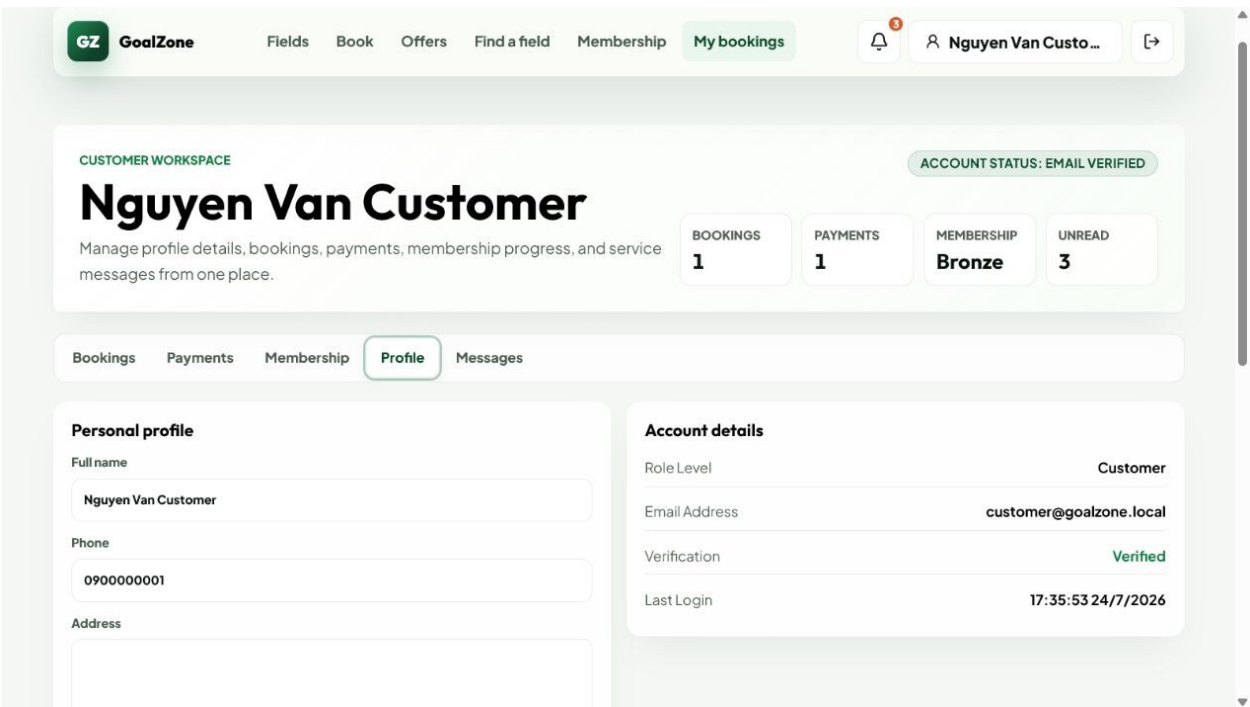
Detailed as-built SQL is specified under UC-06 in the SDS Database Queries sections.

e. Profile / Edit Profile

Displays account status and permits safe profile/password changes.

Related use cases: UC-04, UC-05

UI Design



Field Name	Field Type	Description
Email / role / status	Read-only text	Displays protected identity and access data.
Full name	Text box	Optional profile name update.
Phone	Text box	Optional unique phone update.
Address	Text area	Optional address.
Change password	Action	Opens current/new password fields and revokes active tokens on success.

Database Access

Table	CRUD	Description
app_user	R/U	Loads and updates allowed profile fields or password/auth_version.

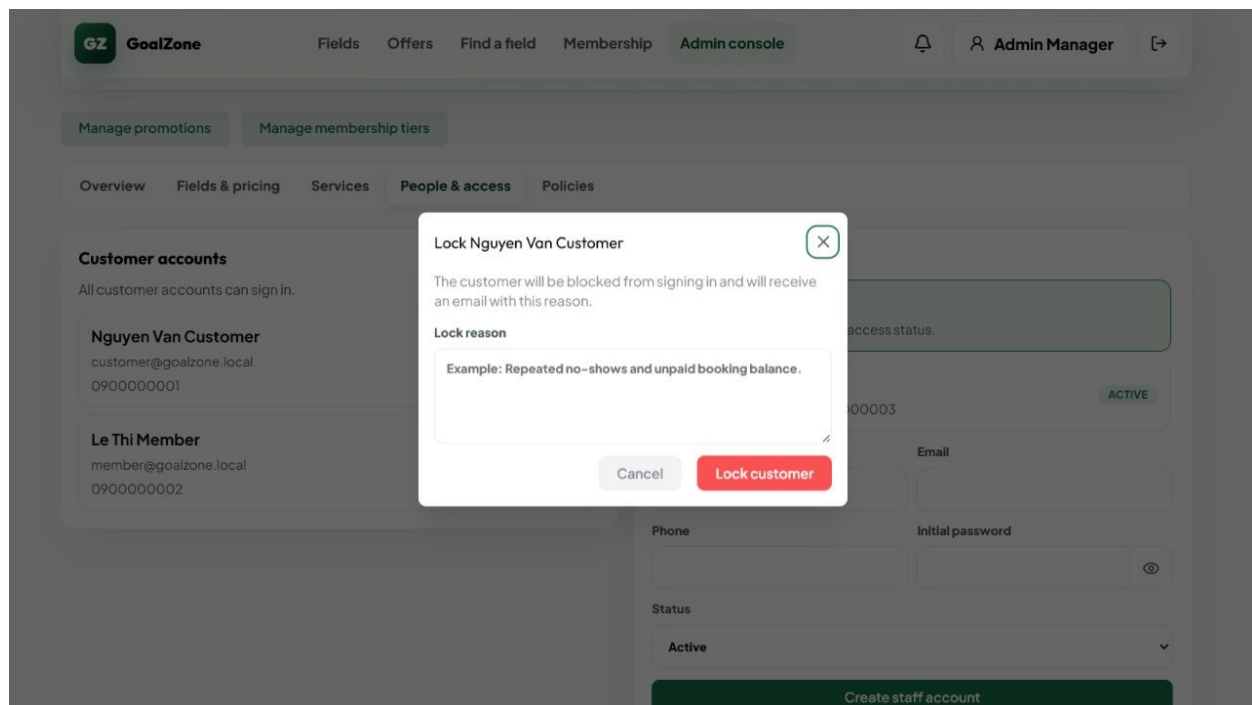
Detailed as-built SQL is specified under UC-04, UC-05 in the SDS Database Queries sections.

f. Customer Management

Admin reviews Customer accounts, manages account lock/unlock, and alone may inspect selected Customer activity.

Related use cases: UC-07, UC-09

UI Design



Field Name	Field Type	Description
<i>Customer list</i>	<i>Data list</i>	<i>Shows Customer name, email, phone, account status, and lock reason.</i>
<i>Lock action</i>	<i>Button</i>	<i>Admin opens the lock-reason dialog for an unlocked Customer.</i>
<i>Lock reason</i>	<i>Modal text area</i>	<i>Admin must enter a reason before locking.</i>
<i>Unlock action</i>	<i>Button</i>	<i>Admin unlocks a locked Customer account.</i>
<i>Account state</i>	<i>Badge</i>	<i>Shows whether the Customer account is active or locked.</i>

Database Access

Table	CRUD	Description
<i>app_user, role</i>	<i>R/U</i>	<i>Lists Customers and updates status/lock_reason/auth_version.</i>
<i>booking</i>	<i>R</i>	<i>Loads Customer booking activity.</i>
<i>issue</i>	<i>R</i>	<i>Loads issues reported by the Customer.</i>
<i>customer_membership, membership_level</i>	<i>R</i>	<i>Displays membership context.</i>

Detailed as-built SQL is specified under UC-07, UC-09 in the SDS Database Queries sections.

[g. Staff Management](#)

Admin creates and maintains Staff identity/status; Staff chooses the password from an emailed invitation.

Related use cases: UC-08

UI Design

The image shows a screenshot of the 'Admin console' in the 'GoalZone' application. The top navigation bar includes 'Fields', 'Offers', 'Find a field', 'Membership', and 'Admin console'. The 'Admin console' section has sub-tabs for 'Manage promotions', 'Manage membership tiers', 'Overview', 'Fields & pricing', 'Services', 'People & access', and 'Policies'. The 'People & access' tab is active, showing two main sections: 'Customer accounts' and 'Staff accounts'.

Customer accounts: This section lists all customer accounts that can sign in. It shows two accounts: 'Nguyen Van Customer' (customer@goalzone.local, 0900000001) and 'Le Thi Member' (member@goalzone.local, 0900000002). Each account has a 'Lock' button.

Staff accounts: This section allows creating a new staff account and managing existing ones. It shows a 'New staff account' form with fields for 'Full name', 'Email', 'Phone', and 'Initial password'. There is also a 'Status' dropdown menu set to 'Active'. A 'Create staff account' button is at the bottom.

Field Name	Field Type	Description
Staff list	Data list	Shows Staff identity and active/locked status.
Full name	Text box	Required Staff name.
Email	Email box	Required unique email.
Phone	Text box	Required unique phone.
Invitation delivery	Status message	Reports whether the one-hour Set my password invitation was sent.
Status	Modal text area	Active or locked/inactive account state.
Save	Button	Creates or updates Staff without exposing any Admin password control.

Database Access

Table	CRUD	Description
app_user, role	R/C/U	Loads Staff role/list and persists identity, status, an unknown initial hash, setup token, and auth_version.

Detailed as-built SQL is specified under UC-08 in the SDS Database Queries sections.

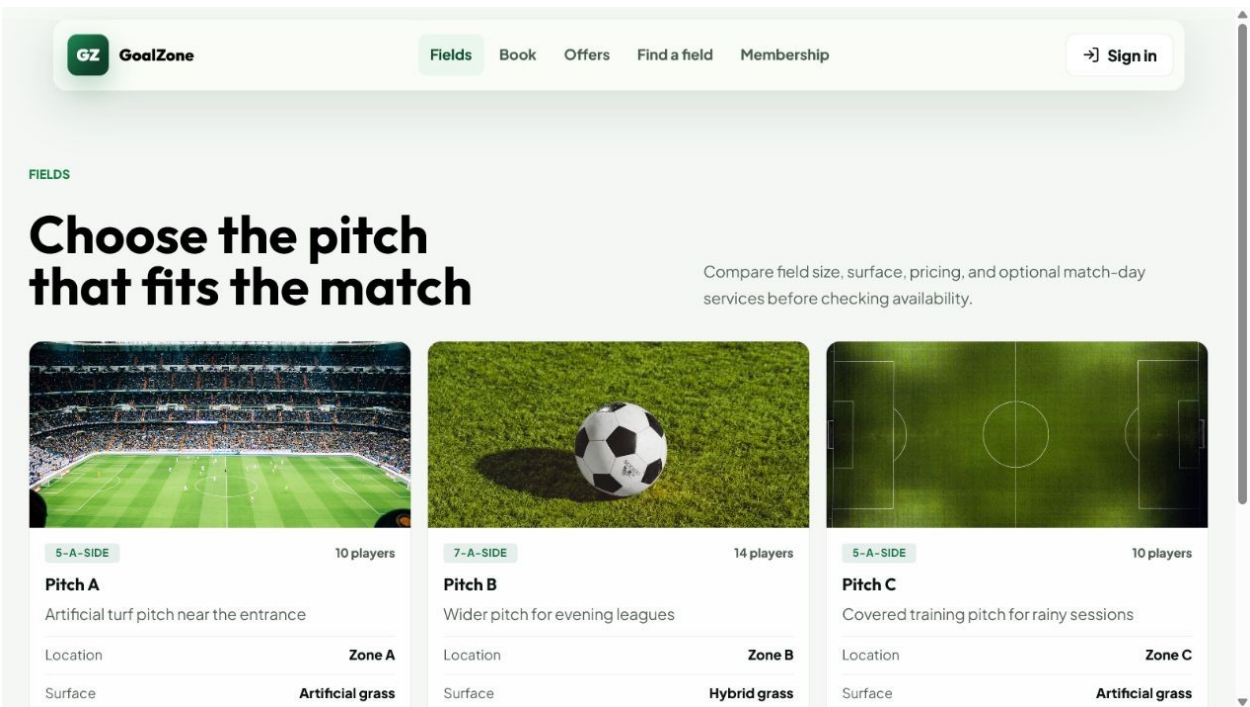
2. Field, Slot, Service & Issue Management

2.1 Screen Designs

a. Field List

Shows the public catalogue of active football fields.

Related use cases: UC-10



Field Name	Field Type	Description
Field cards	Card list	Shows image, name, type, location, surface, capacity, and price range.
View details	Button	Opens selected Field Detail.
Find a slot	Button	Opens availability search for the field.

Database Access

Table	CRUD	Description
field, field_type	R	Loads active field profiles.
field_price	R	Builds public price context.

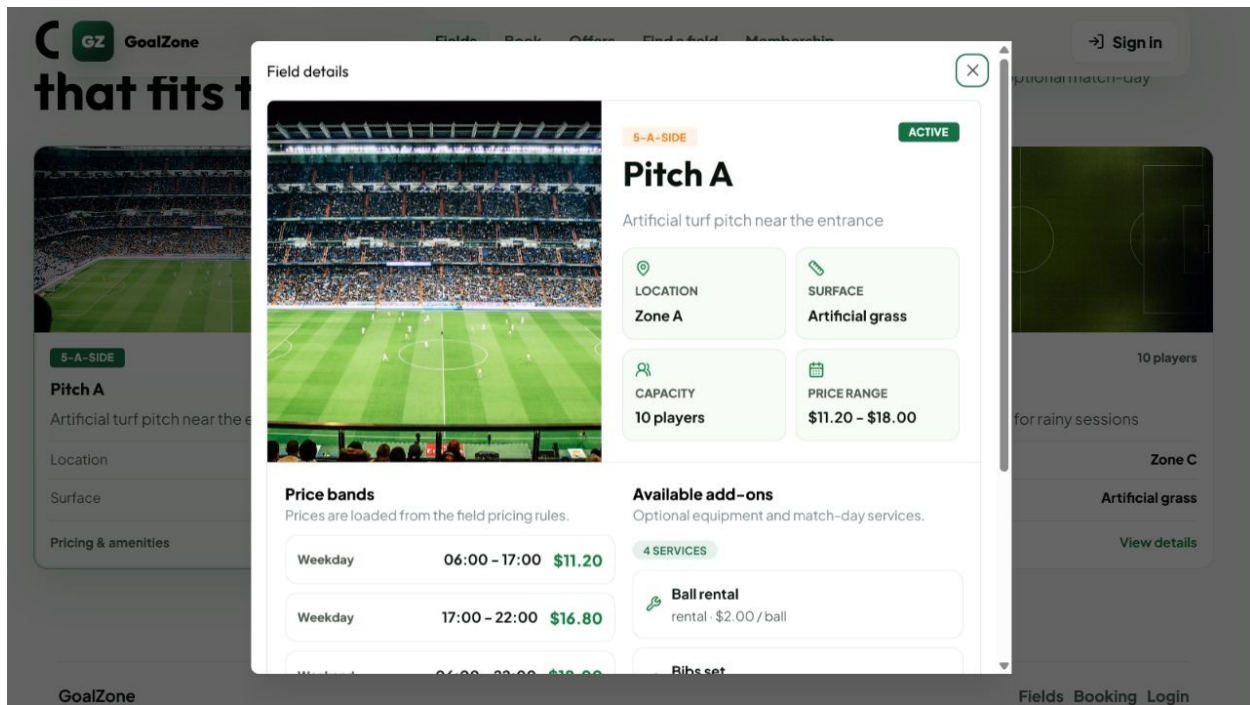
Detailed as-built SQL is specified under UC-10 in the SDS Database Queries sections.

b. Field Detail

Displays one active field with image, facilities, pricing, and booking entry points.

Related use cases: UC-11

UI Design



Field Name	Field Type	Description
Field image	Image	Displays the uploaded field image or safe fallback.
Profile	Read-only group	Name, type, location, surface, capacity, and description.
Price bands	Table	Active weekday/weekend time bands and USD prices.
Available services	List	Active add-ons relevant to booking.
Find available slot	Button	Opens availability search.

Database Access

Table	CRUD	Description
field, field_type	R	Loads selected active field.
field_price	R	Loads active price rules.
extra_service	R	Loads active add-ons.

Detailed as-built SQL is specified under UC-11 in the SDS Database Queries sections.

c. Search Available Fields

Searches generated slots by playing date, field type, and optional field.

Related use cases: UC-12

UI Design

GZ

GoalZone

Fields

Walk-in booking

Offers

Find a field

Membership

Staff workspace

Staff Operator

WALK-IN BOOKING

Create a booking
at the venue

Choose the customer and an available slot, then record the cash payment at the counter.

Playing date

25 / 07 / 2026

Field type

All

Field

All fields

Customer

Nguyen Van Customer

STEP 1

Choose a time

12 available slot(s)

AVAILABLE

\$18.00

Pitch A

06:00 - 08:00

5-a-side

AVAILABLE

\$18.00

Pitch A

08:00 - 10:00

5-a-side

AVAILABLE

\$18.00

Pitch A

17:00 - 19:00

5-a-side

AVAILABLE

\$18.00

AVAILABLE

\$27.20

AVAILABLE

\$27.20

Walk-in payment

Pitch A

2026-07-25 · 06:00 - 08:00

\$18.00

Promotion code

e.g. WELCOME10

Field price

\$18.00

Services

\$0.00

Deposit

\$5.40

Field Name	Field Type	Description
Playing date	Date picker	Required non-past date.
Field type	Modal text area	Optional active field-type filter.
Field	Modal text area	Optional field filter.
Search	Button	Loads bookable slots.
Available slots	Result list	Shows field, date/time, price, and booking action.

Database Access

Table	CRUD	Description
field, field_type, slot	R/C	Loads active fields and generates/persists daily slots when required.
booking	R	Excludes slots with active bookings.
field_price	R	Resolves applicable USD price.

Detailed as-built SQL is specified under UC-12 in the SDS Database Queries sections.

d. Field Management

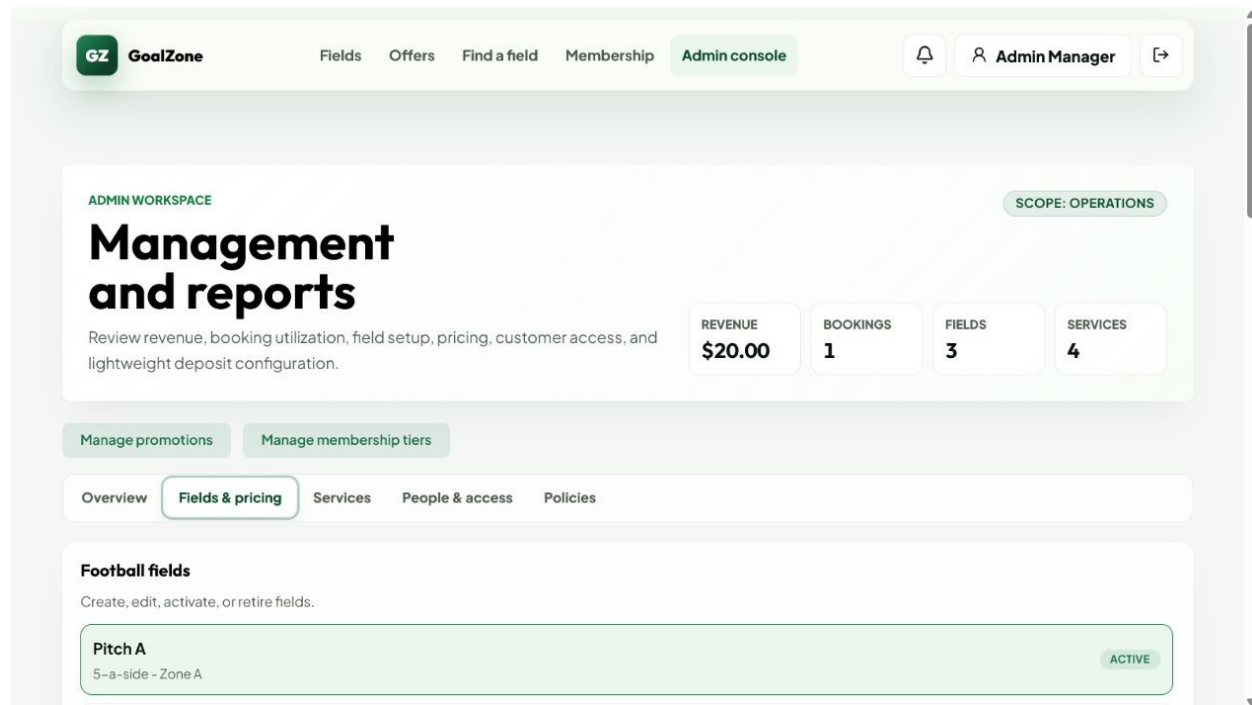
Admin creates, updates, activates, and deactivates football fields, including images.

Related use cases: UC-13

GAMS-SRS_v1.0

Page 100 / 133

UI Design



Field Name	Field Type	Description
<i>Field list</i>	<i>Data list</i>	<i>Shows active and inactive fields.</i>
<i>Name/type/location/surface</i>	<i>Form fields</i>	<i>Required field identity and playing context.</i>
<i>Capacity/description</i>	<i>Form fields</i>	<i>Operational metadata.</i>
<i>Image</i>	<i>File upload</i>	<i>JPEG, PNG, WebP, or GIF up to 5 MB; stores returned image URL.</i>
<i>Status</i>	<i>Modal text area</i>	<i>Active/inactive catalogue state.</i>
<i>Save</i>	<i>Button</i>	<i>Creates or updates the field.</i>

Database Access

Table	CRUD	Description
<i>field, field_type</i>	<i>R/C/U</i>	<i>Loads type data and persists the field profile/image URL/status.</i>

Detailed as-built SQL is specified under UC-13 in the SDS Database Queries sections.

e. Field Pricing Management

Admin maintains effective weekday/weekend price bands for each field.

Related use cases: UC-14

UI Design

The screenshot shows the 'Admin console' of the GoalZone system. The navigation bar includes 'Fields', 'Offers', 'Find a field', 'Membership', and 'Admin console'. The 'Admin console' section displays a form for adding a price rule. The form has the following fields:

- Day type:** A dropdown menu with 'Weekday' selected.
- Start:** A time input field with '06:00 SA' selected.
- End:** A time input field with '08:00 SA' selected.
- Price:** A currency input field.
- Effective from:** A date input field with the format 'dd/mm/yyyy'.
- Effective to:** A date input field with the format 'dd/mm/yyyy'.
- Status:** A dropdown menu with 'Active' selected.

A green button labeled 'Add price rule' is located at the bottom of the form.

Field Name	Field Type	Description
<i>Field</i>	<i>Modal text area</i>	<i>Selects the managed field.</i>
<i>Day type</i>	<i>Modal text area</i>	<i>Weekday or weekend applicability.</i>
<i>Start/end time</i>	<i>Time inputs</i>	<i>Defines a valid non-overlapping pricing range.</i>
<i>USD price</i>	<i>Currency input</i>	<i>Positive field rental price.</i>
<i>Effective dates</i>	<i>Date inputs</i>	<i>Optional rule validity range.</i>
<i>Status</i>	<i>Modal text area</i>	<i>Active/inactive price rule.</i>

Database Access

Table	CRUD	Description
<i>field</i>	<i>R</i>	<i>Validates the selected field.</i>
<i>field_price</i>	<i>R/C/U</i>	<i>Lists and persists field/day/time/effective-date pricing rules.</i>

Detailed as-built SQL is specified under UC-14 in the SDS Database Queries sections.

f. Extra Service Selection

Selects and updates booking add-ons while validating stock/capacity and pre-check-in timing.

Related use cases: UC-18, UC-19, UC-20

UI Design

The screenshot displays the GoalZone booking interface. At the top, there's a navigation bar with 'GZ GoalZone' and links for 'Fields', 'Book', 'Offers', 'Find a field', 'Membership', and 'My bookings'. A user profile 'Nguyen Van Custo...' is visible. The main area shows three available 'Pitch C' options, each for \$14.40, with time slots (08:00-10:00, 17:00-19:00, 19:00-21:00) and '5-a-side' details. Below this, 'STEP 2 Add services' allows selecting optional services: 'Ball rental' (\$2.00/ball), 'Bibs set' (\$2.80/set), 'Water box' (\$3.60/box), and 'Referee service' (\$10.00/match). The right sidebar shows a 'Total' of \$23.60 and two payment options: 'Pay deposit' (\$7.08, leaving \$16.52 for venue payment) and 'Pay in full' (\$23.60). A 'PayPal' button and a 'Thẻ ghi nợ hoặc tín dụng' (Credit card or debit card) option are also present.

Field Name	Field Type	Description
Service	Selectable list	Active rental/sale/staff add-on with USD unit price.
Quantity	Number input	Positive quantity within per-booking and shared availability limits.
Availability message	Status text	Explains stock/capacity rejection.
Service total	Currency summary	Recalculated booking service value.
Save services	Button	Persists selection and updates billing/invoice before check-in.

Database Access

Table	CRUD	Description
extra_service	R	Loads active service rules/stock.
booking_service	R/C/U/D	Reads overlapping usage and persists the booking selection/price snapshot.
booking, invoice, refund	R/U/C	Recalculates total, balance/refund delta, and invoice atomically.

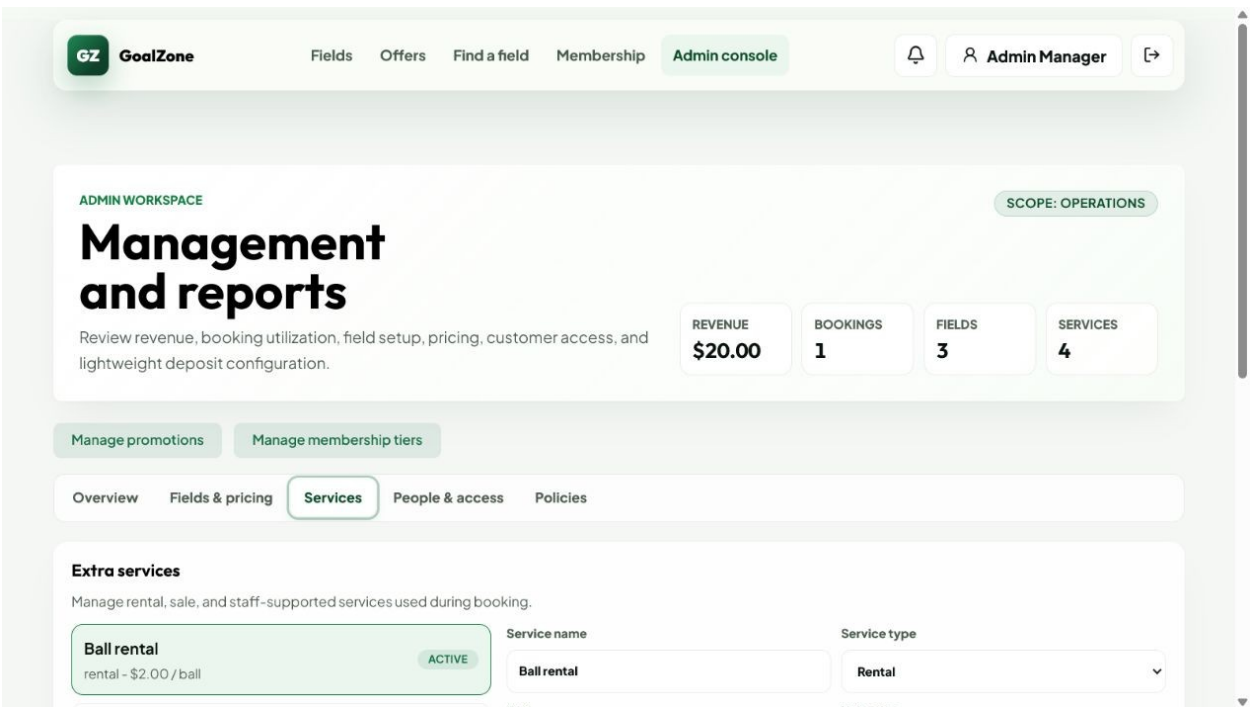
Detailed as-built SQL is specified under UC-18, UC-19, UC-20 in the SDS Database Queries sections.

g. Extra Service Management

Admin maintains rental, sale, and staff-provided add-ons.

Related use cases: UC-17

UI Design



Field Name	Field Type	Description
Service list	Data list	Shows active and inactive services.
Name/type	Form fields	Required service identity and rental/sale/staff category.
USD unit price	Currency input	Non-negative service price.
Stock/max per booking	Number inputs	Optional shared stock and quantity limits.
Status	Modal text area	Active/inactive availability.
Save	Button	Creates or updates the service.

Database Access

Table	CRUD	Description
extra_service	R/C/U	Lists and persists service price, limits, and status.

Detailed as-built SQL is specified under UC-17 in the SDS Database Queries sections.

h. Unavailable Slot Management

Staff blocks/unblocks field time with a reason and reviews daily field operations.

Related use cases: UC-15, UC-16

UI Design

Field Name	Field Type	Description
<i>Date</i>	<i>Date picker</i>	<i>Selected operating date.</i>
<i>Field</i>	<i>Modal text area</i>	<i>Selected active field.</i>
<i>Start/end time</i>	<i>Time inputs</i>	<i>Target slot interval.</i>
<i>Block reason</i>	<i>Text area</i>	<i>Required operational reason.</i>
<i>Block / Unblock</i>	<i>Buttons</i>	<i>Changes slot availability when no protected booking conflict exists.</i>
<i>Daily calendar</i>	<i>Calendar/list</i>	<i>Shows available, blocked, and booked slots with customer/booking context.</i>

Database Access

Table	CRUD	Description
<i>slot, field</i>	<i>R/C/U</i>	<i>Loads/generates daily slots and stores block state/reason/creator.</i>
<i>booking, app_user</i>	<i>R</i>	<i>Shows booked-slot customer context and prevents invalid blocking.</i>

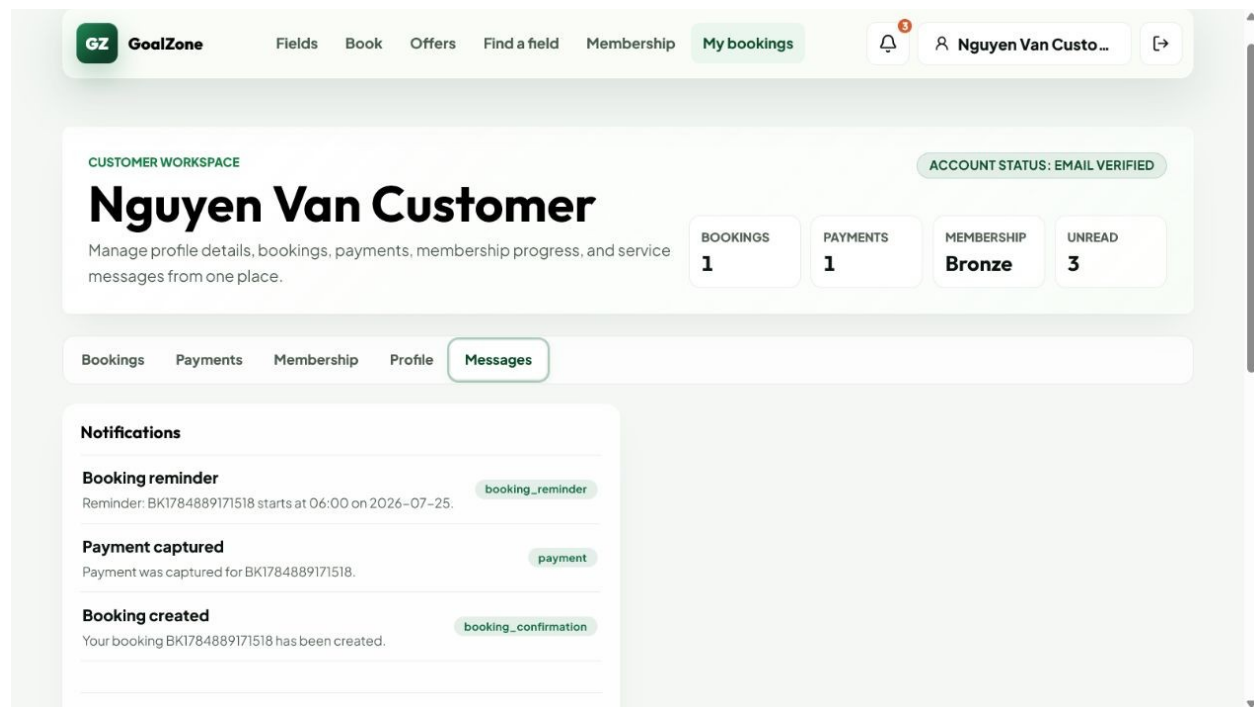
Detailed as-built SQL is specified under UC-15, UC-16 in the SDS Database Queries sections.

i. Issue Report / Issue Management

Customer/Staff reports a field, booking, or service issue; Staff resolves it and notifies the reporter.

Related use cases: UC-21, UC-22

UI Design



Field Name	Field Type	Description
Context	Modal text areas	Optional booking, field, and extra-service references.
Title	Text box	Required concise issue title.
Description	Text area	Required issue details.
Status	Modal text area	Staff-managed issue lifecycle.
Resolution note	Text area	Required operational resolution details when resolved.
Save / Resolve	Buttons	Creates issue or updates its status and notification.

Database Access

Table	CRUD	Description
issue	R/C/U	Stores reporter, context, status, assignee, resolution note/time.
booking, field, extra_service, app_user	R	Validates referenced context and actors.
notification	C	Notifies reporter after resolution is persisted.

Detailed as-built SQL is specified under UC-21, UC-22 in the SDS Database Queries sections.

3. Booking Lifecycle

3.1 Screen Designs

a. Online Booking

Customer creates a pending online booking and continues to Customer-only PayPal checkout.

Related use cases: UC-23, UC-28

UI Design

The screenshot shows the 'Book' section of the GoalZone app. It features a grid of 12 pitch options, each with a time slot, field type (e.g., 5-a-side, 7-a-side), and price. The right sidebar contains a search bar, a user profile, and a payment summary. The summary includes field price, services, deposit, and total, along with two payment options: 'Pay deposit' and 'Pay in full'. A 'PayPal' button and a 'Thẻ ghi nợ hoặc tín dụng' button are also visible.

Field Name	Field Type	Description
<i>Selected field/slot</i>	<i>Read-only summary</i>	<i>Active, available, non-past slot.</i>
<i>Extra services</i>	<i>Selection</i>	<i>Optional validated add-ons.</i>
<i>Promotion code</i>	<i>Text box</i>	<i>Optional eligible promotion.</i>
<i>Price summary</i>	<i>Currency panel</i>	<i>Field, services, discounts, total, deposit, and balance in USD.</i>
<i>Deposit / Full</i>	<i>Choice</i>	<i>Customer online PayPal amount option.</i>
<i>Create and pay</i>	<i>Button</i>	<i>Creates a pending hold then opens PayPal Sandbox checkout.</i>

Database Access

Table	CRUD	Description
<i>booking, slot</i>	<i>C/R/U</i>	<i>Creates pending online booking and protects the slot.</i>
<i>booking_service, booking_promotion</i>	<i>C</i>	<i>Stores service and promotion snapshots.</i>
<i>invoice</i>	<i>C/U</i>	<i>Creates billing summary.</i>
<i>payment</i>	<i>C/U</i>	<i>Stores PayPal order/capture result used for confirmation.</i>

Detailed as-built SQL is specified under UC-23, UC-28 in the SDS Database Queries sections.

b. Walk-in Booking

Staff creates a confirmed booking for an existing on-site Customer and records cash payment.

Related use cases: UC-24, UC-28, UC-41

UI Design

GoalZone Fields **Walk-in booking** Offers Find a field Membership Staff workspace Staff Operator

AVAILABLE **\$14.40**

Pitch C

🕒 08:00 - 10:00

📍 5-a-side

AVAILABLE **\$14.40**

Pitch C

🕒 17:00 - 19:00

📍 5-a-side

AVAILABLE **\$14.40**

Pitch C

🕒 19:00 - 21:00

📍 5-a-side

STEP 2
Add services Optional

Ball rental
\$2.00 / ball

Bibs set
\$2.80 / set

Water box
\$3.60 / box

Referee service
\$10.00 / match

Promotion code
e.g. WELCOME10

Field price **\$18.00**

Services **\$2.00**

Deposit **\$6.00**

Total \$20.00

Payment option

☒ **Pay deposit** **\$6.00**
\$14.00 remains for venue payment

☐ **Pay in full**
No remaining balance after this **\$20.00**

GoalZone Fields Booking Staff Account

Book local football fields, manage payments, and keep match-day service in one place.

Field Name	Field Type	Description
Customer	Modal text area	Existing active Customer selected by Staff.
Field/date/slot	Modal text areas	Available non-past slot.
Extra services	Selection	Optional validated add-ons.
Promotion	Text box	Optional eligible promotion.
Cash option	Choice	Deposit, full, or remaining cash amount recorded on site.
Note	Text area	Optional operational note.
Create booking	Button	Creates confirmed walk-in booking and cash transaction.

Database Access

Table	CRUD	Description
app_user, booking, slot	R/C/U	Validates Customer/Staff and creates confirmed walk-in booking.
booking_service, booking_promotion	C	Stores selected snapshots.
payment, invoice	C/U	Stores on-site cash payment and billing state.

Detailed as-built SQL is specified under UC-24, UC-28, UC-41 in the SDS Database Queries sections.

c. Booking Detail

Role/ownership-protected view of booking, billing, and allowed lifecycle actions.

Related use cases: UC-25, UC-28, UC-29, UC-33, UC-34, UC-35

UI Design

The screenshot displays the 'GoalZone' Staff workspace. At the top, navigation tabs include 'Fields', 'Walk-in booking', 'Offers', 'Find a field', 'Membership', and 'Staff workspace' (which is active). A user profile for 'Staff Operator' is visible in the top right. Below the navigation bar, there are several action buttons: 'Cancel' (red), 'No-show' (yellow), 'Reschedule to...' (dropdown), 'Reschedule' (light blue), and 'Remaining payment' (light blue). The main content area is divided into two panels. The left panel, titled 'Invoice and payment status', shows an invoice for 'INV1784889171578' (BK1784889171518 - Pitch A) with a 'PAID' status. It lists items: Field rental (\$18.00), Extra services (\$2.00), and Discounts (\$0.00). The total is \$20.00, with \$20.00 paid and \$0.00 remaining. A transaction record shows a full payment of \$20.00 via cash on 2026-07-24. The right panel, titled 'Customer booking activity', shows a booking for 'Nguyen Van Customer' with 1 recent booking(s), 0 completed. It lists a booking for 'BK1784889171518 - Pitch A' on 2026-07-25 at 06:00, which is 'confirmed'.

Field Name	Field Type	Description
<i>Booking identity/status</i>	<i>Read-only group</i>	<i>Code, source, lifecycle status, customer, and staff handler.</i>
<i>Field/slot</i>	<i>Read-only group</i>	<i>Field and playing date/time.</i>
<i>Services/promotion</i>	<i>Line list</i>	<i>Selected add-ons and applied promotion snapshot.</i>
<i>Billing</i>	<i>Summary</i>	<i>Total, paid, remaining, invoice, payment, and refund state.</i>
<i>Operational actions</i>	<i>Buttons</i>	<i>Role/status-aware confirm, reject, check-in, complete, no-show, reschedule, cancel, or edit services.</i>

Database Access

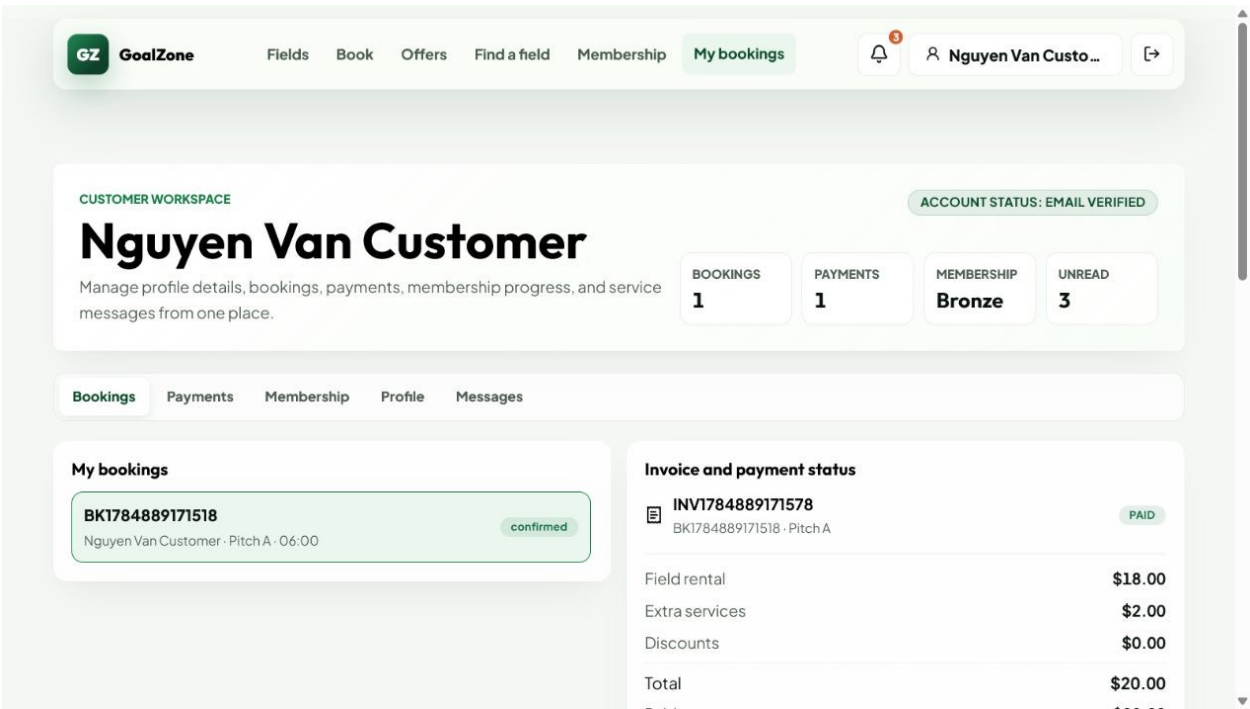
Table	CRUD	Description
<i>booking, slot, field, app_user</i>	<i>R/U</i>	<i>Loads detail and applies valid lifecycle transitions.</i>
<i>booking_service, extra_service, booking_promotion, promotion</i>	<i>R</i>	<i>Loads line items and discounts.</i>
<i>payment, invoice, refund</i>	<i>R</i>	<i>Loads billing and refund history.</i>

Detailed as-built SQL is specified under UC-25, UC-28, UC-29, UC-33, UC-34, UC-35 in the SDS Database Queries sections.

d. My Bookings

Customer workspace for current and past owned bookings.

Related use cases: UC-26
UI Design



Field Name	Field Type	Description
Status filter	Modal text area	Filters current/past lifecycle groups.
Booking list	Data list	Shows code, date/time, field, status, total, and balance.
Open detail	Button	Opens the ownership-protected Booking Detail.

Database Access

Table	CRUD	Description
booking, slot, field	R	Loads bookings for the authenticated Customer.
payment, invoice, refund	R	Builds billing status summaries.

Detailed as-built SQL is specified under UC-26 in the SDS Database Queries sections.

e. Booking Calendar

Staff daily operations queue/calendar with selected-booking controls.

Related use cases: UC-27, UC-28, UC-29, UC-33, UC-34, UC-35, UC-36

UI Design

The screenshot shows the 'GoalZone' Staff workspace. The top navigation bar includes links for Fields, Walk-in booking, Offers, Find a field, Membership, and Staff workspace (which is active). A user profile for 'Staff Operator' is visible. The main section is titled 'Daily field operation' with a subtitle 'Review the booking queue, move bookings through the venue lifecycle, log issues, and process refund cases.' A status filter 'SELECTED: CONFIRMED' is applied. A summary box shows: BOOKINGS (1), PENDING (0), ISSUES (0), and REFUNDS (0). Below this are tabs for 'Bookings', 'Field schedule', and 'Issues & refunds'. The 'Bookings' tab is active, showing a 'Booking calendar' with a confirmed booking for 'BK1784889171518' by 'Nguyen Van Customer' at 'Pitch A' at '06:00'. To the right, 'Lifecycle actions' for this booking are listed: Confirm, Reject, Check-in, Complete, Cancel, and No-show.

Field Name	Field Type	Description
<i>Date</i>	<i>Date picker</i>	<i>Operating day filter.</i>
<i>Field/status</i>	<i>Modal text areas</i>	<i>Optional operation filters.</i>
<i>Daily queue</i>	<i>Calendar/list</i>	<i>Shows booking, slot, customer, source, status, and billing context.</i>
<i>Selected booking</i>	<i>Detail panel</i>	<i>Opens lifecycle, issue, reschedule, cancellation, and payment tools.</i>

Database Access

Table	CRUD	Description
<i>booking, slot, field, app_user</i>	<i>R/U</i>	<i>Loads daily operations and persists valid Staff actions.</i>
<i>issue, payment, invoice, refund</i>	<i>R/C/U</i>	<i>Supports conflict and billing workflows.</i>

Detailed as-built SQL is specified under UC-27, UC-28, UC-29, UC-33, UC-34, UC-35, UC-36 in the SDS Database Queries sections.

f. Reschedule Booking

Moves an eligible owned/operated booking to a free slot and reconciles the price delta.

Related use cases: UC-30, UC-36

UI Design

Field Name	Field Type	Description
<i>Current slot</i>	<i>Read-only summary</i>	<i>Existing field/date/time and price.</i>
<i>New slot</i>	<i>Modal text area</i>	<i>Available active non-past slot.</i>
<i>Price comparison</i>	<i>Currency summary</i>	<i>Old total, new total, paid amount, balance due, or refundable delta.</i>
<i>Confirm reschedule</i>	<i>Button</i>	<i>Atomically moves the booking and updates invoice/refund state.</i>

Database Access

Table	CRUD	Description
<i>booking, slot, field_price</i>	<i>R/U</i>	<i>Validates availability, moves slot, and stores new price summary.</i>
<i>booking_service, booking_promotion</i>	<i>R/U</i>	<i>Revalidates service and discount snapshots.</i>
<i>invoice, refund</i>	<i>U/C</i>	<i>Stores remaining balance or refundable delta while preserving payment history.</i>

Detailed as-built SQL is specified under UC-30, UC-36 in the SDS Database Queries sections.

[g. Cancellation Preview](#)

Shows policy-derived cancellation fee/refundable amount before the user confirms cancellation.

Related use cases: UC-31, UC-32

UI Design

GoalZone Fields Book Offers Find a field Membership **My bookings** Nguyen Van Custo...

Total \$20.00
Paid \$20.00
Remaining \$0.00

Transactions

PAY1784889171573 · Full payment \$20.00
Cash · 17:32 24 Thg 7, 2026 Paid

Cancellation terms
Same-day cancellation before check-in: refund \$16.00, fee \$4.00.

[Refresh terms](#) [Cancel booking](#)

Reschedule to available slot
Choose a new slot

[Reschedule booking](#)

Booking add-ons
Prices, discounts, remaining balance, and invoice are recalculated when you save.

Field Name	Field Type	Description
Booking summary	Read-only group	Selected eligible booking and start time.
Paid amount	Currency	Captured/recorded payment total.
Refund policy	Read-only text	Applicable before-24-hour or same-day rate.
Refundable amount	Currency	Previewed amount before mutation.
Cancel booking	Confirmation button	Applies cancellation and creates eligible refund workflow.

Database Access

Table	CRUD	Description
booking, payment	R/U	Loads eligibility/paid amount and changes status after confirmation.
system_setting	R	Loads configured refund percentages.
refund, invoice, notification	C/U	Creates eligible refund, updates invoice, and notifies after cancellation.

Detailed as-built SQL is specified under UC-31, UC-32 in the SDS Database Queries sections.

4. Payment Gateway, Refund & Invoice

4.1 Screen Designs

a. Checkout Summary

Displays a single USD pricing source of truth before Customer online or Staff walk-in payment choice.

Related use cases: UC-37, UC-38

UI Design

Field Name	Field Type	Description
Field price	Currency line	Applicable field/day/time price.
Services	Currency lines	Quantity and unit-price snapshots.
Promotion discount	Currency line	Eligible promotion amount.
Membership discount	Currency line	Customer benefit amount where applicable.
Total/deposit/balance	Currency summary	Final payable values in USD.
Payment choice	Choice	Customer: online deposit/full; Staff walk-in: cash deposit/full/remaining.

Database Access

Table	CRUD	Description
slot, field_price, extra_service	R	Calculates field/service value.
promotion, customer_membership, membership_level	R	Validates discounts.
system_setting	R	Loads configured deposit percentage.

Detailed as-built SQL is specified under UC-37, UC-38 in the SDS Database Queries sections.

b. Payment Sandbox

Customer-only PayPal Sandbox Orders v2 create/capture/cancel flow.

Related use cases: UC-39, UC-40, UC-42

UI Design

The screenshot displays the GoalZone booking interface. At the top, there's a navigation bar with 'GZ GoalZone' and links for 'Fields', 'Book', 'Offers', 'Find a field', 'Membership', and 'My bookings'. A user profile 'Nguyen Van Custo...' is visible. Below the navigation, three 'Pitch C' options are shown, each for \$14.40, with time slots 08:00-10:00, 17:00-19:00, and 19:00-21:00, all for '5-a-side'. A 'STEP 2 Add services' section allows selecting optional services: 'Ball rental' (\$2.00/ball), 'Bibs set' (\$2.80/set), 'Water box' (\$3.60/box), and 'Referee service' (\$10.00/match). The 'Total' is \$23.60. Payment options include 'Pay deposit' (\$7.08, leaving \$16.52 for venue payment) and 'Pay in full' (\$23.60). A 'PayPal' button and a 'Thẻ ghi nợ hoặc tín dụng' (Credit card or debit card) option are also present. The footer includes 'GoalZone' logo, a description 'Book local football fields, manage payments, and keep match-day service in one place.', and links for 'Fields', 'Booking', and 'Account'.

Field Name	Field Type	Description
Payment amount	Read-only currency	Server-calculated deposit or full amount in USD.
PayPal Buttons	Provider component	Creates and approves the sandbox order for the Customer.
Provider status	Status message	Shows created, captured, cancelled, failed, or unavailable state.
Retry	Button	Allows a safe new attempt after a non-final failure.

Database Access

Table	CRUD	Description
booking, invoice	R/U	Validates ownership/amount and reconciles confirmed booking/billing state.
payment	C/U	Stores provider order ID, capture ID, transaction code, amount, method, status, and time.

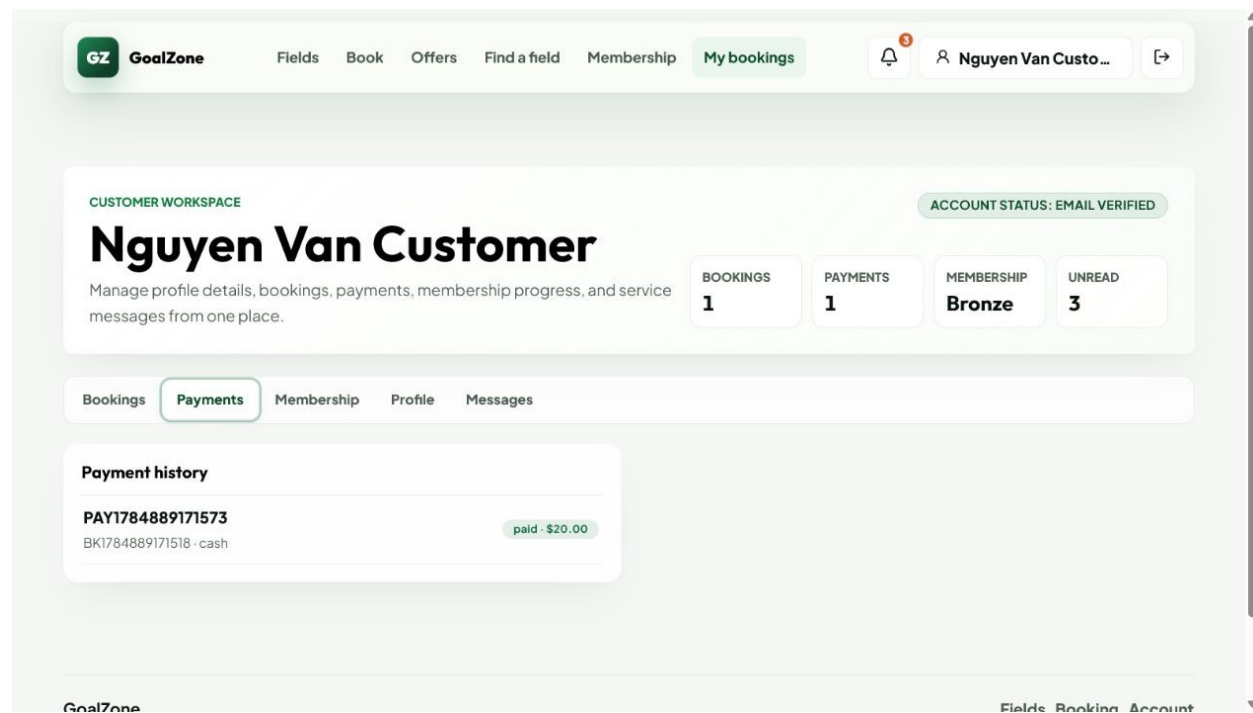
Detailed as-built SQL is specified under UC-39, UC-40, UC-42 in the SDS Database Queries sections.

c. Payment History

Shows real persisted transaction history subject to booking ownership/operator access.

Related use cases: UC-43

UI Design



Field Name	Field Type	Description
Transaction list	Table/list	Payment code, booking, method, amount, status, provider reference, and paid time.
Status filter	Modal text area	Optional payment-status filter.
Open booking	Link	Opens related Booking Detail/Invoice.

Database Access

Table	CRUD	Description
payment, booking, app_user	R	Loads authorized persisted transactions and related booking context.

Detailed as-built SQL is specified under UC-43 in the SDS Database Queries sections.

d. Invoice Detail

Displays the generated booking invoice and current settlement state.

Related use cases: UC-44, UC-45

UI Design

Field Name	Field Type	Description
<i>Invoice code/status</i>	<i>Read-only text</i>	<i>Invoice identity and status.</i>
<i>Field/services</i>	<i>Currency lines</i>	<i>Rental and add-on amounts.</i>
<i>Discounts</i>	<i>Currency lines</i>	<i>Promotion and membership discounts.</i>
<i>Total/paid/remaining</i>	<i>Currency summary</i>	<i>Settlement state in USD.</i>
<i>Refunded amount</i>	<i>Currency</i>	<i>Completed refund total.</i>
<i>Payment history</i>	<i>Transaction list</i>	<i>Related persisted payments.</i>

Database Access

Table	CRUD	Description
<i>invoice, booking</i>	<i>R</i>	<i>Loads invoice snapshot and booking totals.</i>
<i>booking_service, booking_promotion</i>	<i>R</i>	<i>Loads auditable line/discount snapshots.</i>
<i>payment, refund</i>	<i>R</i>	<i>Loads paid and refunded transactions.</i>

Detailed as-built SQL is specified under UC-44, UC-45 in the SDS Database Queries sections.

e. Refund Management

Customer submits an eligible request; Staff reviews it and completes PayPal or cash refund workflows.

Related use cases: UC-46, UC-47

UI Design

STAFF WORKSPACE

Daily field operation

Review the booking queue, move bookings through the venue lifecycle, log issues, and process refund cases.

BOOKINGS: 1, PENDING: 0, ISSUES: 0, REFUNDS: 0

SELECTED: CONFIRMED

Bookings | Field schedule | **Issues & refunds**

Issue report

Title:

Description:

Refunds

[Request refund for selected booking](#)

No records yet.

Field Name	Field Type	Description
Refund queue	Data list	Booking, payment method, requested/approved amount, status, reason, and provider IDs.
Approve / Reject	Buttons	Staff decision with valid state transition.
Submit / Retry PayPal	Button	Calls provider refund by capture ID with idempotency.
Record cash completion	Button	Explicitly confirms cash was returned on site.
Provider result	Status message	Only provider COMPLETED is final for PayPal.

Database Access

Table	CRUD	Description
refund, payment, booking	R/C/U	Creates owner-authorized Customer requests and stores Staff/provider handling state.
invoice	U	Updates refunded amount/status after final completion.
notification	C	Notifies Customer after completed refund is persisted.

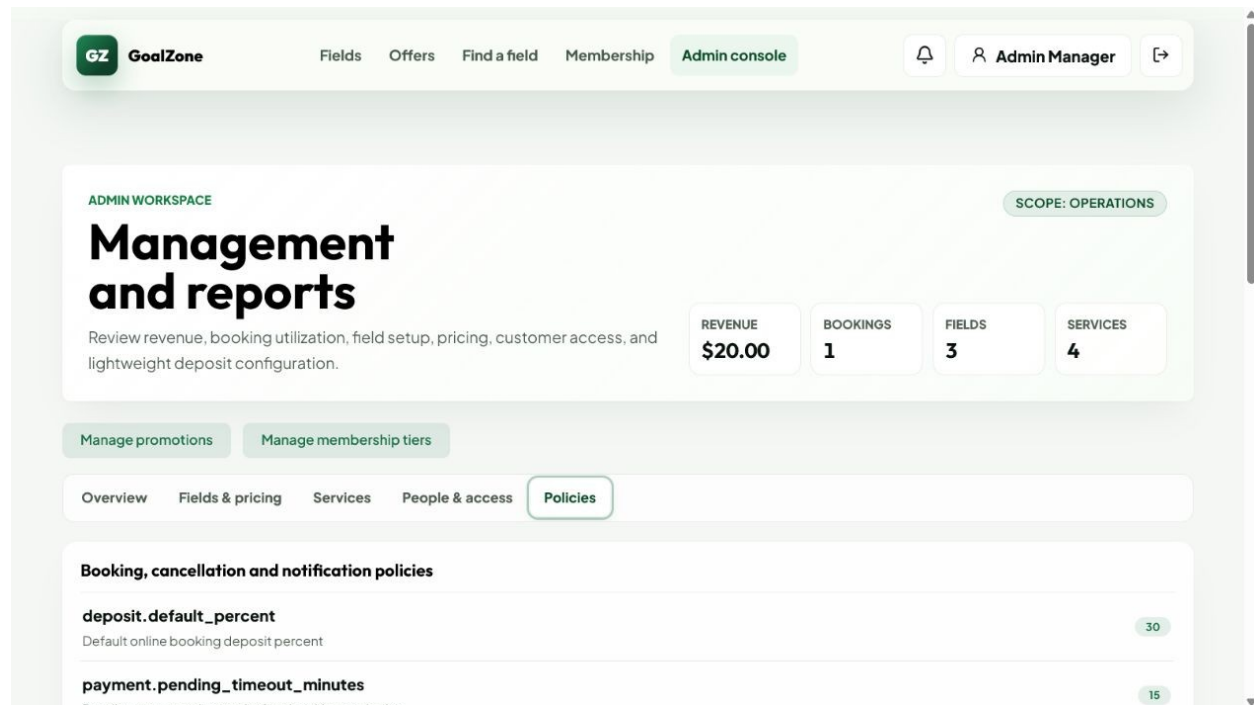
Detailed as-built SQL is specified under UC-46, UC-47 in the SDS Database Queries sections.

[f. Booking and Refund Policies](#)

Admin maintains deposit, payment-timeout, cancellation/refund, and reminder settings.

Related use cases: UC-48, UC-49

UI Design



Field Name	Field Type	Description
Default deposit percent	Number input	Online booking deposit percentage.
Pending payment timeout	Number input	Minutes before an unpaid hold expires.
Before-24-hour refund percent	Number input	Refund percentage for early cancellation.
Same-day refund percent	Number input	Refund percentage for same-day cancellation before check-in.
Reminder hours	Number input	Lead time for one booking reminder.
Save	Button	Validates and persists each system setting.

Database Access

Table	CRUD	Description
system_setting	R/U	Loads and updates typed policy values used by booking/payment schedulers.

Detailed as-built SQL is specified under UC-48, UC-49 in the SDS Database Queries sections.

5. Promotion, Membership, Notification & Report

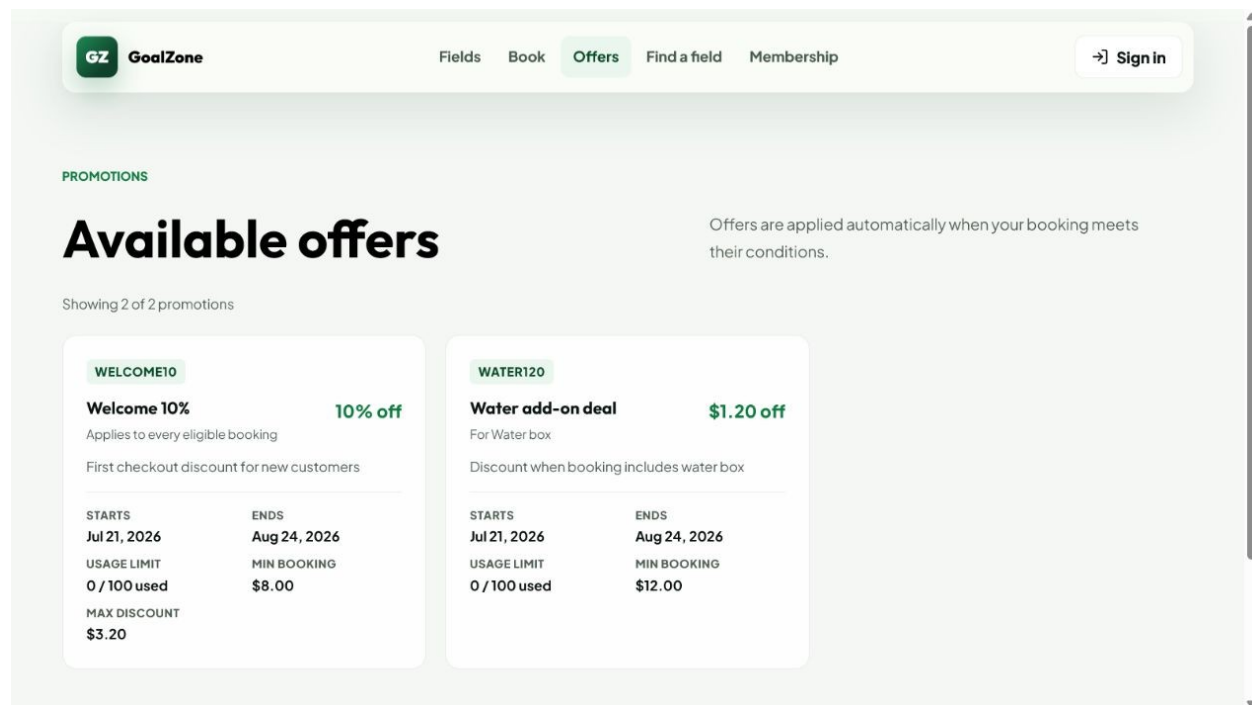
5.1 Screen Designs

a. Promotion List

Public list of active, date-valid promotion campaigns and event sales.

Related use cases: UC-50

UI Design



Field Name	Field Type	Description
Promotion cards	Card list	Banner, name, code, period, and discount summary.
Eligibility summary	Read-only text	Minimum amount and applicable conditions where public.

Database Access

Table	CRUD	Description
promotion	R	Loads active public campaigns.

Detailed as-built SQL is specified under UC-50 in the SDS Database Queries sections.

b. Promotion Management

Admin creates/updates campaigns; checkout validates and snapshots one eligible code.

Related use cases: UC-51, UC-52

UI Design

PROMOTIONS

Promotion catalogue

Create, schedule, and limit offers customers can use at checkout.

PROMOTION EDITOR Set the value first, then narrow where the offer applies.

Create an offer

Code: e.g. SUMMER20

Name: Promotion name

Discount Type: Percent (%)

Discount (%): 10

Max Discount USD (optional): Leave blank for no cap

Min Booking USD (optional): Leave blank for any amount

Start Date: 25/07/2026

End Date: 25/07/2026

Usage Limit: 100

Status: Active

Condition: Field Type: All field types

Condition: Extra Service: All services

Condition: Membership: All memberships

Condition: Day: Any day

Condition: Starts after: ---:--:--

Condition: Ends before: ---:--:--

Field Name	Field Type	Description
<i>Campaign list</i>	<i>Data list</i>	<i>Active/inactive campaigns and usage.</i>
<i>Code/name/banner</i>	<i>Form fields</i>	<i>Unique campaign identity and optional uploaded banner URL.</i>
<i>Discount type/value</i>	<i>Inputs</i>	<i>Percent or fixed USD discount with valid bounds.</i>
<i>Start/end and usage limit</i>	<i>Inputs</i>	<i>Eligibility period and maximum use.</i>
<i>Eligibility conditions</i>	<i>Inputs</i>	<i>Minimum amount, field type, service, membership, day, and time conditions.</i>
<i>Status</i>	<i>Modal text area</i>	<i>Active/inactive campaign.</i>

Database Access

Table	CRUD	Description
<i>promotion</i>	<i>R/C/U</i>	<i>Lists and persists campaign/condition/status data.</i>
<i>booking_promotion, booking</i>	<i>R/C</i>	<i>Validates usage and stores the applied discount snapshot.</i>

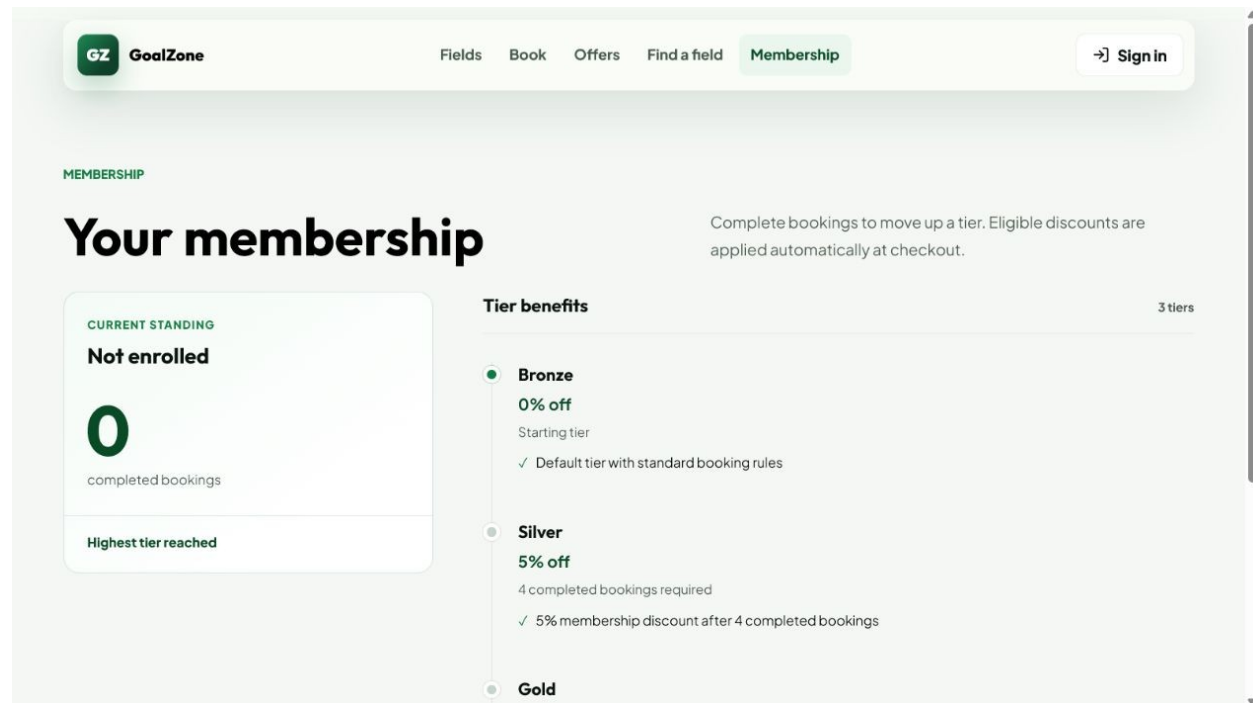
Detailed as-built SQL is specified under UC-51, UC-52 in the SDS Database Queries sections.

c. *Membership Benefits*

Public view of active membership levels, thresholds, and benefit descriptions.

Related use cases: UC-55

UI Design



Field Name	Field Type	Description
Level list	Card/list	Level name, completed-booking threshold, discount percent, and benefit description.

Database Access

Table	CRUD	Description
membership_level	R	Loads active levels ordered by display order.

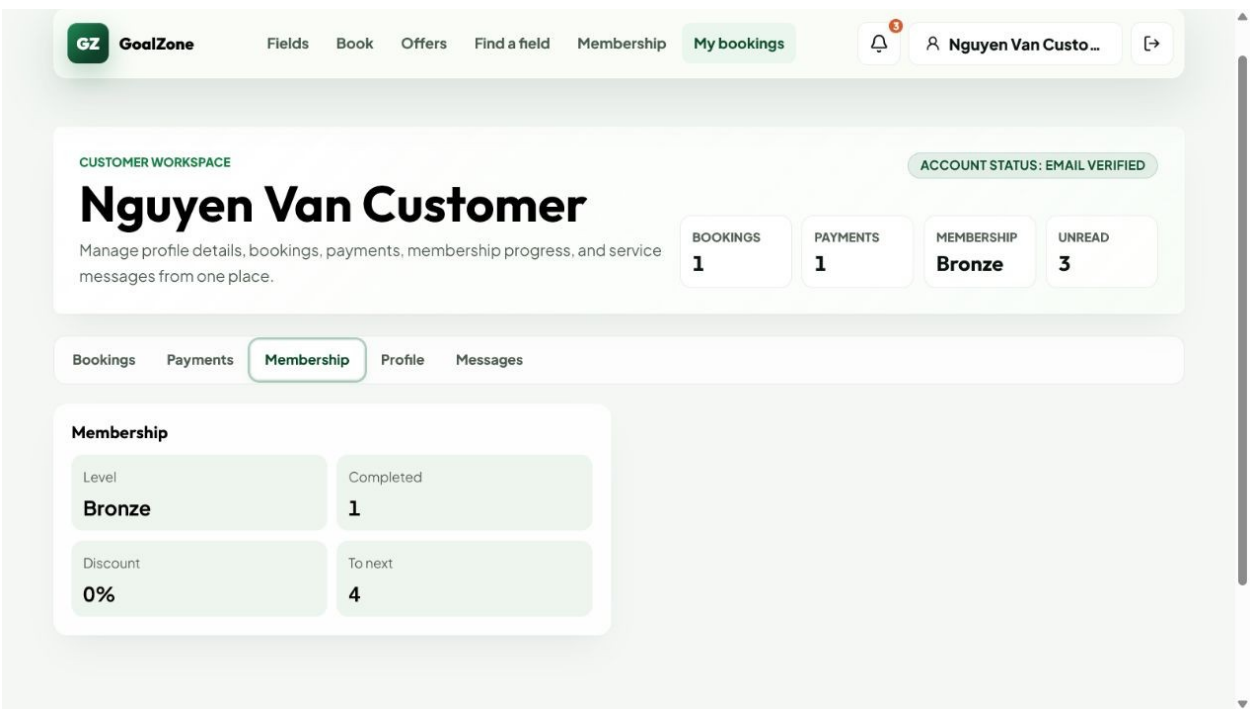
Detailed as-built SQL is specified under UC-55 in the SDS Database Queries sections.

d. Membership Progress

Customer view of current membership and lifetime/monthly/consecutive-week progress toward the next active level.

Related use cases: UC-53

UI Design



Field Name	Field Type	Description
Current level	Read-only text	Current membership level and benefit.
Completed bookings	Number	Lifetime completed-booking audit count.
Next-level rule	Read-only text	Qualification period and remaining booking/streak target.
Progress	Progress indicator	Period-aware progress toward the next active level.

Database Access

Table	CRUD	Description
customer_membership, membership_level	R/U	Loads current progress and updates level after eligible completion.
booking	R	Counts completed bookings only.

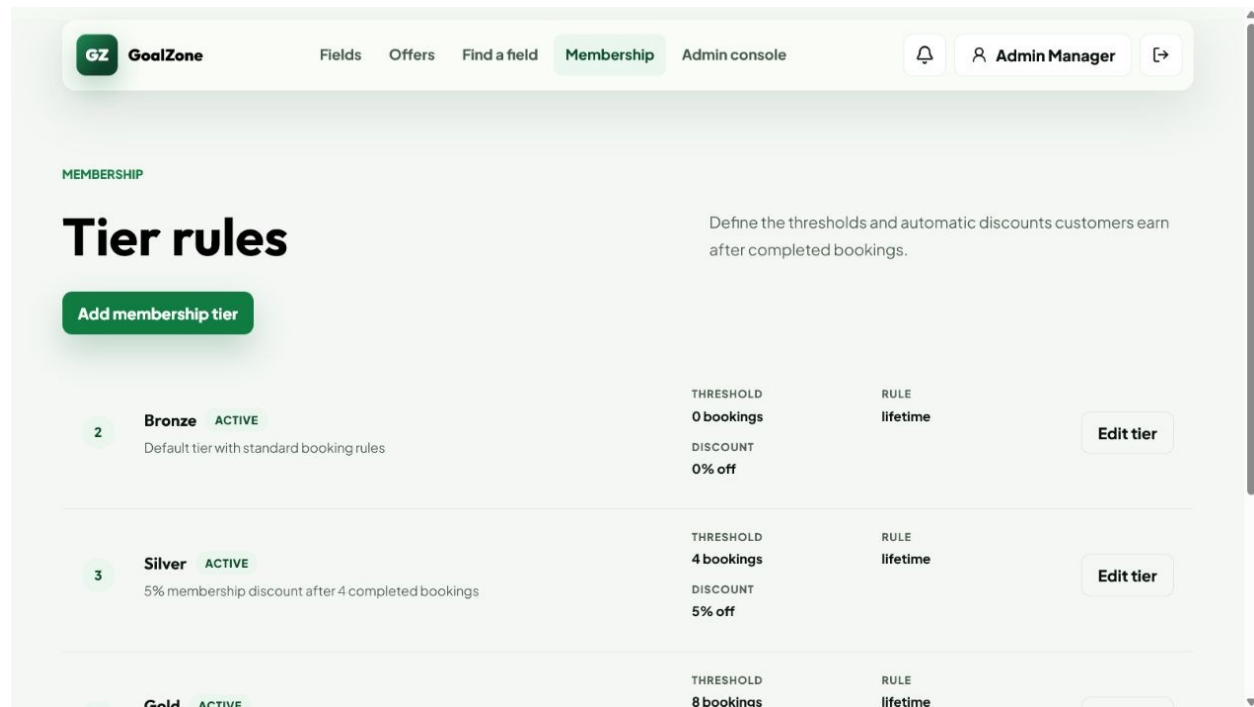
Detailed as-built SQL is specified under UC-53 in the SDS Database Queries sections.

e. Membership Rule Management

Admin maintains ordered membership benefits and lifetime, monthly, or consecutive-week qualification rules.

Related use cases: UC-54

UI Design



Field Name	Field Type	Description
Level list	Data list	Active/inactive levels ordered for progression.
Level name	Text box	Required unique display name.
Required completed bookings	Number input	Lifetime/monthly count or per-week minimum.
Qualification period	Modal text area	Lifetime, monthly, or weekly.
Consecutive weeks	Number input	Required qualifying-week streak for weekly rules.
Discount and benefit	Inputs	Customer-visible benefit configuration.
Display order/status	Inputs	Ordering and active/inactive state.

Database Access

Table	CRUD	Description
membership_level	R/C/U	Lists and persists thresholds, benefits, order, and status.
customer_membership	R/U	Re-evaluates affected Customer level/progress when appropriate.

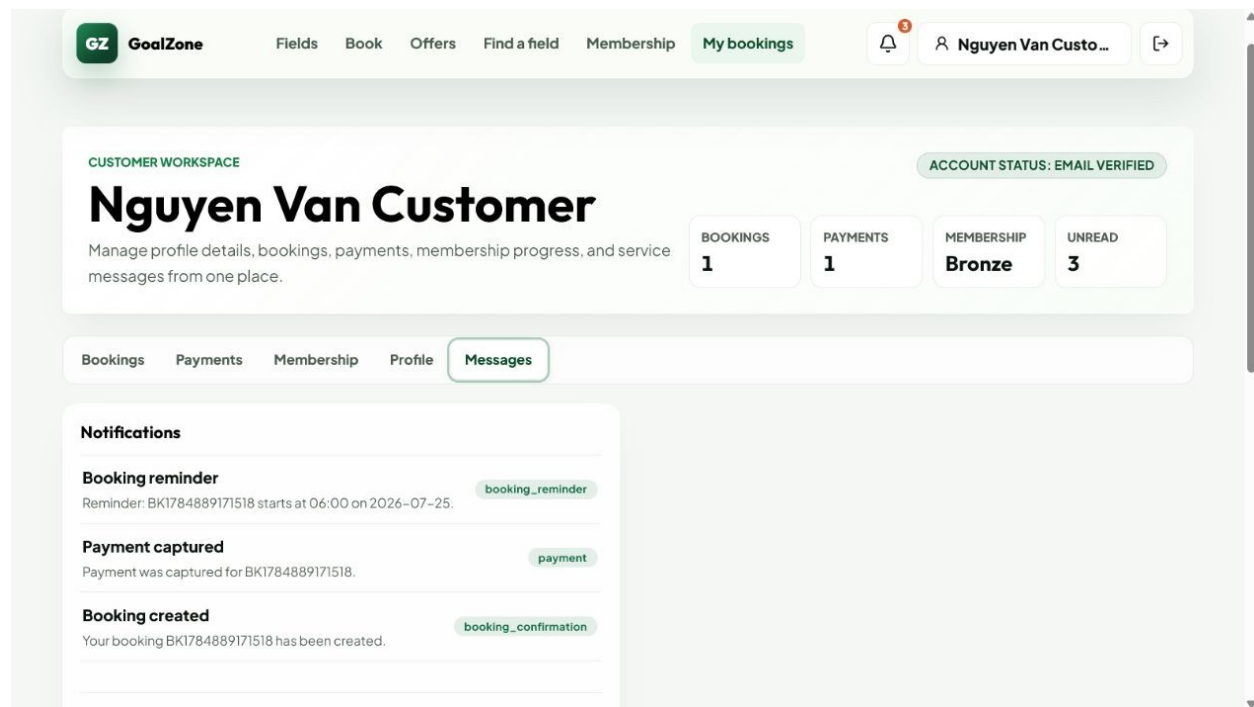
Detailed as-built SQL is specified under UC-54 in the SDS Database Queries sections.

f. Notification List

Displays persisted in-app booking, payment, issue, cancellation, refund, and reminder notifications.

Related use cases: UC-56, UC-57, UC-58

UI Design



Field Name	Field Type	Description
Notification list	List	Type, title, message, related booking/payment/refund, created time, and read state.
Toggle read	Action	Marks one notification read/unread.
Mark all read	Button	Marks the authenticated user's notifications read.

Database Access

Table	CRUD	Description
notification, app_user	R/U	Loads authorized notifications and updates read state.
booking, payment, refund	R	Provides related context links.

Detailed as-built SQL is specified under UC-56, UC-57, UC-58 in the SDS Database Queries sections.

g. Revenue Report

Admin-only aggregate of gross collections, tracked PayPal fees, refunds, net revenue, discounts, and portfolio value in USD.

Related use cases: UC-59

UI Design

The screenshot displays the 'Admin console' of the 'GoalZone' system. The top navigation bar includes links for 'Fields', 'Offers', 'Find a field', 'Membership', and 'Admin console'. The main header area shows 'ADMIN WORKSPACE' and 'SCOPE: OPERATIONS'. The central section is titled 'Management and reports' with a subtitle: 'Review revenue, booking utilization, field setup, pricing, customer access, and lightweight deposit configuration.' To the right of this section are four summary cards: 'REVENUE \$20.00', 'BOOKINGS 1', 'FIELDS 3', and 'SERVICES 4'. Below these are two buttons: 'Manage promotions' and 'Manage membership tiers'. A 'Report date range' section includes 'From' and 'To' date pickers, an 'Apply range' button, and an 'All time' button. A horizontal menu at the bottom of this section includes 'Overview', 'Fields & pricing', 'Services', 'People & access', and 'Policies'. The 'Revenue report' section is partially visible at the bottom.

Field Name	Field Type	Description
Net revenue	KPI	Gross collected minus completed refunds and exact tracked provider fees.
Gross collected	KPI	Successful payment value.
Refunded	KPI	Completed refund value.
PayPal fees	KPI/status	Exact tracked processor fees plus a warning count for legacy rows without provider breakdown.
Field/service value	Breakdown	Booking field and service portfolio amounts.
Promotion/membership discounts	Breakdown	Stored discount snapshots.
Booking totals	Counts	Related lifecycle counts.

Database Access

Table	CRUD	Description
payment, refund, invoice, booking	R	Aggregates stored transaction and invoice amounts.
booking_service, booking_promotion, customer_membership	R	Aggregates service and discount impact.

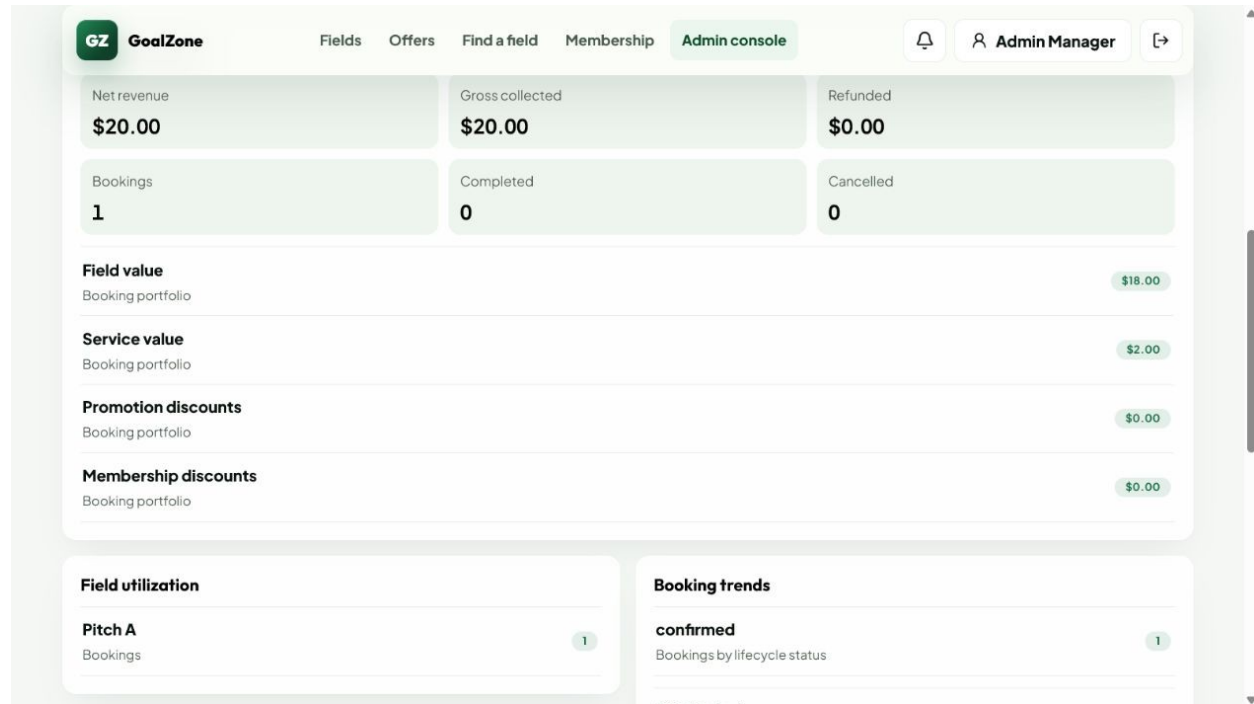
Detailed as-built SQL is specified under UC-59 in the SDS Database Queries sections.

h. Booking Report

Admin-only booking lifecycle, peak-demand, and field-utilization statistics.

Related use cases: UC-60

UI Design



Field Name	Field Type	Description
<i>Lifecycle counts</i>	<i>KPI/list</i>	<i>Pending, confirmed, checked-in, completed, cancelled, no-show, expired, and rejected counts.</i>
<i>Peak start time</i>	<i>Statistic</i>	<i>Most frequent slot start time.</i>
<i>Field utilization</i>	<i>Ranked list</i>	<i>Booking count by field.</i>

Database Access

Table	CRUD	Description
<i>booking, slot, field</i>	<i>R</i>	<i>Aggregates lifecycle, time, and field usage data.</i>

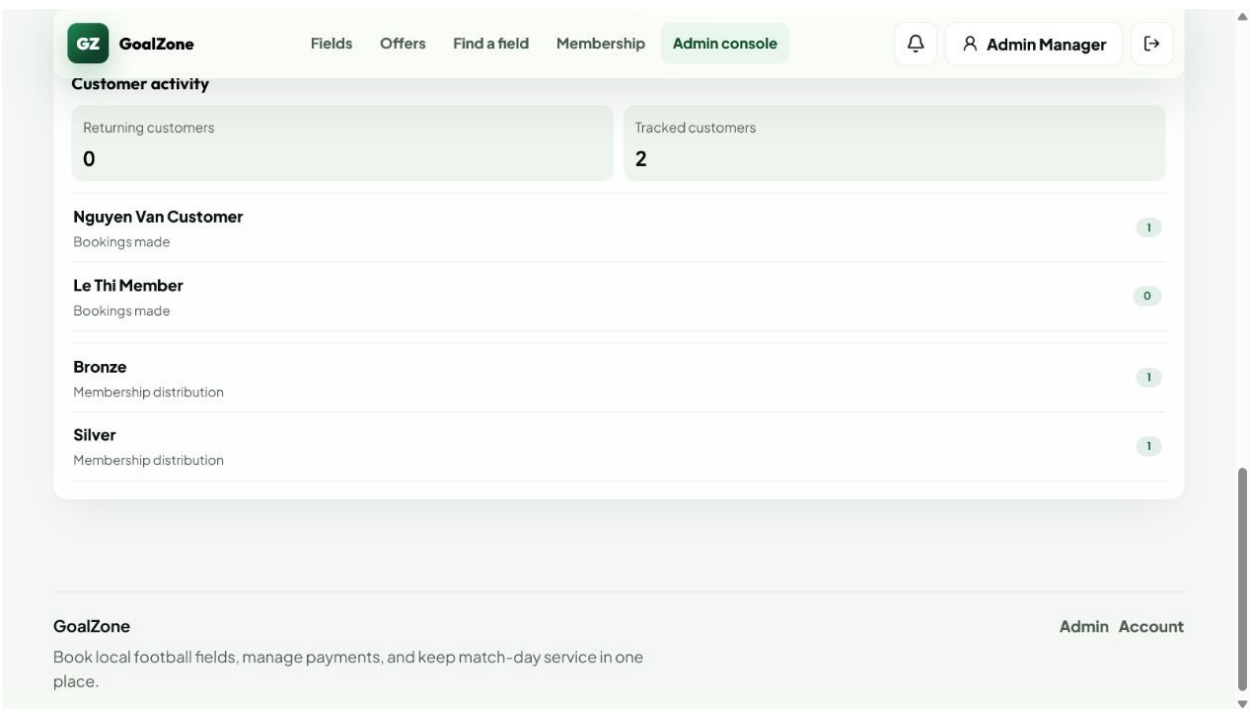
Detailed as-built SQL is specified under UC-60 in the SDS Database Queries sections.

i. Customer Account state Report

Admin-only aggregate of tracked/returning Customers, top activity, and membership distribution.

Related use cases: UC-61

UI Design



Field Name	Field Type	Description
Tracked customers	KPI	Customers included in stored activity.
Returning customers	KPI	Customers with repeated bookings.
Top customers	Ranked list	Customer booking counts.
Membership distribution	Breakdown	Customer counts by membership level.

Database Access

Table	CRUD	Description
app_user, role, booking	R	Aggregates Customer booking behavior.
customer_membership, membership_level	R	Aggregates membership distribution.

Detailed as-built SQL is specified under UC-61 in the SDS Database Queries sections.

j. Availability Assistant

Availability page where a signed-in Customer may ask Gemini and visitors may filter deterministic live ranked slots; a selected result continues into real booking.

Related use cases: UC-62, UC-63

UI Design

GoalZone Fields Book Offers **Find a field** Membership [Sign in](#)

SMART AVAILABILITY

Find the right field

Rule-based suggestions from live availability, your preferred time, budget, and eligible promotions.

Date: 25/07/2026 Preferred time: --:-- -- Field type: Any type Maximum price (USD):

Find best slots

I found 8 available option(s), ranked for your request.

Pitch A \$18.00
5-a-side · 2026-07-25 · 06:00-08:00

- Available 2026-07-25 at 06:00
- Eligible promotion: WELCOME10

Promotion codes: WELCOME10

Pitch B \$27.20
7-a-side · 2026-07-25 · 06:00-08:00

- Available 2026-07-25 at 06:00
- Eligible promotion: WELCOME10

Promotion codes: WELCOME10

Pitch C \$14.40
5-a-side · 2026-07-25 · 06:00-08:00

- Available 2026-07-25 at 06:00
- Eligible promotion: WELCOME10

Promotion codes: WELCOME10

Field Name	Field Type	Description
Date/time/type/budget	Smart form	Structured search criteria.
Ranked suggestions	Result cards	Field, date/time, USD price, reasons, and eligible promotion codes.
Book this slot	Button	Transfers id/date to booking and revalidates availability.

Database Access

Table	CRUD	Description
slot, field, field_type, field_price	R	Loads and prices live candidates.
booking	R	Excludes actively reserved slots.
promotion, membership_level, customer_membership	R	Explains eligible campaign context for the authenticated Customer.

Detailed as-built SQL is specified under UC-62, UC-63 in the SDS Database Queries sections.

IV. Appendix

1. Assumptions & Dependencies

ID	Category	Rule Definition
A-01	Assumption/Dependency	The system is designed for a small or medium football-field venue and covers the approved classroom booking operations.

A-02	Assumption/Dependency	Online Customer payment uses the configured PayPal sandbox; Staff records walk-in cash payment.
A-03	Assumption/Dependency	In-app notifications cover operational events; email is additionally used for verification, password setup/reset, account status, and account-lock reason.
A-04	Assumption/Dependency	Deposit, payment-timeout, refund, and reminder policy values are maintained in the system_setting table.
A-05	Assumption/Dependency	Promotion conditions are kept practical and stored in promotion fields instead of a separate promotion condition table.
A-06	Assumption/Dependency	UC-62 is the optional configured Gemini question flow; UC-63 is the independent deterministic live-ranking and booking-handoff flow.

2. Limitations & Exclusions

ID	Category	Rule Definition
L-01	Limitation/Exclusion	Online payment uses the configured PayPal sandbox; production PayPal credentials and production settlement are outside the classroom deployment scope.
L-02	Limitation/Exclusion	UC-62 requires a configured Gemini API key for its external smoke test; UC-63 remains fully testable and authoritative without external AI.
L-03	Limitation/Exclusion	No complex inventory warehouse module for extra services. The system only checks service availability/capacity at booking level.
L-04	Limitation/Exclusion	No separate HR/payroll/accounting system integration is included.
L-05	Limitation/Exclusion	Use case diagram, screen flow, and UI mockups can be manually drawn in draw.io or Google Docs for better visual quality.

3. Business Rules

ID	Category	Rule Definition
BR-01	Account Uniqueness	Registered account email or phone number must be unique.
BR-02	Authentication	Private screens and functions require a valid authenticated session.
BR-03	Role Authorization	A user can access only screens and functions allowed for their role.
BR-04	Locked Customer	A locked Customer cannot log in or perform authenticated actions until an Admin unlocks the account.
BR-05	Field Active Status	Only active fields can be shown for new customer booking.
BR-06	Field Pricing Validity	Field price must match the selected field, day type, time range, and effective date.
BR-07	Past Slot Rule	Customers and staff must not create or reschedule bookings to

		<i>slots in the past.</i>
BR-08	<i>Booking Conflict</i>	<i>The system must not allow more than one active booking for the same field slot.</i>
BR-09	<i>Unavailable Slot Rule</i>	<i>The system must not allow booking on slots blocked for maintenance, weather, private event, internal use, or other operational reasons.</i>
BR-10	<i>Extra Service Availability</i>	<i>Limited extra services must not be selected beyond available inventory or staff capacity.</i>
BR-11	<i>Extra Service Update Window</i>	<i>Extra services can be added or updated only before booking check-in.</i>
BR-12	<i>Booking Confirmation</i>	<i>A pending online booking becomes Confirmed only after valid payment is captured or staff approval is recorded.</i>
BR-13	<i>Booking Rejection</i>	<i>Only pending booking requests can be rejected, and the rejection reason must be recorded.</i>
BR-14	<i>Check-in Status</i>	<i>Only Confirmed bookings can be checked in.</i>
BR-15	<i>Completion Status</i>	<i>Only Checked-in bookings can be completed.</i>
BR-16	<i>Cancellation Rule</i>	<i>Checked-in and Completed bookings cannot be cancelled through the normal cancellation flow.</i>
BR-17	<i>Cancellation Preview</i>	<i>Before cancellation, the system must show cancellation fee and refundable amount.</i>
BR-18	<i>Refund Calculation</i>	<i>Refund amount is calculated from booking time, cancellation time, paid amount, and cancellation/refund rules.</i>
BR-19	<i>No-show Handling</i>	<i>No-show is not treated as customer-initiated cancellation. The customer may lose the deposit according to policy.</i>
BR-20	<i>Membership Progress</i>	<i>Only Completed bookings count toward membership progress; Cancelled, Rejected, Expired, and No-show bookings do not count.</i>
BR-21	<i>Checkout Summary</i>	<i>Checkout summary must show field price, extra services, promotion discount, membership benefit, deposit amount, and payable total.</i>
BR-22	<i>Online Deposit</i>	<i>Online booking requires at least the configured deposit amount before the booking can be confirmed.</i>
BR-23	<i>Payment Transaction</i>	<i>Payment status must be updated from the sandbox result and stored with amount, method, transaction code, and payment time.</i>
BR-24	<i>Invoice Generation</i>	<i>An invoice must include field rental, extra services, discounts, paid amount, remaining amount, and refund amount if any.</i>
BR-25	<i>Promotion Validity</i>	<i>Promotion can be applied only when date range, usage limit, field/time condition, membership condition, and stacking rule are satisfied.</i>
BR-26	<i>Promotion Audit</i>	<i>Applied promotion details must be stored with the booking so invoice and reports remain auditable.</i>
BR-27	<i>Membership Rule Validity</i>	<i>Membership levels must have non-overlapping thresholds and valid benefit values.</i>
BR-28	<i>Notification Trigger</i>	<i>Booking, payment, cancellation, refund, and reminder notifications are created only after the related event is stored successfully.</i>
BR-29	<i>Issue Tracking</i>	<i>Each issue must store reporter, issue type/context, status, and</i>

		<i>resolution note when resolved.</i>
BR-30	Management Data Status	<i>Master data such as fields, prices, services, promotions, and membership levels should be disabled instead of physically deleted when already used by transactions.</i>
BR-31	Report Access	<i>Only Admin can view system reports, and report filters must use a valid date range.</i>
BR-32	Report Data Source	<i>Reports must aggregate from stored booking, payment, invoice, refund, promotion, and membership records.</i>

4. Booking Status Values

ID	Category	Rule Definition
BS-01	Pending	<i>Booking is created but not yet confirmed.</i>
BS-02	Confirmed	<i>Booking is confirmed after valid payment or staff approval.</i>
BS-03	Checked-in	<i>Customer has arrived and staff has checked in the booking.</i>
BS-04	Completed	<i>Customer has finished using the field and the booking is closed.</i>
BS-05	Cancelled	<i>Booking has been cancelled according to policy.</i>
BS-06	No-show	<i>Customer did not arrive for a confirmed booking.</i>
BS-07	Expired	<i>Booking or payment expired before confirmation.</i>
BS-08	Rejected	<i>Pending booking request was rejected by staff.</i>

5. Code-First Alignment Notes

ID	Category	Rule Definition
CF-01	Field catalogue	<i>UC-10 list response does not contain price; price is resolved in field detail/checkout.</i>
CF-02	Field detail	<i>Public detail rejects inactive fields. Reviews/ratings are not modeled; services shown are the global active catalogue.</i>
CF-03	Slot discovery	<i>UC-12 is the ordinary public date/type search. UC-62/63 add explainable ranking by preferred time, budget and eligible promotions.</i>
CF-04	Field booking	<i>Booking and reschedule reject blocked/reserved/past slots and slots owned by inactive fields.</i>
CF-05	Field pricing	<i>Active price bands honor day/time/effective dates and reject overlapping active rules; USD 12 remains the explicit fallback when no band matches.</i>
CF-06	Role boundary	<i>Customer online booking uses PayPal. Staff creates walk-in bookings and records cash. Admin has no checkout UI even where operator APIs permit Admin.</i>
CF-07	Reschedule	<i>The destination slot, current service inventory, promotion/membership discounts, balance/refund delta, invoice and notification are recalculated.</i>
CF-08	Payment failure	<i>A non-COMPLETED PayPal capture returns an error while preserving the local failed payment/provider status for history and diagnosis.</i>
CF-09	Refund accounting	<i>Only PayPal provider COMPLETED or explicit staff-recorded cash completion reconciles refund, invoice and booking financial snapshots.</i>
CF-10	Reporting	<i>Booked-value report metrics exclude rejected/expired</i>

		<i>bookings but currently include cancelled/no-show snapshots.</i>
<i>CF-11</i>	<i>Conflict handling</i>	<i>UC-36 composes calendar, issue, reschedule, cancel, refund and notification functions; there is no dedicated conflict entity.</i>
<i>CF-12</i>	<i>Assistant scope</i>	<i>UC-62/63 are deterministic and explainable; no external LLM, RAG knowledge base or fabricated result source is included.</i>